Linguaggi

UniVR - Dipartimento di Informatica

Fabio Irimie, Legion

2° Semestre 2025/2026

Indice

1 Linguaggi di programmazione **3**
- 1.1 Obiettivo dei linguaggi di programmazione . . . 3
- 1.2 Aspetti di progettazione . . . 4
  - 1.2.1 Tipi di linguaggi . . . 5
- 1.3 Implementare un linguaggio di programmazione . . . 6

2 Esecuzione dei linguaggi **6**
- 2.1 Notazione . . . 7
- 2.2 Interprete . . . 7
  - 2.2.1 Struttura dell'interprete . . . 7
- 2.3 Compilatore . . . 8
  - 2.3.1 Struttura del compilatore . . . 9
  - 2.3.2 Prima proiezione di Futamura . . . 10

3 Descrivere i linguaggi di programmazione **11**
- 3.1 Sintassi . . . 11
  - 3.1.1 Strumenti di definizione della sintassi . . . 12
  - 3.1.2 Definizione non formale . . . 13
- 3.2 Analisi semantica . . . 15
  - 3.2.1 Induzione strutturale . . . 15
  - 3.2.2 Semantica . . . 20
  - 3.2.3 Composizionalità della semantica . . . 22
  - 3.2.4 Equivalenza di programmi . . . 22
  - 3.2.5 Categorie sintattiche . . . 23
  - 3.2.6 Esempio di sintassi per il linguaggio "PL0" . . . 24

4 Implementazione di un linguaggio di programmazione IMP **26**
- 4.1 Espressioni . . . 26
  - 4.1.1 Aspetti implementativi . . . 26
  - 4.1.2 Sintassi . . . 29
  - 4.1.3 Semantica per espressioni aritmetiche . . . 30
  - 4.1.4 Semantica per espressioni booleane . . . 31
- 4.2 Identificatori . . . 32
  - 4.2.1 Aspetti implementativi . . . 32
  - 4.2.2 Legami, scope e occorrenze . . . 33
  - 4.2.3 Tipi di binding . . . 34
  - 4.2.4 Ambiente . . . 34
  - 4.2.5 Aggiungiamo gli identificatori alla sintassi delle espressioni . . . 35
  - 4.2.6 Semantica per espressioni con identificatori . . . 35
- 4.3 Tipo di dato . . . 37
  - 4.3.1 Ambiente statico . . . 37
  - 4.3.2 Semantica statica per le espressioni . . . 38
  - 4.3.3 Ambienti compatibili . . . 38
- 4.4 Dichiarazioni . . . 39
  - 4.4.1 Sintassi delle dichiarazioni . . . 39
  - 4.4.2 Aggiornamento di un ambiente . . . 41
  - 4.4.3 Identificatori in posizione libera e in posizione di definizione . . . 42
  - 4.4.4 Semantica statica per le dichiarazioni . . . 43

4.4.5 Semantica dinamica per le dichiarazioni 44
4.4.6 Elaborazione 45
4.5 Variabili 48
4.5.1 Locazione 48
4.5.2 Implementazione delle variabili nelle dichiarazioni 50
4.5.3 Semantica statica per le dichiarazioni con variabili 51
4.5.4 Aggiornamento della memoria 51
4.5.5 Semantica dimanica per le espressioni con identificatori variabili 52
4.5.6 Semantica dinamica per le dichiarazioni con variabili 53
4.6 Comandi 54
4.6.1 Sintassi dei comandi 55
4.6.2 Semantica statica dei comandi 55
4.6.3 Semantica dinamica dei comandi 56
4.6.4 Strutture di controllo 57
4.6.5 Semantica dell'if 59
4.6.6 Semantica della composizione 62
4.6.7 Semantica del while 63
4.7 Blocco di codice 67
4.7.1 Blocco inline 68
4.7.2 Blocchi annidati 68
4.7.3 Regola di visitbilità delle dichiarazioni 68
4.7.4 Scope 70
4.7.5 Tempo di vita 70
4.7.6 Semantica dei blocchi 72
4.8 Astrazione del controllo (Procedure) 74
4.8.1 Ambiente di riferimento di una procedura 76
4.8.2 Regole di scoping 76
4.8.3 Scoping statico 77
4.8.4 Scoping dinamico 81
4.9 Allocazione della memoria 84
4.9.1 Allocazione dinamica su pila 85
4.9.2 Procedure di chiamata a funzione 86
4.9.3 Implementazione scoping statico 89
4.9.4 Implementazione scoping dinamico 93
4.10 Implementazione delle procedure senza parametri 95
4.10.1 Dichiarazione di procedure 95
4.10.2 Chiamata di procedure 95
4.11 Implementazione delle procedure con passaggio di parametri 96
4.11.1 Direzione del passaggio dei parametri 96
4.11.2 Passaggio per valore 97
4.11.3 Passaggio per riferimento 98
4.12 Funzioni di ordine superiore 100
4.12.1 Regole di binding 100
4.13 Ricorsione 103
4.13.1 Ricorsione vs iterazione 104
4.13.2 Ricorsione in coda 105

1 Linguaggi di programmazione

Un linguaggio interpretato è un linguaggio di programmazione in cui le istruzioni vengono eseguite direttamente, senza la necessità di essere compilate in un formato eseguibile. Un interprete è rappresentato da un modello di esecuzione, cioè un **control flow graph** (CFG) che rappresenta il flusso di controllo del programma.

I programmi si possono interpretare in diversi modi:

- **Interpretazione diretta**: l'interprete esegue direttamente le istruzioni del programma e segue il flusso di controllo rappresentato dal CFG restituendo i risultati.
- **Interpretazione del segno**: l'interprete esegue le istruzioni del programma ma invece di restituire i risultati, restituisce un segno che rappresenta il risultato. Questo è utile quando non si riesce a restituire un risultato concreto, ad esempio quando un algoritmo non termina.
- **Analisi statica**: l'interprete analizza il programma senza eseguirlo, ad esempio per verificare la correttezza del codice o per ottimizzare le prestazioni.

1.1 Obiettivo dei linguaggi di programmazione

L'obiettivo è creare un modello formale che riesca ad eseguire i costrutti necessari per eseguire un programma. I componenti principali sono:

- **Dati**
- **Algoritmi**: Calcolo da eseguire sui dati, cioè calcola funzioni.

Definizione 1.1 (Algoritmo)**.** Un algoritmo è una sequenza di passi primitivi finita. L'algoritmo è descritto attraverso un programma in un linguaggio di programmazione.

Definizione 1.2 (Linguaggio di programmazione)**.** Un linguaggio di programmazione è uno strumento formale per descrivere algoritmi applicati ai dati.

Definizione 1.3 (Programma)**.** Il programma è la realizzazione nel linguaggio di programmazione di un algoritmo (concetto astratto). È una rappresentazione sintattica, cioè rispetta delle regole sintattiche del linguaggio di programmazione, finita di comportamenti potenzialmente infiniti. I comportamenti potenzialmente infiniti sono funzioni calcolate dall'algoritmo rappresentato (o implementato) dal programma:

$$f : \mathrm{Dom} \to \mathrm{Codom} \cup \{\uparrow\}$$

Il linguaggio di programmazione deve considerare strumenti per descrivere **metodi di calcolo**

Alcuni esempi di linguaggi sono:

- **Linguaggio matematico**: notazione rigorosa per rappresentare funzioni. Questo linguaggio non è adatto per descrivere algoritmi perchè non ha strumenti che descrivono **come** calcolare funzioni.
  - Paradigma funzionale: nascono dal linguaggio matematico per adattarlo alla descrizione di algoritmi. Derivano dal lambda calcolo e alcune implementazioni sono Haskell o Lisp.
- **Linguaggi logici**: notazione rigorosa con regole-assiomi che permettono di specificare un processo di calcolo attraverso regole di derivazione. Il calcolo è rappresentato da un processo di dimostrazione. Manca la descrizione di calcoli potenzialmente infiniti.
  - Paradigma logico: nascono dal linguaggio logico per adattarlo alla descrizione di algoritmi. Derivano dal calcolo dei predicati e un esempio di implementazione è Prolog.
- **Linguaggi di programmazione**: notazione rigorosa, tramite grammatiche CF, con:
  - **Regole**: descrivono la semantica delle istruzioni
  - **Istruzioni**: permettono di descrivere i passi di calcolo

Esempio 1.1. Alcuni esempi di applicazioni dei linguaggi di programmazione sono:

- Applicazioni scientifiche: Fortran, ecc...
- Applicazioni economiche: Cobol, ecc...
- Applicazioni AI: Lisp, ecc...
- Applicazioni sistemi: C, ecc...
- Applicazioni web: JavaScript, Java, ecc...

1.2 Aspetti di progettazione

Gli aspetti per scegliere un linguaggio di programmazione sono:

- **Readability** (Leggibilità): quanto è facile leggere e comprendere i programmi, cioè:
  - Sintassi chiara
  - Nessuna ambiguità
  - Facilità di analisi del codice

- **Writability** (Facilità di scrittura): quanto è facile o agevole scrivere programmi.
- **Reliability** (Affidabilità): quanto il linguaggio è conforme alle specifiche e quanto riduce l'introduzione di errori di programmazione.
- **Cost** (Costo):
  - Costi di efficienza
  - Costi di apprendimento
  - Costi di utilizzo in modo corretto

1.2.1 Tipi di linguaggi

I linguaggi di programmazione si possono classificare come:

- **Linguaggi di basso livello**: sono linguaggi vicini alla macchina fisica, come ad esempio l'assembly o il linguaggio macchina.
- **Linguaggi di alto livello**: sono linguaggi più vicini al linguaggio umano, come ad esempio:
  - Linguaggi funzionali
  - Linguaggi imperativi
  - Linguaggi logici

  I linguaggi si possono ulteriormente dividere in:
  - **Linguaggi impuri**: I linguaggi imperativi e alcuni linguaggi funzionali sono detti **funzionali impuri** perchè permettono di descrivere algoritmi con variabili e assegnamenti che possono ricevere degli "effetti collaterali" (side effects) cioè modifiche dello stato del programma. Quindi la variabile è un'astrazione della cella di memoria, di conseguenza lo stesso nome in punti diversi può avere valori diversi.
  - **Linguaggi puri**: I linguaggi logici e alcuni linguaggi funzionali sono detti **funzionali puri** perchè le variabili sono matematiche e quindi **immutabili**, cioè non possono essere modificate dopo essere state assegnate. In questo modo si evitano gli effetti collaterali.

  I linguaggi di alto livello si possono dividere in base al paradigma di programmazione:
  - **Object Oriented**: linguaggi che permettono di strutturare il codice in oggetti con attributi e metodi.
  - **Procedurali**: linguaggi che permettono di strutturare il codice in procedure o funzioni.
  - **Strutturati**: linguaggi che permettono di strutturare il codice in blocchi di istruzioni con costrutti di controllo come if, while, for, ecc...
  - **Dichiarativi**: linguaggi che si basano sulla descrizione del problema piuttosto che sulla descrizione del processo di calcolo.

1.3 Implementare un linguaggio di programmazione

Le caratteristiche che hanno effetto sulla progettazione di un linguaggio di programmazione sono:

- L'architettura della macchina su cui è implementato
- Le metodologie di progettazione di programmi, cioè i paradigmi di programmazione

2 Esecuzione dei linguaggi

L'obiettivo dei linguaggi è **eseguire i programmi**. Per poter eseguire un programma serve un altro programma che lo esegue che è detto **macchina astratta** per il linguaggio di programmazione.

Definizione 2.1 (Macchina astratta)**.** Dato un linguaggio di programmazione L, la macchina astratta per L M_L è un insieme di **strutture dati** e **algoritmi** che permettono di memorizzare e eseguire programmi scritti in L.

Intuitivamente è la macchina il cui linguaggio è esattamente L.

- Per ogni macchina astratta M_L esiste un unico linguaggio L che è il linguaggio riconosciuto ed eseguito da M_L
- Per ogni linguaggio di programmazione L possono esistere infinite macchine astratte per L

Esempio 2.1. La macchina fisica è la **macchina astratta** per il linguaggio macchina. Mentre tutte le macchine astratte per qualsiasi altro linguaggio di programmazione di alto livello sono software, quindi altri programmi che traducono le istruzioni del linguaggio di programmazione in un'istruzione in un livello più basso.

Definizione 2.2 (Linguaggio macchina)**.** Data una macchina M, il linguaggio macchina di M L_M è il linguaggio che ha come stringhe quelle direttamente interpretabili da M.

Se si ha una pila di macchine astratte $\{M_{i-1}, M_i, M_{i+1}, M_{i+2}\}$ la macchina M_i ha il suo linguaggio macchina L_i che utilizza il linguaggio macchina L_{i-1} della macchina M_{i-1} per eseguire i programmi scritti in L_i. Inoltre il linguaggio L_i può essere usato ai livelli superiori.

Implementare L equivale a realizzare la macchina astratta M_L. Per realizzare M_L ci sono due modi:

1. **Compilazione**: Tradurre L in un linguaggio macchina sottostante

2. **Interpretazione**: Eseguire L sulla macchina sottostante

Definizione 2.3 (Compilazione)**.** La compilazione è una traduzione esplicita di un linguaggio L in un linguaggio L′.

Definizione 2.4 (Interpretazione)**.** L'interpretazione è una traduzione implicita di un linguaggio L che viene **simulato** in L′.

2.1 Notazione

La notazione che verrà utilizzata per descrivere i programmi è la seguente:

- $\mathrm{Prog}^{\mathrm{L}}$: Insieme dei programmi scritti in L
- D: Insieme dei dati manipolati dai programmi in L
- $\mathrm{P}^{\mathrm{L}} \in \mathrm{Prog}^{\mathrm{L}} \quad \mathrm{P}^{\mathrm{L}} : \mathrm{D} \to \mathrm{D}$:

$$\forall \mathrm{input} \in \mathrm{D}.\ \mathrm{P}^{\mathrm{L}}(\mathrm{input}) = \mathrm{output} \in \mathrm{D}$$

  cioè il programma P^{L} è una funzione che prende in input un dato e restituisce un dato in output.

2.2 Interprete

L'interprete è la realizzazione della macchina astratta con soluzione interpretativa (2).

Definizione 2.5 (Interprete)**.** Un interprete per il linguaggio L (scritto nel linguaggio L_0) è un programma $\mathrm{I}^{\mathrm{L}_0\mathrm{L}} : (\mathrm{Prog}^{\mathrm{L}} \times \mathrm{D}) \to \mathrm{D}$ che prende in input un programma P^{L} e un dato e restituisce in output il risultato dell'esecuzione di P^{L} su input. Questa è la realizzazione della **macchina di Turing universale**.

$$\underbrace{\mathrm{I}^{\mathrm{L}_0\mathrm{L}}(\mathrm{P}^{\mathrm{L}}, \mathrm{input}) = \mathrm{output} \quad \text{t.c.} \quad \mathrm{P}^{\mathrm{L}}(\mathrm{input}) = \mathrm{output}}$$

$$\mathrm{I}^{\mathrm{L}_0\mathrm{L}}(\mathrm{P}^{\mathrm{L}}, \mathrm{input}) = \mathrm{P}^{\mathrm{L}}(\mathrm{input}) \Rightarrow \text{Macchina universale}$$

2.2.1 Struttura dell'interprete

La struttura dell'interprete è data dal seguente pseudocodice:

```
1 begin
2   go = true;
3   while go do
4     fetch(opcode, opinfo) at PC; // Program counter
5     decode(opcode, opinfo)
6
7     if opcode needs arguments then
```

```
      fetch(args)

    case opcode
      opcode1: execute(opcode1, args)
      opcode2: execute(opcode2, args)
      ...
      halt: go = false

    if opcode has result then store(result)

    PC = PC + sizeof(opcode)

end
```

Esempio 2.2. Un semplice autointerprete, cioè un interprete scritto nel linguaggio che interpreta.

```
input P, d;
pc = 2;
store = [in -> d, out -> 0, x_1 -> 0, x_2 -> 0, ...];
while pc < length(P) do
  # Find the pc-th instruction
  instruction = lookup(P, pc);

  # Syntax rules
  case instruction of
    skip: pc = pc + 1;
    x = e: store = store[x -> eval(e, store)]; pc = pc + 1;
    ...
  endcase
endwhile;

output store[out]


# Semantic rules
eval(e, store) = case e of
  constant: e
  variable: store(e)
  e1 + e2: eval(e1, store) + eval(e2, store)
  e1 - e2: eval(e1, store) - eval(e2, store)
  e1 * e2: eval(e1, store) * eval(e2, store)
  ...
endcase
```

Dove `in -> d` rappresenta l'assegnamento del dato di input alla variabile `in`.

L'interprete **realizza** la macchina astratta M_L di un linguaggio di programmazione L partendo da un linguaggio L_0 con una macchina astratta M_0 direttamente eseguibile passando attraverso una macchina intermedia M_I (interprete).

2.3 Compilatore

Definizione 2.6. Il compilatore da L a L_0 è un programma del tipo:

$$C^{LL_0} : \text{Prog}^L \to \text{Prog}^{L_0}$$

che prende in input un programma P^L e restituisce in output un programma P^{L_0}.

$$\forall P^L \in Prog^L \; . \; C^{LL_0}(P^L) = P^{L_0} \quad \text{t.c.} \quad \forall d \in D \; . \; P^{L_0}(d) = P^L(d)$$

Non ci sono accessi alla memoria o a dati. Verificare che due programmi siano uguali è un problema non decidibile, quindi il compilatore deve essere costruito in modo da garantire che i programmi compilati siano equivalenti a quelli originali.

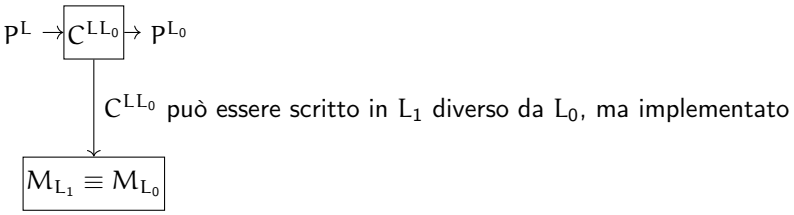


Figura 1: Rappresentazione grafica del compilatore

- Per eseguire il programma bisogna applicare P^{L_0} all'input.
- Il compilatore traduce un codice in un codice diverso, separando la traduzione dall'esecuzione.

2.3.1 Struttura del compilatore

La struttura di un compilatore è data da diversi componenti che lavorano insieme. I componenti principali sono:

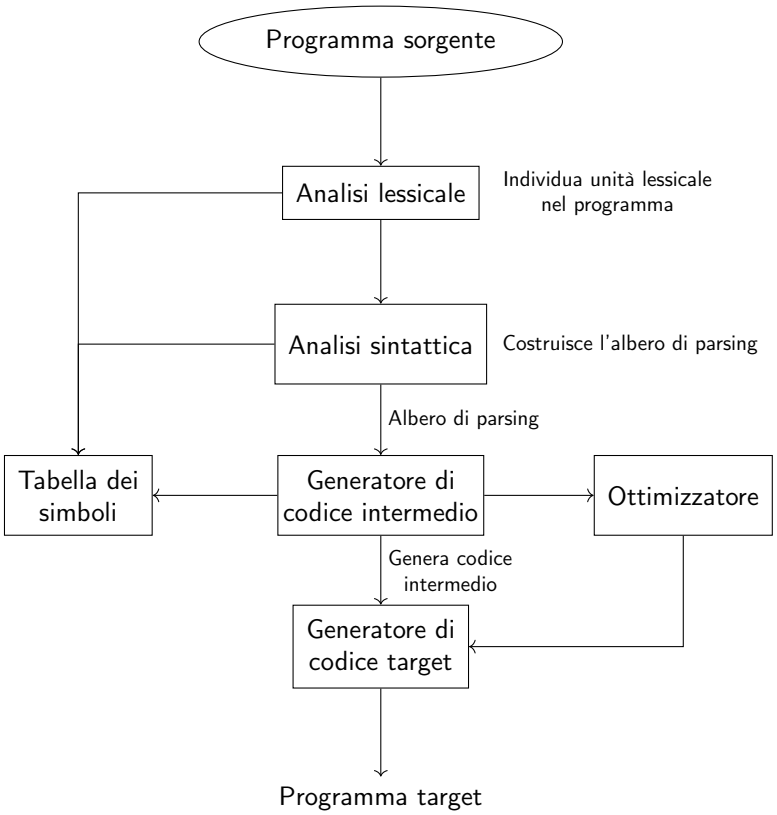


Figura 2: Struttura di un compilatore

Il compilatore **traduce** un programma P^L scritto in un linguaggio di programmazione L in un programma P^{L_0} scritto in un linguaggio L_0 con una macchina astratta M_0 direttamente eseguibile, passando attraverso una macchina intermedia M_I (compilatore).

2.3.2 Prima proiezione di Futamura

Un compilatore può essere ottenuto **specializzando** un interprete su un programma.

Definizione 2.7 (Specializzatore)**.** Uno specializzatore per un linguaggio di programmazione L è un programma che realizza la funzione:

$$\mathrm{Spec}_L : (\mathrm{Prog}^L \times D) \to \mathrm{Prog}^L$$

tale che:

$$\forall P^L \in \mathrm{Prog}^L, \forall d \in D\ .$$
$$\mathrm{Spec}_L(P^L, d) = P^L_d$$
$$\text{t.c. } P^L(d, \mathrm{input}) = P^L_d(\mathrm{input})$$

Quindi d non è più un dato di input, ma è incorporato nel codice.

Lo specializzatore fornisce un legame tra interprete e compilatore. Infatti specializzando un interprete (scritto in L) si ottiene un compilatore da L a L_0.

$$I(P^L, d) \Rightarrow Spec(I, P^L)$$

$Spec$ è un programma scritto in L_0 che integra nel codice l'esecuzione di P^L.

3 Descrivere i linguaggi di programmazione

I linguaggi di programmazione sono dei linguaggi, quindi hanno caratteristiche in comune con i linguaggi e quindi anche quelli naturali. Le caratteristiche sono:

- **Sintassi**: Regole di formazione. Determinano quali sono le frasi ben formate nel linguaggio e descrive relazioni tra simboli
- **Semantica**: Attribuzione del **significato** ai simboli. Determina il significato delle frasi ben formate nel linguaggio e descrive relazioni tra simboli e significati.
- **Pragmatica**: Regole di buon utilizzo del linguaggio.
- **Implementazione** (Caratteristica aggiunta per i linguaggi di programmazione): Esistono strumenti che eseguono frasi ben formate del linguaggio.

Esempio 3.1. Il numero è soltanto sintassi, mentre il significato è il valore che assume:

$$4 \quad \text{(sintassi)} \rightarrow \text{significato} \quad 4$$

$$IV \quad \text{(sintassi)} \rightarrow \text{significato} \quad 4$$

3.1 Sintassi

Le componenti principali della sintassi sono:

- **Parola**: stringa su un alfabeto Σ
- **Frase**: sequenza di parole ben formata, cioè che rispetta le regole della sintassi
- **Linguaggio**: insieme di frasi ben formate

  Esempio 3.2. Prendiamo ad esempio il seguente linguaggio:

  $$\{a^n b^n \mid n \in \mathbb{N}\}$$

  - **Alfabeto**: $\Sigma = \{a, b\}$
  - **Parole**: $\varepsilon, a, b, aa, ab, ba, bb, aaa, aab, \ldots$
  - **Linguaggio**: $\{a^n b^n \mid n \in \mathbb{N}\}$

- **Lessema**: una parola (costituita da soli terminali nell'alfabeto) che ha un significato, cioè è una parola che rappresenta un concetto o un oggetto nel linguaggio.

Esempio 3.3. Alcuni esempi di lessemi sono:

- `if exp then com else com`:
  I lessemi sono `if`, `then`, `else`, mentre `exp` e `com` non sono lessemi perchè rappresentano un'espressione e un comando, cioè sono dei simboli non terminali.

- **Token**: categorie sintattiche, cioè simboli non terminali

Esempio 3.4. Alcuni esempi di token sono:

`if exp then com else com`:
I token sono `exp` e `com` perchè sono simboli non terminali.

3.1.1 Strumenti di definizione della sintassi

Gli strumenti per definire la sintassi di un linguaggio di programmazione sono:

- **Riconoscitore**: è uno strumento per il **riconoscimento** di stringhe appartenenti al linguaggio. Ad esempio gli automi a stati finiti, utilizzati nell'analisi lessicale per il riconoscimento dei lessemi.
- **Generatore**: è uno strumento per la **generazione** di stringhe appartenenti al linguaggio. Ad esempio le grammatiche CF, utilizzate nella fase di parsing della compilazione per riconoscere programmi scritti in un linguaggio di programmazione.

Definizione utile 3.1. Un linguaggio di programmazione è dipendente dal contesto, quindi si cerca di svilupparli "a livelli", cioè si fa prima tutto ciò che si può fare con gli automi, poi tutto ciò che si può fare con le grammatiche CF, e così via.

Definizione 3.1. Una grammatica context free è definita da:

$$G = \langle V, T, P, S \rangle$$

dove:

- V: è un insieme finito di simboli non terminali (variabili)
- T: è un insieme finito di simboli terminali

$$V \cap T = \varnothing$$

- P: è un insieme finito di procedure:

$$A \to \alpha \quad A \in V, \alpha \in (V \cup T)^*$$

- $S \in V$: è il simbolo iniziale

In un linguaggio di programmazione la grammatica CF è mappata nel seguente modo:

- V: categorie sintattiche (ad esempio le espressioni, le variabili, ecc...)
- T: vocabolario di parole (`if`, `then`, `else`, ecc...)
- P: regole di formazione
- $S \in V$: categoria della frase (il programma)

Definizione utile 3.2 (Notazione BNF)**.** Per i linguaggi di programmazione viene utilizzata la notazione BNF (Backus-Naur Form) che cambia i simboli utilizzati per rappresentare i componenti della grammatica CF, ad esempio:

$$\rightarrow \quad \text{diventa} \quad ::=$$

Un linguaggio BNF è CF.

Esempio 3.5. Costruiamo una grammatica per espressioni booleane:

$$G = \langle \{\mathrm{Exp}\}, \{0, 1, \mathrm{or}, \mathrm{and}, \mathrm{not}, (,)\}, P, \mathrm{Exp} \rangle$$

$$P = \begin{cases} \mathrm{Exp} \rightarrow 0 \\ \mathrm{Exp} \rightarrow 1 \\ \mathrm{Exp} \rightarrow (\mathrm{Exp}\ \mathrm{or}\ \mathrm{Exp}) \\ \mathrm{Exp} \rightarrow (\mathrm{Exp}\ \mathrm{and}\ \mathrm{Exp}) \\ \mathrm{Exp} \rightarrow (\mathrm{not}\ \mathrm{Exp}) \end{cases}$$

$$= 0|1|(\mathrm{Exp}\ \mathrm{or}\ \mathrm{Exp})|(\mathrm{Exp}\ \mathrm{and}\ \mathrm{Exp})|(\mathrm{not}\ \mathrm{Exp})$$

3.1.2 Definizione non formale

Un linguaggio si può definire anche in un modo non formale, definendo i costrutti base:

- Niente dichiarazioni
- Solo variabili ed espressioni booleane
- Istruzioni:
  - Assegnamento
  - Composizione sequenziale
  - Comando iterativo (ciclo)

– Comando condizionale (if)

Il linguaggio è definito su livelli, partendo dal basso verso l'alto:

1. Notazione della grammatica
2. Linguaggio regolare per riconoscere gli identificatori e le parole riservate al linguaggio di programmazione
3. Espressioni booleane che parte dai simboli definiti ai livelli sottostanti

$$B \rightarrow true \mid false \mid (B \text{ or } B) \mid (B \text{ and } B) \mid (\text{not } B) \mid x$$

4. Comandi atomici, ad esempio l'assegnamento. Utilizza gli identificatori, un simbolo di assegnamento e un espressione (booleana in questo caso).

$$A \rightarrow x := B$$

5. Comandi, ad esempio l'if o il ciclo. Utilizzano i comandi atomici e le espressioni booleane.

$$C \rightarrow A \mid C; C \mid while\ B\ do\ C \mid if\ B\ then C\ else\ C$$

6. Programmi, cioè sequenze di comandi.

$$P \rightarrow \{C\}$$

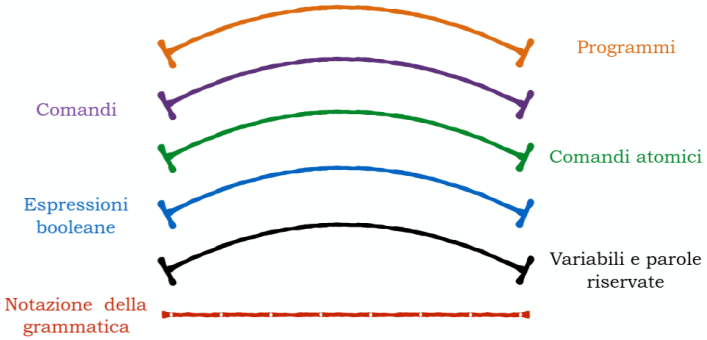


Figura 3: Rappresentazione grafica dei livelli di un linguaggio di programmazione

Si utilizzano espressioni parentesizzate per evitare ambiguità. Se non si utilizzassero le parentesi si potrebbe scegliere di eseguire prima un'operatore invece di un altro e questo porterebbe a risultati diversi dalla stessa espressione.

3.2 Analisi semantica

L'analisi semantica verifica i **vincoli sintattici**, ad esempio che il numero di parametri formali sia uguale al numero di parametri attuali, oppure che l'uso di una variabile sia possibile solo dopo che sia stata dichiarata. Questi sono vincoli sul codice, quindi **non catturati dalla grammatica** e si dicono **contestuali**.

- Sintassi $\rightarrow$ Grammatica CF
- Semantica $\rightarrow$ Tutti i processi di verifica ed esclusione che non sono descritti dalla sintassi

3.2.1 Induzione strutturale

L'induzione strutturale permette di dimostrare proprietà di oggetti descritti in modo induttivo o ricorsivo. Una grammatica descrive in modo induttivo il linguaggio (insieme di stringhe) che genera, quindi si può utilizzare l'induzione strutturale per dimostrare proprietà delle stringhe generate.

Definizione 3.2. Per dimostrare una proprietà Π su tutti gli elementi di un insieme definito in modo strutturale da una grammatica bisogna:

1. (Caso base): Dimostrare la proprietà su tutti gli elementi base, cioè quelli che sono esplicitamente inseriti nell'insieme di stringhe generate dalla grammatica.
2. (Passo induttivo): Dimostrare la proprietà sugli elementi composti sotto **l'ipotesi induttiva** che la proprietà valga per gli elementi che la compongono

Esempio 3.6. Definiamo l'insieme $X \subseteq \mathbb{N}$:

$$\begin{cases} \text{Base} & : 0 \in X \\ \text{Passo induttivo} & : \text{se } n \in X \text{ allora } n+3 \in X \end{cases}$$

La proprietà è $X = 3\mathbb{N}$

La proprietà dimostrabile in modo strutturale è: $X \subseteq 3\mathbb{N}$:

1. **Caso base**: 0 è la base di X e $0 \in 3\mathbb{N}$
2. **Passo induttivo**: dimostriamo $\forall n.\ n \in X \Rightarrow n \in 3\mathbb{N}$.
   Prendiamo $n \in X$, l'ipotesi induttiva è la seguente:

$$n \in X \Rightarrow n \in 3\mathbb{N}$$

bisogna dimostrare che $n + 3 \in 3\mathbb{N}$. Quindi sappiamo che

$$\begin{aligned} n \in 3\mathbb{N} &\Rightarrow \exists i.\ n = 3i \\ &\Rightarrow n + 3 = 3i + 3 = 3(i + 1) \in 3\mathbb{N} \end{aligned}$$

Esempio 3.7. Definiamo induttivamente gli alberi binari in cui i nodi sono numeri $n \in \mathbb{N}$. Ogni numero è un albero binario, perchè è un albero con un solo nodo:

$$n \in BT \quad \text{(Base)}$$

Consideriamo due alberi binari $T_1 \in BT$ e $T_2 \in BT$ allora:

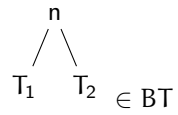


Figura 4: Albero binario composto da due alberi binari

Quindi:

$$br(T_1, T_2) \in BT$$

dove $br()$ è l'operatore che aggiunge un nodo una radice comune a T_1 e T_2

IMPORTANTE: Dimostriamo che la funzione che calcola il numero di foglie (`#foglie(T)`) e la funzione che restituisce il numero di nodi che hanno figli (`#branch(T)`) sono uguali a:

$$\forall T \in BT.\ \texttt{\#foglie}(T) = \texttt{\#branch}(T) + 1$$

Definiamo la funzione `#foglie(T)`:

$$\begin{cases} \#foglie(n) = 1 \\ \#foglie(br(T_1, T_2)) = \#foglie(T_1) + \#foglie(T_2) \end{cases}$$

Definiamo la funzione `#branch(T)`:

$$\begin{cases} \#branch(n) = 0 \\ \#branch(br(T_1, T_2)) = \#branch(T_1) + \#branch(T_2) + 1 \end{cases}$$

Dimostrazione per induzione sulla costruzione degli elementi di BT:

1. **Caso Base**: è il nodo che non ha figli in BT:

$$T = n \in BT$$

Sappiamo che:

$$\#foglie(T) = \#foglie(n) = 1 = 1 + 0 = 1 + \#branch(T)$$

e questo soddisfa la proprietà.

2. **Passo induttivo**: $T = br(T_1, T_2)$. Ipotesi induttiva: supponiamo che T_1 e T_2 soddisfino la proprietà, quindi:

$$\#foglie(T_1) = \#branch(T_1) + 1$$
$$\#foglie(T_2) = \#branch(T_2) + 1$$

Dobbiamo dimostrare che $\#foglie(T) = \#branch(T) + 1$:

$$\begin{aligned}
\#foglie(T) &\overset{def\ T}{=} \#foglie(br(T_1, T_2)) \\
&\overset{def\ \#foglie}{=} \#foglie(T_1) + \#foglie(T_2) \\
&\overset{Hp\ Ind}{=} \underbrace{\#branch(T_1) + 1 + \#branch(T_2)}_{def\ \#branch(br(T_1,T_2))} + 1 \\
&= \#branch(\underbrace{br(T_1, T_2)}_{T}) + 1 \\
&= \#branch(T) + 1
\end{aligned}$$

Quindi anche T soddisfa la proprietà.

Esempio 3.8. Consideriamo la seguente grammatica:

$$S \to aa \mid bb \mid aSa \mid bSb$$

questa grammatica genera tutte le stringhe palindrome di lunghezza pari. Siccome aa e bb sono nel linguaggio, se si prende una qualsiasi stringa nel linguaggio S e si aggiungono a o b all'inizio e alla fine, si ottiene una stringa ancora palindroma, quindi anche questa stringa è nel linguaggio. Si può quindi definire questa grammatica in modo induttivo:

$$LS : \begin{cases} aa, bb, \in LS \quad \text{(Base)} \\ \sigma \in LS \Rightarrow a\sigma a, b\sigma b \in LS \quad \text{(Passo induttivo)} \end{cases}$$

Quindi $L(S) = LS$ e la proprietà da dimostrare è:

$$\sigma \in LS \Rightarrow \exists x, y.\ \sigma = xy \wedge x = reverse(y)$$

1. **Caso base**: Prendiamo:

$$aa = \sigma \Rightarrow x = a, y = a \quad \text{e} \quad x = a = reverse(a) = reverse(y)$$

analogo per $\sigma = bb$.

2. **Passo induttivo**: Prendiamo $\sigma \in LS$, l'ipotesi induttiva è la seguente:

$$\sigma = xy \text{ t.c. } x = reverse(y)$$

Prendiamo $\sigma' = a\sigma a$. Per ipotesi induttiva:

$$\sigma' = a\sigma a = a\overbrace{xy}^{\sigma}a = x'y'$$

con $x' = ax$ e $y' = ya$. Quindi:

$$\begin{aligned} x' = ax &= areverse(y) \\ &= reverse(\underbrace{ya}_{y'}) \\ &= reverse(y') \end{aligned}$$

Abbiamo dimostrato che σ' è palindroma, quindi $\sigma' \in LS$.

Esercizio 3.1. Consideriamo la seguente grammatica:

$$E \to n \mid E + E \mid E * E$$

1. Definire strutturalmente l'insieme Exp
2. Si considerino le seguenti funzioni:
   - Conta il numero di operatori:
     $op(E) : op(n) = 0 \quad op(E_1 + E_2) = op(E_1 * E_2) = 1 + op(E_1) + op(e_2)$
   - Conta il numero di valori:
     $val(E) : val(n) = 1 \quad val(E_1 + E_2) = val(E_1 * E_2) = val(E_1) + val(E_2)$

   Dimostrare che:
   $$\forall E \,.\, val(E) = op(E) + 1$$

Definizione strutturale dell'insieme Exp:

- **Base**: $n \in L(E)$
- **Passo induttivo**: se $e_1, e_2 \in Exp$ allora $e_1 + e_2, e_1 * e_2 \in Exp$. Le espressioni sono il minimo insieme chiuso che soddisfa queste condizioni, quindi non ci sono altre espressioni in $L(E)$. Quindi ad esempio:

$$3 + 5 * 2 \in Exp$$

Sappiamo che:

$$\begin{cases} val(n) = 1 \\ val(e_1 + e_2) = val(e_1 * e_2) = val(e_1) + val(e_2) \end{cases}$$

$$\begin{cases} op(n) = 0 \\ op(e_1 + e_2) = op(e_1 * e_2) = op(e_1) + op(e_2) + 1 \end{cases}$$

Quindi sull'esempio precedente:

$$\begin{aligned} val(\underbrace{3+5}_{e_1} * \underbrace{2}_{e_2}) &= val(3+5) + val(2) \\ &= val(3) + val(5) + val(2) \\ &= 1 + 1 + 1 = 3 \end{aligned}$$

$$\begin{aligned} op(\underbrace{3+5}_{e_1} * \underbrace{2}_{e_2}) &= op(3+5) + op(2) + 1 \\ &= op(3) + op(5) + op(2) + 1 + 1 \\ &= 0 + 0 + 0 + 1 + 1 = 2 \end{aligned}$$

Dimostriamo che $\Pi : \forall e \in Exp \; . \; val(e) = op(e) + 1$ per induzione strutturale su Exp

- **Caso base**: Verifichiamo Π sugli elementi esplicitamente inseriti nell'insieme Exp, cioè sui numeri $n \in \mathbb{N}$: consideriamo le espressioni $e = n$

$$val(e) = val(n) = 1 \quad = \quad 1 + 0 = 1 + op(n) = 1 + op(e)$$

- **Ipotesi induttiva**: Dimostriamo per gli elementi composti supponendo l'ipotesi induttiva sugli elementi che li compongono. Sia $e = e_1 + e_2$ (ipotesi1) (analogo per $e = e_1 * e_2$), l'ipotesi induttiva è la seguente:

$$val(e_1) = op(e_1) + 1 \quad \wedge \quad val(e_2) = op(e_2) + 1$$

  Dobbiamo dimostrare che $val(e) = op(e) + 1$:

$$\begin{aligned} val(e) &\overset{ipotesi1}{=} val(e_1 + e_2) \\ &\overset{def\,val}{=} val(e_1) + val(e_2) \\ &\overset{Hp.\,Ind.}{=} \underbrace{op(e_1) + 1 + op(e_2)}_{op(e_1+e_2)} + 1 \\ &\overset{def\,op}{=} op(e_1 + e_2) + 1 \\ &\overset{ipotesi1}{=} op(e) + 1 \end{aligned}$$

Abbiamo dimostrato che Π è vera per tutti gli elementi di Exp.

3.2.2 Semantica

La semantica è l'attribuzione di significato ai simboli. Il significato è un'entità autonoma che esiste indipendentemente dai simboli usati per descriverlo. Esistono 3 tipi di semantica:

- **Denotazionale**: È un modello completamente matematico che descrive il programma come una funzione su stati (programmi come funzioni). Lo **stato** descrive il valore di tutte le variabili in un certo momento.

  $$\mathrm{Stato} : \mathrm{Var} \mapsto \mathrm{Val}$$

  La semantica denotazionale è una funzione che prende un programma e restituisce una funzione che prende uno stato iniziale e restituisce uno stato finale:

  $$\mathrm{E} : \mathrm{Prog} \mapsto \underbrace{(\mathrm{Stato} \to \mathrm{Stato})}_{\substack{\text{Denotazione associata} \\ \text{al programma}}}$$

  Un esempio di semantica denotazionale è il comportamento input/output

- **Operazionale**: Descrive il significato del programma come sequenza di trasformazione di stati, **eseguendo** i suoi comandi sulla macchina astratta. Per semplificare vedremo lo stato come se fosse la memoria:

  $$\text{Memoria: } \sigma : \mathrm{Var}^n \mapsto \mathrm{Val}^n$$

  $$\text{Stato: } (\mathrm{Prog} \times \mathrm{Mem}) \cup \mathrm{Mem}$$

  dove:

  - Prog è il programma che stiamo eseguendo
  - Mem è la memoria su cui stiamo eseguendo il programma
  - $\cup\mathrm{Mem}$ è l'insieme degli stati finali

  Ad esempio:

  $$\text{Variabili: } < x, y, z > \mapsto < 2, 5, 3 > \rightsquigarrow < x \mapsto 2, y \mapsto 5, z \mapsto 3 >$$

  Per eseguire un programma si utilizza un **sistema di transizione**:

  $$< \Sigma, \to >$$

  dove:

  - Σ è l'insieme degli stati
  - $\to$ è la relazione di transizione che descrive come si passa da uno stato ad un altro eseguendo un comando del programma

- **Assiomatica**: Descrive il programma attraverso le proprietà che soddisfa. Si calcola utilizzando un sistema di derivazione che fornisce regole per dimostrare proprietà sui programmi:

Notazione: {P} statement {Q}. Questa semantica si calcola mediante un sistema di prova: abbiamo una tripla da verificare e cerco di dimostrarla applicando le regole.

$$\frac{\{P\}\,S\,\{Q\}, P' \Rightarrow P, Q \Rightarrow Q'}{\{P'\}\,S\,\{Q'\}}$$

$$\frac{\{P1\}\,S1\,\{P2\}, \{P2\}\,S2\,\{P3\}}{\{P1\}\,S1; S2\,\{P3\}}$$

$$\frac{\{B \text{ and } P\}\,S1\,\{Q\}, \{(\text{not } B) \text{ and } P\}\,S2\,\{Q\}}{\{P\}\ \texttt{if}\ B\ \texttt{then}\ S1\ \texttt{else}\ S2\ \{Q\}}$$

$$\frac{(I \text{ and } B)\,S\,\{I\}}{\{I\}\,\text{while } B \text{ do } S\,\{I \text{ and } (\text{not } B)\}}$$

Figura 5: Esempio di sistema di derivazione per la semantica assiomatica

Quindi descrive relazioni tra proprietà di stati. Ad esempio fornendo le proprietà invarianti dei programmi, cioè le proprietà che sono vere prima e dopo l'esecuzione di un programma.

Esempio 3.9. Consideriamo il seguente programma:

$$P = \begin{cases} z := 2; \\ y := z; \\ y := y + 1; \\ z := y; \end{cases}$$

Applichiamo i tre tipi diversi di semantica al programma P:

- **Semantica denotazionale**:

$$\begin{aligned} &\mathcal{D}[\overbrace{P}^{\text{Prog}}](\overbrace{z \mapsto \bot, y \mapsto \bot}^{\text{Stato iniziale}}) = \\ &= \mathcal{D}[z := 2; y := z; y := y + 1; z := y](z \mapsto \bot, y \mapsto \bot) \\ &= \mathcal{D}[z := y] \circ \mathcal{D}[y := y + 1] \circ \mathcal{D}[y := z] \circ \mathcal{D}[z := 2](z \mapsto \bot, y \mapsto \bot) \\ &= \mathcal{D}[z := y] \circ \mathcal{D}[y := y + 1] \circ \mathcal{D}[y := z](z \mapsto 2, y \mapsto \bot) \\ &= \mathcal{D}[z := y] \circ \mathcal{D}[y := y + 1](z \mapsto 2, y \mapsto 2) \\ &= \mathcal{D}[z := y](z \mapsto 2, y \mapsto 3) \\ &= (z \mapsto 3, y \mapsto 3) \end{aligned}$$

Quindi $\mathcal{D}[P](z \mapsto \bot, y \mapsto \bot) = (z \mapsto 3, y \mapsto 3)$ è usata per:

- Dimostrare proprietà (correttezza)
- Fornisce uno strumento formale per ragionare sulle funzionalità dei programmi
- Non è utile a chi usa il linguaggio di programmazione

- **Semantica assiomatica**: Fornisce proprietà relazionali sull'input e sull'output del programma, ad esempio:

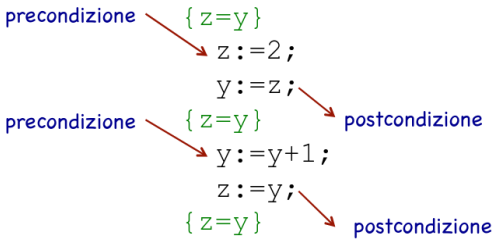


Figura 6: Esempio di proprietà assiomatica per il programma P

- **Semantica operazionale**:

$$
\begin{aligned}
&< P, (z \mapsto \bot, y \mapsto \bot) >= \\
&=< z := 2; y := z; y := y + 1; z := y, (z \mapsto \bot, y \mapsto \bot) > \\
&=< y := z; y := y + 1; z := y, (z \mapsto 2, y \mapsto \bot) > \\
&=< y := y + 1; z := y, (z \mapsto 2, y \mapsto 2) > \\
&=< z := y, (z \mapsto 2, y \mapsto 3) > \\
&=< z \mapsto 3, y \mapsto 3 >
\end{aligned}
$$

3.2.3 Composizionalità della semantica

Il significato di ogni programma è funzione del significato dei suoi componenti immediati, cioè si possono ottenere le proprietà semantiche di un programma a partire dalle proprietà semantiche dei suoi componenti.

3.2.4 Equivalenza di programmi

La semantica viene utilizzata per verificare l'equivalenza tra programmi.

- $P_1 = P_2$ dice che P_1 è esattamente lo stesso programma di P_2, cioè sono sintatticamente uguali. Questa si dice **uguaglianza sintattica**.
- $P_1 \equiv P_2$ dice che P_1 e P_2 hanno lo stesso comportamento (stesso significato), anche se sono scritti in modo diverso. Questa si dice **equivalenza** o uguaglianza semantica.

Esempio 3.10. Consideriamo i seguenti programmi:

- P_1 insertion sort
- P_2 bubblue sort

Sappiamo che i due programmi sono diversi, quindi:

$$P_1 \neq P_2$$

Ma sappiamo che entrambi ordinano una lista, quindi il loro comportamento è lo stesso:

- Con la semantica denotazionale si ha:

$$\mathcal{D}[P_1] = \mathcal{D}[P_2] \rightsquigarrow P_1 \equiv P_2$$

- Con la semantica operazionale si ha:

$$[\![P_1]\!] \neq [\![P_2]\!] \rightsquigarrow P_1 \not\equiv P_2$$

Quindi l'equivalenza dipende dalla scelta della semantica

3.2.5 Categorie sintattiche

Le categorie sintattiche sono insiemi di stringhe che condividono una certa proprietà per:

- Manipolare dati (espressioni)
- Trasformare stati (comandi)
- Creare e manipolare i legami (dichiarazioni), ad esempio variabile-valore, variabile-tipo, funzione-codice, ecc...

Si hanno quindi:

- **Stato**: descrive gli effetti sugli identificatori
- **Ambiente**: è un insieme di legami tra identificatori e oggetti denotati

Le categorie sintattiche principali sono:

- **Espressioni**: denotano e manipolano valori. Le espressioni possono essere complesse e quindi devono essere **valutate** per restituire il singolo valore che sta denotando. Espressioni **sintatticamente diverse** possono denotare lo stesso valore. Ad esempio $1 + 5$ e $3 * 2$ denotano lo stesso valore.

  Definizione 3.3. Due espressioni sono equivalenti se vengono valutate nello **stesso** valore a partire da qualunque stato/memoria iniziale.

- **Comandi**: denotano richieste di modifica della memoria.
  - Le strutture di controllo decidono quali comandi eseguire

– Gli assegnamenti modificano effettivamente la memoria

Rappresentano funzioni: $\mathcal{Memoria} \mapsto \mathcal{Memoria}$. I comandi vengono **eseguiti** per attuare la corrispondente richiesta di trasformazione della memoria. Queste trasformazioni sono **irreversibili**, cioè non possono essere cancellate, ma solo annullate semanticamente, ad esempio:

$$\mathrm{x} := \mathrm{x} + 1;\ \mathrm{x} := \mathrm{x} - 1$$

Questi comandi si annullano, ma lo stato è stato comunque modificato.

Definizione 3.4. Due comandi sono equivalenti se per ogni stato iniziale in input producono lo stesso stato in output. Quindi l'equivalenza è vista come uguaglianza della funzione calcolata.

- **Dichiarazioni**: denotano richieste di creazione o modifica di legami, ovvero di **ambienti**. Le dichiarazioni vengono **elaborate** per attuare le richieste di creazione o modifica degli ambienti.

  Definizione 3.5. Due dichiarazioni sono equivalenti se producono lo stesso ambiente (e la stessa memoria) a partire da qualunque stato iniziale.

La separazione tra le categorie sintattiche non è rigida, ad esempio un'espressione può avere anche side effects, quindi modificare la memoria, ecc...

3.2.6 Esempio di sintassi per il linguaggio "PL0"

Il seguente è un esempio di sintassi per il linguaggio di programmazione PL0 simile al linguaggio Pascal, in cui:

- Esiste un singolo tipo di dato `integer`
- I booleani sono rappresentati come interi
- Strutture di controllo standard
- Astrazione del controllo
- Le procedure non hanno parametri

Il linguaggio è definito nel seguente modo:

```
<Program>  P → B
  <Block>  B → DS
 <Declar>  D → [const I = N{,I=N};]
           | [var I{,I};]
           | [procedure I;B;]
  <Stmts>  S → ε | I := E
           | call I
           | if C then S
           | while C do S
           | begin S{;S} end
   <Cond>  C → odd E | E > E
           | E >= E | E = E
           | E # E | E < E
           | E <= E
   <Expr>  E → I | N | E op E | (E)
```

- I programmi P sono blocchi B.
- I blocchi B sono una dichiarazione D seguita da un comando S. Si noti che D ed S sono definiti in modo libero dal contesto, ovvero quello che possiamo generare da S non deve avere necessariamente nessun vincolo con quello che generiamo da D. In altre parole questa grammatica non ci obbliga in nessun modo ad usare identificatori precedentemente definiti. Quindi dalla descrizione sintattica rimangono fuori gli aspetti context sensitive (vincoli sintattici) che la grammatica CF non è in grado di rappresentare. Servirà la definizione di una "semantica statica" che avrà il ruolo di verificare questi vincoli prima di compilare e/o eseguire.
- Le dichiarazioni D sono dichiarazioni di valori costanti con nome (`const`), dichiarazioni di variabili (`var`) e dichiarazioni di procedure (ovvero di un blocco riferibile con un nome).
- I nomi sono gli identificatori I, considerati qui parte del vocabolario; N sono i numeri naturali, considerati anch'essi parte del vocabolario.
- Non abbiamo bisogno del concetto di tipo in quanto abbiamo solo un tipo di dato.
- Si noti che le produzioni di D sono tutte opzionali, questo significa che un programma può essere un blocco senza dichiarazioni.
- I comandi sono: il comando vuoto, l'assegnamento di un valore (denotato da una espressione) ad un identificatore, la chiamata ad una procedura mediante il suo nome, il comando condizionale che esegue un comando al verificarsi di una condizione booleana C, il ciclo che ripete un comando finché una data condizione C è vera e la composizione sequenziale di comandi.

- Le condizioni C sono espressioni dal valore booleano: `odd` è un predicato unario che verifica se l'espressione è 0, poi ci sono vari operatori di confronto di espressioni E.
- Le espressioni E sono identificatori, valori naturali, operazioni tra espressioni e espressioni tra parentesi.
- Si sintetizza con op la metavariabile dell'insieme $\{+, -, *, /\}$. La stessa cosa si potrebbe fare per gli operatori di confronto, mettendo in C la produzione E cop E con cop metavariabile per gli elementi in $\{>, >=, =, \#, <=, <\}$.

La grammatica delle espressioni è ambigua, ma più comoda da usare e si può utilizzare per definire la semantica. Per il compilatore però bisogna fornire una grammatica equivalente precisa e quindi non ambigua, come la seguente:

```
  <Expr>  E → [+,-]T[AT]
<Add op>  A → + | -
 <Terms>  T → F[MF]
          M → * | /
<Factor>  F → I | N | (E)
<Numbers> N → [+,-]d{d}
 <Digit>  d → 0 | ... | 9
 <Ident>  I → l{l, d}
<Letter>  l → A | ... | Z
```

4 Implementazione di un linguaggio di programmazione IMP

4.1 Espressioni

4.1.1 Aspetti implementativi

Gli aspetti progettuali da definire per le espressioni sono:

- **Arietà**: cioè a quanti argomenti un operatore si applica
- **Notazione**:
  - **Infissa**: `e1 + e2`
    Questa notazione richiede la definizione di regole di **associatività** e **precedenza** per descrivere quale operatore applicare prima. Le seguentinotazioni non ne hanno bisogno.
  - **Prefissa**: `+ e1 e2`
    Si utilizza una variabile contatore `arietà` che tiene traccia del numero di operandi dell'operatore da applicare e uno stack in cui vengono messi gli operandi.

> **Esempio 4.1.** Consideriamo la seguente espressione:
>
> $$+ * 325$$

dove:

* $*$ accetta due operandi
* $+$ accetta due operandi

Si inizia con uno stack vuoto e `arietà` $= 0$, quindi si legge da sinistra a destra:

1. Si legge $+$, è un operatore, quindi si mette nello stack e si aggiorna `arietà`:

$$\mathtt{Stack} = [+], \quad \texttt{arietà} = 2$$

2. Si legge $*$, è un operatore, quindi si mette nello stack. L'`arietà` rimane a 2 perchè fa riferimento all'ultimo operatore inserito nello stack:

$$\mathtt{Stack} = [+, *], \quad \texttt{arietà} = 2$$

3. Si legge 3, è un operando, quindi si mette nello stack e si aggiorna `arietà`:

$$\mathtt{Stack} = [+, *, 3], \quad \texttt{arietà} = 1$$

4. Si legge 2, è un operando, quindi si mette nello stack e si aggiorna `arietà`:

$$\mathtt{Stack} = [+, *, 3, 2], \quad \texttt{arietà} = 0$$

   A questo punto `arietà` $= 0$ quindi si applica l'operatore $*$ agli ultimi due operandi inseriti nello stack, si mette il risultato nello stack e si aggiorna `arietà`:

$$\mathtt{Stack} = [+, 6], \quad \texttt{arietà} = 1$$

5. Si legge 5, è un operando, quindi si mette nello stack e si aggiorna `arietà`:

$$\mathtt{Stack} = [+, 6, 5], \quad \texttt{arietà} = 0$$

   A questo punto `arietà` $= 0$ quindi si applica l'operatore $+$ agli ultimi due operandi inseriti nello stack, si mette il risultato nello stack e si aggiorna `arietà`:

$$\mathtt{Stack} = [11], \quad \texttt{arietà} = 0$$

– **Postfissa**: `e1 e2 +`
Si utilizza uno stack che conterrà gli operandi di ciascun operatore, quindi quando si incontra un operatore si prendono dal stack gli operandi necessari.

Esempio 4.2. Consideriamo la seguente espressione:

$$5\,3\,2\,*\,+$$

dove:

- $*$ accetta due operandi
- $+$ accetta due operandi

Si inizia con uno stack vuoto, quindi si legge da sinistra a destra:

1. Si legge 5, è un operando, quindi si mette nello stack:
$$\mathrm{Stack} = [5]$$
2. Si legge 3, è un operando, quindi si mette nello stack:
$$\mathrm{Stack} = [5, 3]$$
3. Si legge 2, è un operando, quindi si mette nello stack:
$$\mathrm{Stack} = [5, 3, 2]$$
4. Si legge $*$, è un operatore, quindi si prendono dal stack i due operandi necessari, si applica l'operatore e si mette il risultato nello stack:
$$\mathrm{Stack} = [5, 6]$$
5. Si legge $+$, è un operatore, quindi si prendono dal stack i due operandi necessari, si applica l'operatore e si mette il risultato nello stack:
$$\mathrm{Stack} = [11]$$

- **Regole di associatività**: decidono l'ordine di applicazione di operatori allo stesso livello di precedenza. Solitamente per gli operatori aritmetici si sceglie l'associatività a sinistra, quindi si applica prima l'operatore più a sinistra.
- **Regole di precedenza**: cioè in che ordine valutare le espressioni quando non sono esplicitamente raggruppate da parentesi.

  Per gli operatori aritmetici le **precedenze** sono:

  1. $+, -$ sono allo stesso livello
  2. $*, /$ sono allo stesso livello e hanno precedenza su $+, -$
- **Ordine di valutazione degli operandi**: cioè in che ordine valutare gli operandi di un operatore. Bisogna essere consapevoli delle seguenti problematiche legate all'ordine di valutazione degli operandi:
  - **Operandi non definiti**: ad esempio in alcuni linguaggi è possibile scrivere espressioni non definite, ad esempio:
    ```
    a == 0 ?  b :  b / a
    ```
    è un'espressione non definita se $\mathtt{a} = 0$ perchè si avrebbe una divisione per zero. Ci sono due tipi di valutazioni:

    - **Valutazione eager**: si valutano prima tutte le parti dell'espressione
    - **Valutazione lazy**: si valutano solo le parti necessarie per applicare l'operatore, quindi in questo caso si valuterebbe solo a e soltanto se $a \neq 0$ si valuterebbe b/a.
  - **Ordine dei side effects**: Consideriamo il seguente codice:

    ```
    a := 10;
    b := a + fun(&a);
    ```

    dove `fun` è una funzione che prende un puntatore ad una variabile e modifica il suo parametro. Supponiamo che `fun(&a)` imposti il valore di `a` a 12 e lo ritorni.
    - Se si valuta prima `a` e poi si esegue `fun(&a)`, si avrebbe `b = 10 + fun(&a)` e dopo l'esecuzione di `fun(&a)` si avrebbe `a = 12` e

      ```
      b = 10 + 12 = 22
      ```

    - Se si valuta prima `fun(&a)` e poi `a`, si avrebbe `b = a + fun(&a)` e dopo l'esecuzione di `fun(&a)` si avrebbe `a = 12` e

      ```
      b = 12 + 12 = 24
      ```

    Quindi l'ordine di valutazione degli operandi può influenzare il risultato di un'espressione se ci sono side effects.

- **Presenza di side effects**: cioè se la valutazione di un'espressione modifica lo stato/memoria
- **Overloading degli operatori**: cioè se un operatore può essere applicato a più tipi di operandi, ad esempio in alcuni linguaggi il $+$ può essere applicato a interi come somma, oppure a stringhe come concatenazione
- **Espressioni con operandi di tipo misto**: cioè se un operatore può essere applicato a operandi di tipi diversi, ad esempio in alcuni linguaggi è possibile sommare un intero ad una stringa applicando casting automatici

4.1.2 Sintassi

Definiamo la sintassi del linguaggio IMP per le espressioni:

<Expr> `E` $\rightarrow$ `A` | `B`
`A` $\rightarrow$ `N` | `A op A`
`B` $\rightarrow$ `true` | `false`
| `A = A` | `not B` | `B or B`

Dove:

- `N` sono i numeri naturali, considerati parte del vocabolario
- `op` è l'insieme di operatori aritmetici $\{+, -, *\}$
- `A` sono le espressioni aritmetiche
- `B` sono le espressioni booleane

4.1.3 Semantica per espressioni aritmetiche

Consideriamo:

- $\mathtt{N}$: l'insieme dei valori numerici $\mathtt{n}, \mathtt{m}, \mathtt{p}$
- $\mathcal{E}$: l'insieme di espressioni valutabili, ovvero alberi di valutazione di espressioni e. Cioè una stringa di soli terminali generati in Exp

Diamo una semantica per le **espressioni aritmetiche** definendo un sistema di transizione:

$$\mathtt{Stati} : \mathcal{E} \cup \mathtt{N} \text{ (stati terminali)}$$

Le **regole di transizione** sono definite attraverso regole costruite induttivamente sulla grammatica:

$$\mathrm{op} \in \{+, -, *\}$$

Le regole di transizione per le espressioni sono:

- Regola 1 (Assioma): Definisce p come la semantica del calcolo effettivo di un operatore

$$\mathcal{E}_1 : \overbrace{\mathtt{m}\ \ \mathtt{op}\ \ \mathtt{n}}^{\text{simboli}} \to \mathrm{p} = \underbrace{\mathtt{m}\ \ \mathtt{op}\ \ \mathtt{n}}_{\text{significato}}$$

- Regola 2: Definisce la valutazione dell'espressione a sinistra per prima

$$\mathcal{E}_2 : \frac{e_0 \to e_0'}{e_0\ \ \mathrm{op}\ \ e_1 \to e_0'\ \ \mathrm{op}\ \ e_1}$$

- Regola 3: Definisce la valutazione dell'espressione a destra con un'espressione già valutata a sinistra

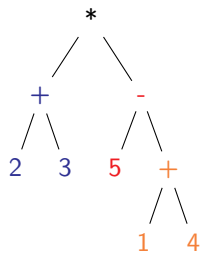
$$\mathcal{E}_3 : \frac{e_1 \to e_1'}{\mathtt{m}\ \ \mathrm{op}\ \ e_1 \to \mathtt{m}\ \ \mathrm{op}\ \ e_1'}$$

Le regole $\mathcal{E}_2$ e $\mathcal{E}_3$ impongono un ordine di valutazione delle espressioni, in questo caso da sinistra a destra.

Esempio 4.3. Consideriamo la seguente espressione valutandola seguendo le regole appena definite:

$$\underbrace{(2+3)}_{e_0} * \underbrace{(5-(1+4))}_{e_1}$$

Si ha il seguente albero di valutazione:



Le regole che abbiamo fissato non ci permettodno di valutare prima e_1 e poi e_0, ma ci obbliga a valutare prima e_0 e poi e_1. Si potrebbe aggiungere la seguente regola alternativa per permettere di valutare prima e_1:

$$\mathcal{E}_3' : \frac{e_1 \to e_1'}{e_0 \ \texttt{op} \ e_1 \to e_0 \ \texttt{op} \ e_1'}$$

4.1.4 Semantica per espressioni booleane

Diamo una semantica per le **espressioni booleane**: Gli stati terminali sono i seguenti:

$$\mathtt{B} = \{\mathtt{true}, \mathtt{false}\}$$

Definiamo le regole di transizione continuando da quelle già definite per le espressioni aritmetiche:

- Regola 4:
$$\mathcal{E}_4 : \underbrace{\mathtt{t}_1 \ \mathtt{or} \ \mathtt{t}_2}_{\text{simbolo}} \to \mathtt{t} = \underbrace{\mathtt{t}_1 \ \mathtt{or} \ \mathtt{t}_2}_{\text{significato}} \quad \mathtt{t}, \mathtt{t}_1, \mathtt{t}_2 \in \mathtt{B}$$
- Regola 2: Vale anche per le espressioni booleane
- Regola 3: Adattata per le espressioni booleane
$$\mathcal{E}_3' : \frac{e_1 \to e_1'}{\mathtt{t} \ \mathtt{or} \ e_1 \to \mathtt{t} \ \mathtt{or} \ e_1'}$$
- Uniamo $\mathcal{E}_3$ con $\mathcal{E}_3'$
$$\frac{e_1 \to e_1'}{\mathtt{k} \ \mathtt{bop} \ e_1 \to \mathtt{k} \ \mathtt{bop} \ e_1'}$$
con $\mathtt{k} \in \mathtt{N} \cup \mathtt{B}$ e $\mathtt{bop} \in \{\mathtt{or}, +, -, *\}$
- Regola 5:
$$\mathcal{E} : \frac{e \to e'}{\mathtt{not} \ e \to \mathtt{not} \ e'}$$
- Regola 6:
$$\mathcal{E}_6 : \mathtt{not} \ e_t \to \mathtt{t}' = \mathtt{not} \ \mathtt{t} \quad \mathtt{t}, \mathtt{t}' \in \mathtt{B}$$

Il sistema di transizione che abbiamo definito è: $\left\langle \mathcal{E} \cup \underbrace{\mathtt{N} \cup \mathtt{B}}_{\text{Terminali}} \right\rangle$

Definizione 4.1 (Valutazione)**.** La funzione di valutazione:

$$\mathtt{Eval} : \ \mathtt{Exp} \to \overbrace{\mathtt{N} \cup \mathtt{B}}^{\text{Con}}$$

descrive il comportamento dinamico delle espressioni:

$$\forall e \in \mathcal{E} \ . \ \mathtt{Eval}(e) = \mathtt{k} \iff e \to_e^* \mathtt{k}$$

dove $\rightarrow_e^*$ è la chiusura transitiva di $\rightarrow$, cioè:

$$\exists n \,.\, e \rightarrow e_1 \rightarrow e_2 \rightarrow \ldots \rightarrow e_n = k$$

con n transizioni.

Definizione 4.2 (Equivalenza di espressioni)**.** Definiamo la relazione di equivalenza $\equiv \subseteq Exp \times Exp$ tra espressioni come:

$$e_1 \equiv e_2 \iff Eval(e_1) = Eval(e_2)$$

cioè due espressioni sono equivalenti se vengono valutate allo stesso valore.

4.2 Identificatori

Gli identificatori sono **sequenze di caratteri** usate per riferirsi ad **oggetti denotabili**, ovvero a variabili, costanti, funzioni, procedure, ecc...

Esempio 4.4. Alcuni esempi di identificatori sono i seguenti:

```
const pi = 3.14;
int x;
void f(...){...}
```

Ogni identificatore crea un legame tra il nome dell'identificatore e l'oggetto che denota, ad esempio `pi` denota la costante 3.14, `x` denota una variabile di tipo intero e `f` denota una procedura.

Definizione 4.3 (Oggetti denotabili)**.** Sono oggetti del linguaggio di programmazione a cui si può fare riferimento mediante l'uso di identificatori

Nel nostro linguaggio gli identificatori sono definiti nel seguente insieme:

$$Id = \langle a, \texttt{rate}, \texttt{b20}, \ldots, \texttt{id}, \texttt{x}, \ldots \rangle$$

4.2.1 Aspetti implementativi

Gli aspetti implementativi da tenere in considerazione nella definizione degli identificatori sono i seguenti:

- **Nomi case sensitive o case insensitive**
- **Parole riservate**
- **Vincoli sulla lunghezza**
- **Presenza di caratteri speciali**

Le parole chiave del nostro linguaggio oppure altre specifiche che dipendono dal linguaggio non si possono usare come identificatori.

4.2.2 Legami, scope e occorrenze

Definizione 4.4 (Legami (binding))**.** Consideriamo il seguente esempio:

$$\left(\sum_{i=1}^{n}\left(\sum_{j=1}^{m} a_{ij}\dots b_{ik}\right)\right)$$

- Le **definizioni** creano dei legami tra il nome e il significato, cioè crea **occorrenze di binding**.
- Gli usi del legame, cioè **occorrenze d'uso (applied)** sono quelli che utilizzano il nome per far riferimento al significato
- Le **occorrenze libere** sono quelle che non fanno riferimento ad alcun legame, quindi il significato non è definito.

Definizione 4.5 (Scope)**.** Lo scope è il **raggio di azione** di una definizione. Determina lo spazio in cui le potenziali occorrenze dell'identificatore definito fanno riferimento a quella definizione $\mathtt{def}$.

$$\underbrace{\left(\sum_{\underbrace{i=1}_{def}}^{n}\underbrace{\left(\sum_{\underbrace{j=1}_{def}}^{m}\overbrace{a_{ij}\dots b_{ik}}^{uso}\right)}_{scope\ j}\right)}_{scope\ i}$$

Nei linguaggi di programmazione gli identificatori possono essere in diverse posizioni e sono le seguenti:

- **Definizione** (occorrenze di binding): Un identificatore è in posizione di definizione quando si attribuisce il significato dell'identificatore
- **Uso** (occorrenze di uso): Un identificatore è in posizione di uso quando l'occorrenza serve per riferirsi al significato determinato da una definizione. Deve trovarsi nello scope (raggio di azione) di una definizione.
- **Libera**: Un identificatore è in posizione libera se la sua occorrenza non si trova nello scope (raggio di azione) di una definizione.

4.2.3 Tipi di binding

Nel seguente esempio i legami tra i nomi e i significati sono:

`const pi = 3.14;`	costante
`int x;`	locazione in memoria
`void f(...){...}`	pezzo di codice

Esistono due tipi di binding:

- **Binding statici**: Sono legami che non cambiano durante l'esecuzione del programma, a meno che non vengano sovrascritti da un altro legame. Un legame può essere:
  - Creato
  - Distrutto
  - Sovrascritto
  - **Non può essere modificato**
- **Binding dinamici**: Alcuni legami possono cambiare, ad esempio il legame tra una locazione e un valore. Questi legami sono **dinamici**, cioè uno degli elementi del legame cambia durante l'esecuzione del programma. Ad esempio il valore di una variabile può cambiare, ma il nome della variabile e la locazione in memoria a cui fa riferimento non cambiano.

Definizione 4.6 (Tempo di binding)**.** È il momento in cui un legame viene creato e dipende dal linguaggio e anche dagli oggetti. Ad esempio le costanti presenti nel linguaggio vengono legate a tempo di progettazione, mentre le costanti definite dall'utente vengono legate a tempo di implementazione, cioè quando si compila il programma.

4.2.4 Ambiente

I legami vengono inseriti nell'**ambiente**, che non è altro che un **insieme di binding**. In un programma tutti gli identificatori devono essere definiti, cioè non possono mai essere in **posizione libera**.

Ci sono due tipi di termini:

- **Termini ground**: Un termine è detto **ground** se non ha identificatori e ha sempre significato. Nel nostro caso le espressioni che abbiamo definito sono ground.
- **Tremini chiusi**: Un termine è detto **chiuso** se non ha identificatori in posizione libera

Entrambi questi tipi di termini hanno significato senza dover ricorrere ad un contesto esterno, cioè ad un ambiente.

4.2.5 Aggiungiamo gli identificatori alla sintassi delle espressioni

Riprendiamo la seguente notazione:

- $\mathcal{E}$: l'insieme di espressioni
- $\mathrm{DVal} = \mathtt{N} \cup \mathtt{B}$: l'insieme di valori numerici e booleani, che sono anche **valori denotabili**

Definiamo un ambiente per le espressioni:

$$\mathrm{Env} = \bigcup_{I \subseteq_f \mathrm{Id}} \mathrm{Env}_I$$

$$\mathrm{Env}_I : I \to \mathrm{Dval} \quad \rho \text{ metavariabile per un ambiente specifico}$$

L'ambiente è una funzione che associa un valore denotabile ad ogni identificatore in un insieme I finito ($\subseteq_f$ sottoinsieme finito).

Esempio 4.5. Consideriamo i seguenti identificatori:

$$I = \{x, y\}$$

Allora un esempio di ambiente è il seguente:

$$\rho \in \mathrm{Env}_I : \quad \rho(x) = 5 \qquad \rho(y) = 5$$

L'espressione $x + y - 5$ ha significato soltanto se valutata nell'ambiente ρ:

$$\rho \vdash x + y - 5$$

Ora aggiungiamo alla sintassi delle espressioni gli identificatori con $x \in \mathrm{Id}$:

$$\begin{aligned} \texttt{<Expr} \quad & \texttt{E} \to \texttt{A} \mid \texttt{B} \\ & \texttt{A} \to \texttt{N} \mid x \mid \texttt{A op A} \\ & \texttt{B} \to \texttt{true} \mid \texttt{false} \mid x \\ & \mid \texttt{A = A} \mid \texttt{not B} \mid \texttt{B or B} \end{aligned}$$

4.2.6 Semantica per espressioni con identificatori

Definiamo la semantica per le espressioni da valutare in un ambiente ρ:

$$\rho \vdash e \to e'$$

questo significa che l'espressione e viene valutata in e' nell'ambiente ρ. Le regole di transizione per le espressioni con identificatori diventano:

- Regola 1 (Assioma):

$$\rho \vdash \mathtt{m\ op\ n} \to p = \mathtt{m\ op\ n}$$

- Regola 2:

$$\frac{\rho \vdash e_0 \to e_0'}{\rho \vdash e_0 \ \texttt{bop} \ e_1 \to e_0' \ \texttt{bop} \ e_1}$$

  con $\texttt{bop} \in \{\texttt{or}, +, -, *\}$

- Regola 3+4:

$$\frac{\rho \vdash e_1 \to e_1'}{\rho \vdash \texttt{k} \ \texttt{bop} \ e_1 \to \texttt{k} \ \texttt{bop} \ e_1'}$$

  con $\texttt{bop} \in \{\texttt{or}, +, -, *\}$

- Regola 5:

$$\frac{\rho \vdash e \to e'}{\rho \vdash \texttt{not} \ e \to \texttt{not} \ e'}$$

- Regola 6:

$$\rho \vdash \texttt{not} \ \texttt{t} \to \texttt{t}' = \texttt{not} \ \texttt{t}$$

Sono le stesse regole di transizione che avevamo definito per le espressioni senza identificatori, ma ora sono valutate in un ambiente ρ che viene propagato in tutte le transizioni.

Ora definiamo una nuova regola che utilizza l'ambiente ρ per valutare un identificatore x:

- Regola 7:

$$\rho \vdash x \to k \text{ se } \rho(x) = k$$

Esempio 4.6. Consideriamo la seguente espressione:

```
x + y - 5
```

da valutare nell'ambiente:

$$\rho = [x \mapsto 2, y \mapsto 5]$$

Applichiamo le regole:

1.

$$\dfrac{\dfrac{\dfrac{\rho \vdash \mathtt{x} \to 2}{\rho \vdash \mathtt{x\ +\ y\ -\ 5} \to 2 + \mathtt{y\ -\ 5}}\ \rho(\mathrm{x}) = 2}{\rho \vdash \mathtt{y\ -\ 5} \to 5 - 5}\ \rho(\mathrm{y}) = 5}{\rho \vdash \mathtt{2\ +\ y\ -\ 5} \to 2 + 5 - 5}$$

$$\downarrow$$

$$\dfrac{\dfrac{\rho \vdash 5 - 5 \to 0}{\rho \vdash 2 + 5 - 5 \to 2 + 0}}{\rho \vdash 2 + 0 \to 2}$$

4.3 Tipo di dato

Il tipo è l'insieme dei valori denotabili da un identificatore e l'insieme degli operatori applicabili, tali valori devono condividere una certa proprietà. Ad esempio, il tipo `int` è l'insieme dei numeri interi, il tipo `bool` è l'insieme dei valori booleani, ecc...

L'utilizzo del tipo ha varie funzionalità:

- **A tempo di progettazione**: permette di organizzare le funzionalità del linguaggio.
- **A tempo di sviluppo**: servono a prevenire errori di programmazione, ad esempio se si cerca di sommare una stringa ad un intero, il compilatore può segnalare un errore di tipo.
- **A tempo di implementazione**: Servono per capire quanta memoria allocare per una certa variabile.

Il tipo deve essere legato alla variable, quindi viene aggiunto al legame con l'oggetto denotato. Si dovrà creare un ambiente separato per i tipi e si avrà:

- **Semantica statica**, che viene utilizzata durante la compilazione per verificare la correttezza dei tipi e per allocare la memoria necessaria per le variabili. **Verifica che le espressioni siano scritte correttamente**.
- **Semantica dinamica**, come quella definita per le espressioni (ρ), che viene utilizzata durante l'esecuzione. Infatti la valutazione delle espressioni **assume che siano scritte correttamente**.

4.3.1 Ambiente statico

È l'insieme di legami tra identificatori e tipi. I tipi denotabili sono definiti dal seguente insieme:

$$\mathrm{DTyp} = \{\mathtt{int}, \mathtt{bool}\}$$

Con valori denotabili DVal che sono i valori numerici e booleani, quindi:

$$\mathrm{DVal} = \mathtt{N} \cup \mathtt{B}$$

L'insieme di tutti gli ambienti è l'insieme di tutti i possibili ambienti di un certo identificatore I:

$$\mathrm{TEnv} = \bigcup_{I \subseteq_f \mathrm{Id}} \mathrm{TEnv}_I$$

dove per ogni identificatore in I associa un tipo in DTyp

$$\mathrm{TEnv}_I : I \to \mathrm{DTyp}$$

La metavariabile utilizzata per rappresentare un generico ambiente statico è $\Delta \in \mathrm{TEnv}$. La seguente notazione asserisce che l'espressione e è di tipo $\tau \in \mathrm{DTyp}$ nell'ambiente statico Δ definito sugli identificatori in I:

$$\Delta \vdash_I e : \tau$$

Quando l'ambiente non è indicato, si sottintende l'insieme vuoto:

$$\vdash e : \tau \equiv \varnothing \vdash e : \tau$$

Cioè per ogni ambiente è possibile applicare la regola di semantica statica.

Se le regole della semantica statica permettono di associare un tipo ad un'espressione, allora l'espressione è **ben formata**

4.3.2 Semantica statica per le espressioni

La semantica statica per le espressioni associa un tipo alle espressioni ben formate, quindi verifica solo vincoli di tipo tra operandi. Definiamo la semantica statica per le espressioni:

- Regola 1: con $n \in \mathtt{N}$
$$\mathcal{E}_{S_1} : \vdash n : \mathtt{int}$$
- Regola 2: con $t \in \mathtt{B}$
$$\mathcal{E}_{S_2} : \vdash t : \mathtt{bool}$$
- Regola 3:
$$\mathcal{E}_{S_3} : \Delta \vdash_I x : \tau \quad \text{se} \quad x \in I,\ \Delta(x) = \tau$$
- Regola 4:
$$\mathcal{E}_{S_4} : \frac{\Delta \vdash_I e_1 : \mathtt{int} \quad \Delta \vdash_I e_2 : \mathtt{int}}{\Delta \vdash_I e_1 \ \mathtt{op}\ e_2 : \mathtt{int}}$$
- Regola 5:
$$\mathcal{E}_{S_5} : \frac{\Delta \vdash_I e_1 : \mathtt{bool} \quad \Delta \vdash_I e_2 : \mathtt{bool}}{\Delta \vdash_I e_1 \ \mathtt{or}\ e_2 : \mathtt{bool}}$$
- Regola 6:
$$\mathcal{E}_{S_6} : \frac{\Delta \vdash_I e : \mathtt{bool}}{\Delta \vdash_I \mathtt{not}\ e : \mathtt{bool}}$$
- Regola 7:
$$\mathcal{E}_{S_7} : \frac{\Delta \vdash_I e_1 : \mathtt{int} \quad \Delta \vdash_I e_2 : \mathtt{int}}{\Delta \vdash_I e_1 = e_2 : \mathtt{bool}}$$

4.3.3 Ambienti compatibili

Definizione 4.7. Considerando $I \subseteq_f Id$

- Ambiente dinamico $\rho \in Env_I$ che associa valori a nomi
- Ambiente statico $\Delta \in TEnv_I$ che associa tipi a identificatori

Si dice che ρ e Δ sono compatibili se

$$\forall id \in I \,.\, \Delta(id) = \tau \iff \rho(id) \in \tau$$

cioè se per ogni identificatore id in I, il tipo associato a id in Δ è τ se e solo se il valore associato a id in ρ è un elemento di τ.

Per dire che due ambienti ρ e Δ sono compatibili, si scrive:

$$\rho \vdash_\Delta$$

Esempio 4.7. Consideriamo ad esempio i seguenti ambienti:

- Ambienti compatibili

$$\Delta = [x \mapsto \texttt{int}, y \mapsto \texttt{bool}]$$
$$\rho = [x \mapsto 2, y \mapsto \texttt{true}]$$

- Ambienti non compatibili

$$\Delta = [x \mapsto \texttt{int}, y \mapsto \texttt{bool}]$$
$$\rho = [x \mapsto 2, y \mapsto 3]$$

Per definire gli ambienti (sia Δ che ρ) c'è bisogno di una categoria sintattica che permette di creare e modificare legami che vanno a costituire gli ambienti

4.4 Dichiarazioni

La dichiaraazione è una **categoria sintattica** che fornisce il meccanismo (implicito o esplicito) per **creare legami** tra identificatori e tipi (statico) o oggetti (dinamico). Le dichiarazioni denotano richieste di **modifica o creazione di ambienti**.

Le dichiarazioni vengono **elaborate** per ottenere le richieste di modifica o creazione di ambienti

4.4.1 Sintassi delle dichiarazioni

Definiamo la sintassi delle dichiarazioni:

$$\begin{aligned} \mathcal{D} : D \to\ & \texttt{nil} \\ & \mid \texttt{const x: } \tau = e \\ & \mid D; D \mid D \texttt{ in } D \\ & \mid \rho \end{aligned}$$

dove:

- `nil`: corrisponde a $\Delta = \varnothing$
- `const x:` $\tau = e$: è la dichiarazione di un `id` costante di nome x, tipo τ e valore rappresentato da e.
- $D; D$: è la composizione sequenziale, cioè ogni dichiarazione successiva o aggiunge nuovi legami o sovrascrive legami esistenti. L'ambiente risultante è così visibile al codice che viene successivamente

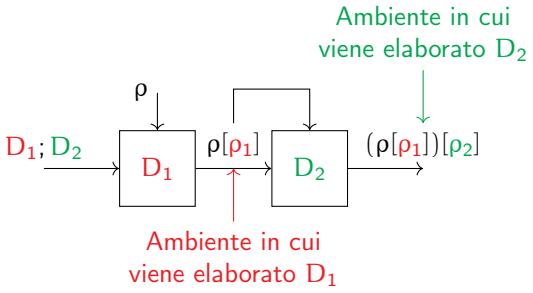


Figura 7: Composizione sequenziale $D_1; D_2$

Esempio 4.8. Consideriamo ad esempio il seguente codice:

```
1 const x: int = 3;
2 const x: int = 5; const y: int = 6 * x;
3 const z: int = x + y;
```

Osserviamo i valori di questo codice:

1. A riga 1 il valore assegnato a x è $\rho = [x \mapsto 3]$
2. A riga 2 il valore assegnato a x è $\rho[x \mapsto 5] = [x \mapsto 5] = \rho_1$ e viene usata per elaborare y che assume il valore $\rho_2 = [x \mapsto 5, y \mapsto 6 * 5 = 30]$
3. A riga 3 viene usato il valore di x dichiarato a riga 2 e il valore di y dichiarato a riga 2, cioè $(\rho[\rho_1])[\rho_2] = [x \mapsto 5, y \mapsto 30]$, per elaborare z che assume il valore $\rho_3 = [x \mapsto 5, y \mapsto 30, z \mapsto 5+30 = 35]$

- D `in` D: è la composizione privata, cioè I legami creati o sovrascritti da D_1 sono **ad uso esclusivo** dell'elaborazione di D_2 e **non visibili** al codice successivo.

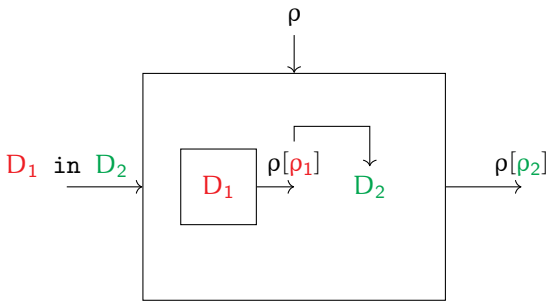


Figura 8: Composizione privata D_1 `in` D_2

Esempio 4.9. Consideriamo ad esempio il seguente codice:

```
1 const x: int = 3;
2 const x: int = 5 in y: int = 6 * x;
3 const z: int = x + y;
```

Osserviamo i valori di questo codice:

1. A riga 1 il valore assegnato a x è $\rho = [x \mapsto 3]$
2. A riga 2 il valore assegnato a x è $\rho_1 = [x \mapsto 5]$ e viene usata per elaborare y che assume il valore $\rho_2 = [y \mapsto 6 * 5 = 30]$
3. A riga 3 viene usato il valore di x dichiarato a riga 1 e il valore di y dichiarato a riga 2, cioè $\rho[\rho_2] = [x \mapsto 3, y \mapsto 30]$, per elaborare z che assume il valore $\rho_3 = [x \mapsto 3, y \mapsto 30, z \mapsto 3 + 30 = 33]$

- ρ: è un ambiente dinamico presente nella grammatica per motivi formali

4.4.2 Aggiornamento di un ambiente

Consideriamo gli ambienti β e β'. La modifica di β da parte di β' si scrive:

$$\beta\,[\beta']$$

Consideriamo che β sia definito sugli identificatori I:

$$\beta : I \quad I \subseteq_f Id$$

e β' sia definito sugli identificatori I':

$$\beta' : I' \quad I' \subseteq_f Id$$

Il nuovo ambiente parlerà sia degli identificatori in I che di quelli in I':

$$\beta\,[\beta'] = \overline{\beta} \quad \overline{\beta} : I \cup I'$$

Se l'identificatore è presente sia nell'ambiente β che in β', allora la priorità è data a β':

$$\forall x \in I' \,.\, \overline{\beta}(x) = \beta'(x)$$

Se l'identificatore è presente solo in β allora si mantiene lo stesso legame:

$$\forall x \in I \setminus I' \,.\, \overline{\beta}(x) = \beta(x)$$

Quindi $\overline{\beta}$ è un nuovo ambiente in cui si prendono tutti i legami di β e β' e per quelli in comune si tengono solo quelli di β'. Di fatto β' **sovrascrive** β.

Esempio 4.10. Consideriamo i seguenti ambienti:

$$\rho = [x \mapsto 2, y \mapsto \texttt{true}, z \mapsto 5]$$
$$\rho' = [x \mapsto 5, w \mapsto \texttt{false}]$$

Allora l'aggiornamento di ρ da parte di ρ' è il seguente:

$$\rho\,[\rho'] = [x \mapsto 5, w \mapsto \texttt{false}, y \mapsto \texttt{true}, z \mapsto 5]$$

Si vede che x è stato sovrascritto da ρ', mentre y e z sono rimasti invariati e w è stato aggiunto.

4.4.3 Identificatori in posizione libera e in posizione di definizione

Caratterizziamo le dichiarazioni:

- **In posizione libera (FI)**: Associa ad ogni costrutto sintattico l'insieme degli identificatori **liberi**, cioè identificatori che non hanno un legame e richiedono un contesto per avere significato.

  $$\mathrm{FI}(\mathrm{Exp} \cup \mathrm{Dec}) = I \subseteq \mathrm{Id} \quad \mathrm{FI} : \mathrm{Exp} \cup \mathrm{Dec} \to \mathcal{P}(\mathrm{Id})$$

  Definiamo FI per induzione:

  - **Caso base**: $\mathrm{FI}(k) = \varnothing$ con $k \in \mathtt{N} \cup \mathtt{B}$
  - **Passo induttivo**:
    - Espressioni:
      - $\mathrm{FI}(x) = \{x\}$
      - $\mathrm{FI}(e_1 \texttt{ op } e_2) = \mathrm{FI}(e_1) \cup \mathrm{FI}(e_2) = \mathrm{FI}(e_1 \texttt{ or } e_2)$
      - $\mathrm{FI}(\texttt{not } e) = \mathrm{FI}(e)$
    - Dichiarazioni:
      - $\mathrm{FI}(\texttt{nil}) = \varnothing$
      - $\mathrm{FI}(\texttt{const x: } \tau = e) = \mathrm{FI}(e)$
      - $\mathrm{FI}(d_1; d_2) = \mathrm{FI}(d_1) \cup (\mathrm{FI}(d_2) \setminus \mathrm{DI}(d_1) = \mathrm{FI}(d_1 \texttt{ in } d_2)$ dove $\mathrm{DI}(d_1)$ è l'insieme degli identificatori definiti da d_1
      - $\mathrm{FI}(\rho) = \varnothing$

  Questa funzione fornisce l'insieme degli identificatori per cui bisogna cercare un significato nell'ambiente per poter valutare l'espressione o elaborare la dichiarazione.

- **In posizione di definizione (DI)**: Restituisce gli identificatori che nel costrutto vengono legati ad un significato che è visibile all'esterno del costrutto

$$\mathrm{DI}: \ \mathrm{Exp} \cup \mathrm{Dec} \to \mathcal{P}(\mathrm{Id})$$

Definiamo DI:

  - Espressioni: Le espressioni non definiscono identificatori, quindi:

$$\forall e \in \mathrm{Exp} \ . \ \mathrm{DI}(e) = \varnothing$$

  - Dichiarazioni:
    * $\mathrm{DI}(\texttt{nil}) = \varnothing$
    * $\mathrm{DI}(\texttt{const x: } \ \tau = e) = \{x\}$
    * $\mathrm{DI}(\rho) = I \quad \rho : I \ (\rho \in \mathrm{Env}_I)$
    * $\mathrm{DI}(d_1 \ \texttt{in} \ d_2) = \mathrm{DI}(d_2)$: solo gli identificatori legati in d_2 hanno significato **dopo** l'elaborazione della composizione
    * $\mathrm{DI}(d_1; d_2) = \mathrm{DI}(d_1) \cup \mathrm{DI}(d_2)$: tutti gli identificatori legati in d_1 e d_2 restano legati al loro significato dopo l'elaborazione della composizione

4.4.4 Semantica statica per le dichiarazioni

La semantica statica per le dichiarazioni associa un ambiente statico Δ alle dichiarazioni ben formate. Definiamo la semantica statica per le dichiarazioni:

$$\Delta \vdash d : \Delta'$$

dove:

- Δ è il contesto, cioè eventuali dichiarazioni precedenti
- Δ' è un ambiente definito (e quindi associato) da d

Le regole per le dichiarazioni sono:

- Regola 1:

$$\mathcal{D}_{S_1} : \vdash \texttt{nil} : \varnothing$$

- Regola 2:

$$\mathcal{D}_{S_2} : \vdash \rho : \Delta$$

  dove ρ e Δ sono compatibili.

> **Esempio 4.11.** Ad esempio:
>
> $$\rho = [x \mapsto 3, y \mapsto \texttt{true}]$$
>
> $$\downarrow$$
>
> $$\Delta = [x \mapsto \texttt{int}, y \mapsto \texttt{bool}]$$

- Regola 3:

$$\mathcal{D}_{S_3}: \frac{\Delta \vdash e : \tau}{\Delta \vdash \texttt{const x: } \tau = e : [x \mapsto \tau]}$$

  I tipi **devono essere tutti uguali**

- Regola 4:

$$\mathcal{D}_{S_4}: \frac{\Delta \vdash d_1 : \Delta_1 \qquad \Delta[\Delta_1] \vdash d_2 : \Delta_2}{\Delta \vdash d_1 \texttt{ in } d_2 : \Delta_2}$$

- Regola 5:

$$\mathcal{D}_{S_5}: \frac{\Delta \vdash d_1 : \Delta_1 \qquad \Delta[\Delta_1] \vdash d_2 : \Delta_2}{\Delta \vdash d_1; d_2 : \Delta_1[\Delta_2]}$$

4.4.5 Semantica dinamica per le dichiarazioni

L'ambiente statico generato dalla semantica statica è compatibile con l'ambiente dinamico. La forma della regola di transizione per le dichiarazioni è la seguente:

$$\rho \vdash_\Delta d \to_d d'$$

rappresenta che d viene elaborata, con un passo di elaborazione, in d' nell'ambiente dinamico ρ.

Gli stati sono dichiarazioni generati dalla grammatica o ambienti

$$\mathcal{D} \cup \mathrm{Env}$$

Le regole per le dichiarazioni sono:

- Regola 1: Assioma

$$\mathcal{D}_1 : \vdash \texttt{nil} \to \varnothing$$

  $d = \rho$ è già terminale, non c'è nulla da elaborare

- Regola 2: Dichiarazione di una costante

$$\mathcal{D}_2 : \ \rho \vdash \texttt{const x: } \tau = k \to [x \mapsto k]$$

- Regola 3: Elaborazione di un'espressione

$$\mathcal{D}_3 : \frac{\rho \vdash e \to_e e'}{\rho \vdash \texttt{const x: } \tau = e \to_d \texttt{const x: } \tau = e'}$$

- Regola 4: Elaborazione di una composizione sequenziale

$$\mathcal{D}_4 : \frac{\rho \vdash d_1 \to_d d_1'}{\rho \vdash d_1; d_2 \to_d d_1'; d_2}$$

- Regola 5: Elaborazione di una composizione sequenziale con un ambiente già elaborato

$$\mathcal{D}_5 : \frac{\rho[\rho_1] \vdash d_2 \to_d d_2'}{\rho \vdash \rho_1; d_2 \to_d \rho_1; d_2'}$$

- Regola 6: Assioma per la composizione privata, quando entrambi gli ambienti sono stati elaborati

$$\mathcal{D}_6 : \ \rho \vdash \rho_1; \rho_2 \rightarrow_d \rho_1[\rho_2]$$

- Regola 7: Elaborazione di una composizione privata

$$\mathcal{D}_7 : \ \frac{\rho \vdash d_1 \rightarrow_d d_1'}{\rho \vdash d_1 \ \texttt{in} \ d_2 \rightarrow_d d_1' \ \texttt{in} \ d_2}$$

- Regola 8: Elaborazione di una composizione privata con un ambiente già elaborato

$$\mathcal{D}_8 : \ \frac{\rho[\rho_1] \vdash d_2 \rightarrow_d d_2'}{\rho \vdash \rho_1 \ \texttt{in} \ d_2 \rightarrow_d \rho_1 \ \texttt{in} \ d_2'}$$

- Regola 9: Assioma per la composizione privata, quando entrambi gli ambienti sono stati elaborati

$$\mathcal{D}_9 : \ \rho \vdash \rho_1 \ \texttt{in} \ \rho_2 \rightarrow_d \rho_2$$

4.4.6 Elaborazione

Definizione 4.8 (Funzione di elaborazione)**.** La funzione di elaborazione genera un ambiente dinamico a partire da una dichiarazione della grammatica:

$$\mathtt{Elab} : \mathcal{D} \rightarrow \mathtt{Env}$$

$$\forall d \in \mathcal{D} \ . \ \mathtt{Elab}(d) = \rho \iff \vdash d \rightarrow_d^* \rho$$

cioè l'elaborazione di d è l'ambiente ρ se e solo se d viene elaborata in ρ con un numero finito di passi di elaborazione.

Definizione 4.9 (Equivalenza tra dichiarazioni)**.** La relazione di equivalenza tra due dichiarazioni è definita come:

$$\equiv \subset \mathcal{D} \times \mathcal{D}$$

$$\forall d_1, d_2 \in \mathcal{D} \ . \ d_1 \equiv d_2 \iff \mathtt{Elab}(d_1) = \mathtt{Elab}(d_2)$$

cioè due dichiarazioni sono equivalenti se e solo se la loro elaborazione genera lo stesso ambiente dinamico.

Esempio 4.12. Un esempio di applicazione delle regole della semantica per elaborare una dichiarazione è il seguente:

$$\underbrace{\texttt{const x: int = 2}}_{d_1} \ \texttt{in} \ \underbrace{\overbrace{\texttt{const y: int = x+1}}^{d_3}; \overbrace{\texttt{const z: int = x+y}}^{d_4}}_{d_2}$$

L'obiettivo è quello di elaborare d_1 `in` d_2 applicando l'assioma per la compo-

sizione privata. Prima di elaborare d_1 `in` d_2 bisogna prima elaborare i singoli componenti d_1 e d_2 per poter applicare l'assioma.

1. Quindi il primo passo di elaborazione è quello di elaborare d_1: La regola da applicare è la seguente:

$$\frac{\vdash d_1 \to_d d_1' \text{ (oppure } \rho)}{\vdash d_1 \texttt{ in } d_2 \to_d d_1' \texttt{ in } d_2} \tag{1}$$

   Applichiamo questa regola:

$$\frac{\vdash \texttt{const x: int = 2} \to_d \overbrace{[x \mapsto 2]}^{\rho_1}}{\vdash d_1 \texttt{ in } d_2 \to_d [x \mapsto 2] \texttt{ in } d_2}$$

   Alla fine d_1 è stata elaborata in $\rho_1 = [x \mapsto 2]$.

2. Ora bisogna elaborare d_2 nell'ambiente appena elaborato $\rho_1 = [x \mapsto 2]$. La regola da applicare è la seguente:

$$\frac{\rho_1 \vdash d_2 \to_d d_2'}{\vdash \rho_1 \texttt{ in } d_2 \to_d \rho_1 \texttt{ in } d_2'} \tag{2}$$

   Oppure una regola equivalente che esegue transitivamente l'elaborazione completa di d_2 prima di applicare l'assioma per d_1 `in` d_2:

$$\frac{\rho_1 \vdash d_2 \to_d^* \rho_2}{\vdash \rho_1 \texttt{ in } d_2 \to_d \rho_1 \texttt{ in } \rho_2} \tag{3}$$

   Siccome $d_2 = d_3; d_4$, bisogna elaborare prima i singoli componenti d_3 e d_4 nell'ambiente ρ_1:

   (a) Per prima cosa bisogna elaborare d_3 e la regola da applicare è la seguente:

$$\frac{\rho_1 \vdash d_3 \to_d d_3'}{\rho_1 \vdash d_3; d_4 \to_d d_3'; d_4} \tag{4}$$

   oppure la sua versione transitiva:

$$\frac{\rho_1 \vdash d_3 \to_d^* \rho_3}{\rho_1 \vdash d_3; d_4 \to_d \rho_3; d_4} \tag{5}$$

   Siccome $d_3 =$ `const y: int = x+1` bisogna prima elaborare `x+1` nell'ambiente ρ_1. Applichiamo la regola per elaborare un'espressione:

$$\frac{\rho_1 \vdash x \to 2}{\rho_1 \vdash x + 1 \to 2 + 1} \; \rho_1(x) = 2$$

   Applichiamo l'assioma:

$$\rho_1 \vdash 2 + 1 \to 3$$

Ora che abbiamo elaborato $x + 1$ possiamo elaborare d_3 usando la regola (5):

$$\frac{\rho_1 \vdash x + 1 \to_d^* 3}{\rho_1 \vdash \underbrace{\texttt{const y: int = x+1}}_{d_3} \to_d \underbrace{\texttt{const y: int = 3}}_{d_3'}}$$

Applichiamo l'assioma:

$$\rho_1 \vdash \texttt{const y: int = 3} \to_d [y \mapsto 3]$$

Quindi dopo aver elaborato d_3 otteniamo l'ambiente $\rho_3 = [y \mapsto 3]$

(b) Dopo aver elaborato d_3 bisogna elaborare d_4 nell'ambiente appena elaborato: $\rho_1[\rho_3] = [x \mapsto 2, y \mapsto 3]$. La regola da applicare è la seguente:

$$\frac{\rho_1[\rho_3] \vdash d_4 \to_d d_4'}{\rho_1 \vdash \rho_3; d_4 \to_d \rho_3; d_4'} \tag{6}$$

oppure la sua versione transitiva:

$$\frac{\rho_1[\rho_3] \vdash d_4 \to_d^* \rho_4}{\rho_1 \vdash \rho_3; d_4 \to_d \rho_3; \rho_4} \tag{7}$$

Siccome $d_4 = \texttt{const z: int = x+y}$, bisogna prima elaborare x e poi y nell'ambiente $[x \mapsto 2, y \mapsto 3]$ per ottenere il valore di $x+y$:

$$\frac{\rho_1[\rho_3] \vdash x \to 2}{\rho_1[\rho_3] \vdash x + y \to 2 + y} \; \rho_1[\rho_3](x) = 2$$

$$\frac{\rho_1[\rho_3] \vdash y \to 3}{\rho_1[\rho_3] \vdash 2 + y \to 2 + 3} \; \rho_1[\rho_3](y) = 3$$

Applichiamo l'assioma:

$$\rho_1[\rho_3] \vdash 2 + 3 \to 5$$

Ora che sappiamo il valore di $x+y$ possiamo elaborare d_4 usando la regola (7):

$$\frac{\rho_1[\rho_3] \vdash x+y \to_d^* 5}{\rho_1[\rho_3] \vdash \underbrace{\texttt{const z: int = x+y}}_{d_4} \to_d \underbrace{\texttt{const z: int = 5}}_{d_4'}}$$

Applichiamo l'assioma:

$$\rho_1[\rho_3] \vdash \texttt{const z: int = 5} \to_d [z \mapsto 5]$$

Quindi dopo aver elaborato d_4 otteniamo l'ambiente $\rho_4 = [z \mapsto 5]$.

Dopo aver elaborato d_3 e d_4 possiamo elaborare $d_2 = d_3; d_4$ applicando l'assioma:

$$\rho_1 \vdash \underbrace{\rho_3 ; \rho_4}_{d_2} \rightarrow_d \rho_3[\rho_4] = \rho_2 = [y \mapsto 3, z \mapsto 5]$$

Abbiamo ottenuto l'ambiente $\rho_2 = [y \mapsto 3, z \mapsto 5]$ dopo aver elaborato d_2.

Ora che abbiamo elaborato d_1 nell'ambiente ρ_1 e d_2 nell'ambiente ρ_2, possiamo elaborare d_1 `in` d_2 applicando l'assioma per la composizione privata:

$$\vdash \rho_1 \texttt{ in } \rho_2 \rightarrow_d \rho_2$$

e alla fine otteniamo:

$$\vdash d_1 \texttt{ in } d_2 \rightarrow_d^* \rho_2 = [y \mapsto 3, z \mapsto 5]$$

4.5 Variabili

L'introduzione di **identificatori variabili** richiede l'introduzione del concetto della memoria e degli strumenti per manipolarla, cioè i **comandi**. Un classico assegnamento di una variabile è il seguente:

$$\underbrace{\texttt{x}}_{\text{Target}} \texttt{ := } \underbrace{\texttt{x}}_{\text{Sorgente}} \texttt{ +1}$$

In questo caso si osserva che il ruolo della `x` cambia a seconda di dove si trova:

- Se si trova a sinistra è detto **L-value** (target) e il valore viene usato in scrittura
- Se si trova a destra è detto **R-value** (sorgente) e il valore viene usato in lettura

Per gestire questo cambio di ruolo è necessario:

- Gestire diversi oggetti con lo stesso nome (ad esempio nella ricorsione o nella definizione delle variabili)
- Gestire diversi nomi per lo stesso oggetto (ad esempio puntatori o aliasing)
- Gestire nomi e valori che cambiano durante l'esecuzione

È stato quindi introdotto un concetto astratto detto **locazione** che gestisce questi aspetti.

4.5.1 Locazione

La locazione è un elemento per le variabili che viene inserito tra l'identificatore e il valore o oggetto denotato. Fino ad ora abbiamo avuto soltanto un legame identificatore-valore:

$$\texttt{id} \longleftrightarrow \texttt{valore/oggetto}$$

Con l'introduzione delle locazioni, il legame diventa:

$$\texttt{id} \overset{\text{Ambiente}}{\longleftrightarrow} \texttt{locazione} \overset{\text{Memoria}}{\longleftrightarrow} \texttt{valore/oggetto}$$

- L'ambiente gestisce delle **trasformazioni reversibili**, quindi potenzialmente temporanee perchè possono essere sovrascritte da altre trasformazioni per poi essere recuperate in un secondo momento
- La memoria gestisce delle **trasformazioni irreversibili**, quindi permanenti perchè cambiare il valore in memoria significa perdere il valore precedente e sovrascriverlo con un nuovo valore. Nonostante il valore precedente possa essere riassegnato in un secondo momento, non è lo stesso valore di prima, ma è un nuovo valore che sovrascrive quello precedente

Definizione 4.10 (Memoria)**.** La memoria tiene traccia delle trasformazioni effettuate dall'esecuzione di programmi

Definizione 4.11 (Locazione)**.** La locazione viene creata dalle dichiarazioni:

- Se la dichiarazione è esplicita, allora viene creata a tempo di compilazione
- Se la dichiarazione è implicita, allora viene creata dai comandi a tempo di esecuzione (nei linguatti dinamici/interpretati)

Le locazioni:

- rappresentano una posizione in memoria e ha un **tempo di vita**, cioè un intervallo di tempo in cui è attiva. Il tempo di vita di una locazione dipende dal linguaggio di programmazione, solitamente coincide con la durata di tutto il programma, ma in alcuni linguaggi può essere limitato a un blocco di codice o alle allocazioni e deallocazioni dinamiche.
- Vengono usate per contenere/riferire oggetti che possono cambiare durante l'esecuzione
- Sono considerate illimitate
- Sono oggetti denotabili

Ci sono più tipi di legami:

- **Identificatori costanti**: non usano la memoria perchè vengono associati a valori che non cambiano durante l'esecuzione. Sono quindi legami che **restano nell'ambiente dinamico**. Come notazione si usa:

  `const`

  che crea un legame (binding) in un ambiente.
- **Variabili** (nei linguaggi imperativi): sono identificate da due legami:
  - Uno nell'ambiente che rappresenta la locazione

    $$x \to \square$$

– Uno nella memoria che rappresenta il valore a cui la locazione fa riferimento che può anche cambiare

$$\square \to 2$$

Come notazione si usa:

$$\texttt{var}$$

che crea un **legame** in un ambiente e una **associazione** in memoria:

$$\mathrm{x} \underbrace{\longrightarrow}_{\substack{\text{Legame}\\ \text{in ambiente}}} \underbrace{\square}_{\text{Locazione}} \underbrace{\longrightarrow}_{\substack{\text{Associazione}\\ \text{in memoria}}} 2$$

Nel nostro linguaggio abbiamo che:

- I valori **esprimibili** sono elementi primitivi di Exp:

$$\mathrm{B} \cup \mathrm{N}$$

- I valori **memorizzabili**, cioè che possono essere associati ad una locazione, sono:

$$\mathrm{B} \cup \mathrm{N}$$

- I valori **denotabili**, cioè che possono essere legati ad un identificatore, sono:

$$\mathrm{B} \cup \mathrm{N} \cup \underbrace{\mathrm{L}}_{\text{Locazioni}}$$

4.5.2 Implementazione delle variabili nelle dichiarazioni

Modifichiamo le dichiarazioni per introdurre le variabili:

$$\begin{aligned}\mathcal{D} : \mathrm{D} \to\ & \texttt{nil}\\ & \mid \texttt{const x: } \tau = e\\ & \mid \texttt{var x: } \tau = e\\ & \mid \mathrm{D}; \mathrm{D} \mid \mathrm{D}\ \texttt{in}\ \mathrm{D}\\ & \mid \rho\end{aligned}$$

Questa sintassi dichiara una variabile di nome x, tipo τ e valore **iniziale** descritto da e.

- Gli identificatori liberi sono:

$$\mathrm{FI}(\texttt{var x: } \tau = e) = \mathrm{FI}(e)$$

- Gli identificatori definiti sono:

$$\mathrm{DI}(\texttt{var x: } \tau = e) = \{\mathrm{x}\}$$

4.5.3 Semantica statica per le dichiarazioni con variabili

La semantica statica verifica che la dichiarazione sia ben formata associando un ambiente statico alla dichiarazione. Bisogna verificare, a livello statico, che non sia possibile modificare il valore di una costante. Per fare ciò bisogna aggiungere i tipi per le locazioni:

$$\mathrm{DTyp} = \{\texttt{int}, \texttt{bool}, \texttt{intloc}, \texttt{boolloc}\}$$

Aggiungiamo una nuova regola alla semantica statica delle dichiarazioni per gestire le variabili:

- Regola 6:

$$\frac{\Delta \vdash e : \tau}{\Delta \vdash \texttt{var x: } \tau = e : [x \mapsto \tau\texttt{loc}]}$$

Definizione utile 4.1. Anche nelle espressioni gli identificatori possono essere variabili e questo non cambia la sintassi.

Ricordiamo l'assioma per gli identificatori

$$\Delta \vdash x : \tau \quad \text{se } \Delta(x) = \tau$$

Dove τ è il tipo associato a x. Ora che abbiamo aggiunto le locazioni nei tipi, bisogna gestire anche i tipi delle locazioni e la nuova regola per gli identificatori diventa:

$$\Delta \vdash x : \tau \quad \text{se } \Delta(x) \in \{\tau, \tau\texttt{loc}\}$$

Quindi i valori a cui x si riferisce sono sempre di tipo τ, ma la dichiarazione di x può essere di **costante** ($\Delta(x) = \tau$) o di **variabile** ($\Delta(x) = \tau\texttt{loc}$).

4.5.4 Aggiornamento della memoria

Bisogna definire la **memoria** e integrarla nelle regole della semantica dinamica per le dichiarazioni. Consideriamo:

- Loc l'insieme delle locazioni
- $new(L)$ è la funzione che restituisce una nuova, cioè che non appartiene a L, locazione
- $\mathrm{Mem}_L : L \to \mathcal{V}$ è una funzione che associa ad ogni locazione un valore
- $\mathrm{Mem} = \bigcup_{L \subseteq_f \mathrm{Loc}} \mathrm{Mem}_L$ è l'insieme di tutte le funzioni di memoria.
- $\mathrm{Env}_I : I \to \mathcal{V} + \mathrm{Loc}$ è una funzione che associa ad ogni identificatore un valore o una locazione
- $\mathrm{Env} = \bigcup_{I \subseteq_f \mathrm{Id}} \mathrm{Env}_I$ è l'insieme di tutte le funzioni di ambiente

Siano $\sigma \in \mathrm{Env}_I$ e $\sigma' \in \mathrm{Env}_{I'}$, l'aggiornamento della memoria si rappresenta come:

$$\sigma[\sigma']$$

dove σ è modificata da σ' e genera una nuova memoria σ'' tale che:

$$\forall x \in I' \,.\, \sigma''(x) = \sigma'(x)$$
$$\forall x \in I \setminus I' \,.\, \sigma''(x) = \sigma(x)$$

cioè σ'' è una memoria che contiene tutte le associazioni di σ' e tutte le associazioni di σ che non sono sovrascritte da σ':

$$\sigma'' \in Env_{I \cup I'}$$

4.5.5 Semantica dimanica per le espressioni con identificatori variabili

Consideriamo:

- **Stati**: $\mathcal{E} \times Mem$. Gli stati intermedi sono l'espressione da valutare e la memoria iniziale
- **Stati terminali**: $\mathcal{V} \times Mem$ cioè il valore associato all'espressione più la memoria

La forma della regola sarà del tipo:

$$\rho \vdash \langle e, \sigma \rangle \rightarrow_e \langle e', \sigma \rangle$$

Seguendo questa forma possiamo aggiornare gli assiomi per le espressioni in modo da gestire le variabili:

- $$\rho \vdash \langle m \ \texttt{op} \ n, \sigma \rangle \rightarrow \langle p, \sigma \rangle \quad p = m \ \texttt{op} \ n$$
- $$\rho \vdash \langle t_1 \ \texttt{or} \ t_2, \sigma \rangle \rightarrow \langle t, \sigma \rangle \quad t = t_1 \ \texttt{or} \ t_2$$
- $$\rho \vdash \langle \texttt{not} \ t, \sigma \rangle \rightarrow \langle t', \sigma \rangle \quad t' = \texttt{not} \ t$$

Quindi si ha che:

$$\rho \vdash \langle x, \sigma \rangle \rightarrow \langle k, \sigma \rangle$$

- Se $\rho(x) = l \in Loc$ allora $\sigma(l) = k$, cioè x è variabile
- Altrimenti $\rho(x) = k$ cioè x è costante

Ora bisogna aggiornare anche le altre regole per gestire le variabili:

- $$\frac{\rho \vdash \langle e_1, \sigma \rangle \rightarrow_e^* \langle k, \sigma \rangle}{\rho \vdash \langle e_1 \ \texttt{bop} \ e_2, \sigma \rangle \rightarrow \langle k \ \texttt{bop} \ e_2, \sigma \rangle}$$
- $$\frac{\rho \vdash \langle e_2, \sigma \rangle \rightarrow_e^* \langle k', \sigma \rangle}{\rho \vdash \langle e_2 \ \texttt{bop} \ e_2, \sigma \rangle \rightarrow \langle k \ \texttt{bop} \ k', \sigma \rangle}$$

- $$\frac{\rho \vdash \langle e, \sigma\rangle \to_e^* \langle t, \sigma\rangle}{\rho \vdash \langle \texttt{not } e, \sigma\rangle \to \langle \texttt{not } t, \sigma\rangle}$$

- **Valutazione**: La valutazione per gestire le variabili diventa:

  $$\mathit{Eval} : \mathit{Exp} \times \mathit{Mem} \to \mathcal{V}$$

  dove:

  $$\mathit{Eval}(e, \sigma) = k \iff \vdash \langle e, \sigma\rangle \to^* k$$

- **Equivalenza**: L'equivalenza per gestire le variabili diventa:

  $$\equiv \subseteq \mathit{Exp} \times \mathit{Exp}$$

  dove:

  $$e_1 \equiv e_2 \iff \forall \sigma \in \mathit{Mem} \,.\, \mathit{Eval}(e_1, \sigma) = \mathit{Eval}(e_2, \sigma)$$

  Cioè qualunque sia il valore associato agli identificatori la valutazione delle espressioni deve restare la stessa

4.5.6 Semantica dinamica per le dichiarazioni con variabili

- **Stati**: $\mathit{Dec} \times \mathit{Mem}$
- **Stati terminali**: $\mathit{Env} \times \mathit{Mem}$

La forma della regola sarà del tipo:

$$\vdash \langle d, \sigma\rangle \to_d \langle d', \sigma'\rangle$$

L'elaborazione porta ad una memoria σ' perchè la memoria σ può essere stata modificata siccome le dichiarazioni di variabile **inizializzano gli identificatori anche nella memoria**.

Usando questa forma possiamo aggiornare gli assiomi per le dichiarazioni in modo da gestire le variabili:

- $$\rho \vdash \langle \texttt{var } x: \quad \tau = k, \sigma\rangle \to \langle [x \mapsto l] \,, \sigma[l \mapsto k]\rangle$$

  dove $l \in \mathit{Loc}$ è una nuova locazione per x e la memoria σ viene aggiornata con l'associazione $l \mapsto k$ per inizializzare la variabile x con il valore k.

Bisogna aggiornare anche le regole della dichiarazione composta per gestire le variabili (semplicemente propagando la memoria):

- $$\rho \vdash \langle \texttt{const } x: \quad \tau = k, \sigma\rangle \to_d \langle [x \mapsto k] \,, \sigma\rangle$$

- $$\rho \vdash \langle \texttt{nil}, \sigma\rangle \to_d \langle \varnothing, \sigma\rangle$$

- $$\frac{\rho \vdash \langle d_1, \sigma\rangle \to_d^* \langle \rho_1, \sigma'\rangle}{\rho \vdash \langle d_1; d_2, \sigma\rangle \to_d \langle \rho_1; d_2, \sigma'\rangle}$$
- $$\frac{\rho[\rho_1] \vdash \langle d_2, \sigma\rangle \to_d^* \langle \rho_2, \sigma'\rangle}{\rho \vdash \langle \rho_1; d_2, \sigma\rangle \to_d \langle \rho_1; \rho_2, \sigma'\rangle}$$
- $$\rho \vdash \langle \rho_1; \rho_2, \sigma\rangle \to_d \langle \rho_1[\rho_2], \sigma\rangle$$

E anche le regole per la dichiarazione privata:

- $$\frac{\rho \vdash \langle d_1, \sigma\rangle \to_d^* \langle \rho_1, \sigma'\rangle}{\rho \vdash \langle d_1 \ \texttt{in}\ d_2, \sigma\rangle \to_d \langle \rho_1 \ \texttt{in}\ d_2, \sigma'\rangle}$$
- $$\frac{\rho[\rho_1] \vdash \langle d_2, \sigma'\rangle \to_d^* \langle \rho_2, \sigma'\rangle}{\rho \vdash \langle \rho_1 \ \texttt{in}\ d_2, \sigma'\rangle \to_d \langle \rho_1 \ \texttt{in}\ \rho_2, \sigma'\rangle}$$
- $$\rho \vdash \langle \rho_1 \ \texttt{in}\ \rho_2, \sigma\rangle \to_d \langle \rho_1[\rho_2], \sigma\rangle$$

- **Elaborazione**: L'elaborazione per gestire le variabili diventa:

$$\mathit{Elab} : \mathit{Dec} \times \mathit{Mem} \to \mathit{Env} \quad \underbrace{\times \mathit{Mem}}_{\text{Se vogliamo considerare anche i side effects}}$$

  dove:

$$\forall d \in \mathit{Dec}, \sigma \in \mathit{Mem} \,.\, \mathit{Elab}(d, \sigma) = (\rho, \sigma') \iff \exists \sigma' \,.\, \vdash \langle d, \sigma\rangle \to_d^* \langle \rho, \sigma'\rangle$$

  Oppure $\nexists \sigma'$ se si vogliono considerare anche i side effects

- **Equivalenza**: L'equivalenza per gestire le variabili diventa:

$$\equiv \subseteq \mathit{Dec} \times \mathit{Dec}$$

  dove:

$$\forall d_1, d_2 \,.\, d_1 \equiv d_2 \iff \forall \sigma \ \mathit{Elab}(d_1, \sigma) = \mathit{Elab}(d_2, \sigma)$$

4.6 Comandi

I comandi sono una categoria sintattica il cui ruolo principale consiste nella trasformazione (irreversibile) della memoria. I comandi vengono **eseguiti** per attuare le richieste di trasformazione della memoria che rappresentano.

Definizione utile 4.2 (Reversibilità)**.** Non esiste alcun meccanismo automatico che annulla la trasformazione

Esistono due tipi di comandi

1. **Comandi base**: sono i comandi che denotano le trasformazioni della memoria. Ad esempio, l'assegnamento o il comando nullo
2. **Comandi di controllo del flusso**: sono i comandi che determinano un controllo del flusso di esecuzione. Ad esempio i cicli o le istruzioni condizionali

4.6.1 Sintassi dei comandi

Aggiungiamo i comandi alla sintassi del linguaggio:

$$\begin{aligned} \mathrm{Com}: \ \mathrm{C} \to & \overbrace{\texttt{skip} \mid \mathrm{x} := e}^{\text{Comandi base}} \\ & \mid \mathrm{C}; \mathrm{C} \\ & \mid \texttt{while B do C} \\ & \mid \texttt{if B then C else C} \end{aligned}$$

- **Assegnamento**: modifica il valore della variabile x con la valutazione dell'espressione e
- **Variabile**: la variabile ha le seguenti caratteristiche:
  - Nome
  - Tipo
  - Valore memorizzato (ciò che è salvato in memoria)
  - Locazione (la posizione in memoria a cui la variabile fa riferimento)
  - **Scope** (l'area di codice del programma in cui la variabile è visibile e accessibile)
  - **Lifetime o tempo di vita** (l'intervallo di tempo durante il quale la variabile esiste e può essere utilizzata)

4.6.2 Semantica statica dei comandi

La semantica statica dei comandi verifica che i comandi siano ben formati e quindi fa solo **controlli di coerenza di tipi**. Consideriamo l'ambiente statico Δ e definiamo la semantica statica per i comandi:

- $\mathrm{FI}(\texttt{x := e}) = \mathrm{FI}(e) \cup \{\mathrm{x}\}$. Per far si che l'assegnamento abbia significato, tutti devono avere un significato, quindi x deve essere definito e e deve essere ben formata
- $\mathrm{DI}(\texttt{x := e}) = \varnothing$. L'assegnamento non definisce nuovi identificatori, ma modifica il valore di un identificatore **già definito**

- $$\frac{\Delta \vdash e : \tau}{\Delta \vdash \texttt{x := e}} \; \Delta(x) = \tau\texttt{loc}$$

  $\Delta(x)$ deve essere di tipo locazione $\tau\texttt{loc}$ perchè x è una variabile e quindi può essere usata come L-value in un assegnamento. Quindi si ha che `x := e` è ben formata se:

  1. x è di tipo locazione
  2. x ed e hanno lo stesso tipo di valore τ

- $\Delta \vdash \texttt{skip}$ non ha vincoli da verificare

4.6.3 Semantica dinamica dei comandi

La semantica dinamica dei comandi descrive come i comandi modificano la memoria durante l'esecuzione. Consideriamo:

- **Stati**: $\mathrm{Com} \times \mathrm{Mem}$
- **Stati terminali**: Mem, cioè i comandi vengono eseguiti per manipolare la memoria

La forma della regola sarà del tipo:

$$\rho \vdash \langle c, \sigma \rangle \to_c \langle c', \sigma' \rangle$$

Usando questa forma possiamo definire la semantica dinamica per i comandi:

- Skip
$$\rho \vdash \langle \texttt{skip}, \sigma \rangle \to_c \sigma$$
- Assegnamento: si valuta l'espressione e in k con un certo numero di passi partendo da σ
$$\frac{\rho \vdash \langle e, \sigma \rangle \to_e^* \langle k, \sigma' \rangle}{\rho \vdash \langle \texttt{x := e}, \sigma \rangle \to \langle \texttt{x := k}, \sigma \rangle}$$
- Assioma per l'assegnamento:
$$\rho \vdash \langle \texttt{x := k}, \sigma \rangle \to_c \sigma[l \mapsto k], \;\; \rho(x) = l$$
  dove ρ indica quale locazione è legata ad x

Esempio 4.13. Un esempio di esecuzione del seguente assegnamento:

$$\texttt{x := x * y}$$

x e y sono **liberi** e quindi per eseguire l'assegnamento abbiamo bisogno degli ambienti contestuali e poi dobbiamo usare una memoria. Supponiamo che l'ambiente ρ sia:

$$\rho = [x \mapsto l_x, y \mapsto 2]$$
$$\Delta = [x \mapsto \texttt{intloc}, y \mapsto \texttt{int}]$$

e la memoria σ sia:

$$\sigma = [l_x \mapsto 3]$$

Andiamo a verificare se l'assegnamento è ben formato usando l'ambiente statico:

$$\dfrac{\dfrac{\Delta \vdash \texttt{x: int} \quad \Delta \vdash \texttt{y: int}}{\Delta \vdash \texttt{x * y: int}}\ \Delta(x) = \texttt{intloc} \wedge \Delta(y) = \texttt{int}}{\Delta \vdash \texttt{x := x * y}}\ \Delta(x) = \texttt{intloc}$$

cioè se `x * y` è di tipo `int` allora `x := x * y` è ben formata.

Esecuzione: La regola da applicare è quella per l'assegnamento:

$$\dfrac{\rho \vdash \langle \texttt{x * y}, \sigma\rangle \to_e^* \langle k, \sigma\rangle}{\rho \vdash \langle \texttt{x := x * y}, \sigma\rangle \to \langle \texttt{x: = k}, \sigma\rangle}$$

Valutiamo `x * y`, ma prima per farlo dobbiamo valutare i singoli componenti `x` e `y`:

1. Valutiamo `x`:

$$\dfrac{\rho \vdash \langle x, \sigma\rangle \to \langle l_x, \sigma\rangle}{\rho \vdash \langle \texttt{x * y}, \sigma\rangle \to \langle 3 * y, \sigma\rangle}\ \rho(x) = l_x \wedge \sigma(l_x) = 3$$

2. Valutiamo `y`:

$$\dfrac{\rho \vdash \langle y, \sigma\rangle \to \langle 2, \sigma\rangle}{\rho \vdash \langle 3 * y, \sigma\rangle \to \langle 3 * 2, \sigma\rangle}$$

Ora possiamo completare la valutazione di `x * y`:

$$\rho \vdash \langle 3 * 2, \sigma\rangle \to \langle 6, \sigma\rangle$$

Si conclude applicando la regola per l'assegnamento:

$$\dfrac{\rho \vdash \langle \texttt{x * y}, \sigma\rangle \to_e^* \langle 6, \sigma\rangle}{\rho \vdash \langle \texttt{x := x * y}, \sigma\rangle \to \langle \texttt{x: = 6}, \sigma\rangle}$$

Infine applichiamo l'assioma per l'assegnamento:

$$\rho \vdash \langle \texttt{x: = 6}, \sigma\rangle \to_c \sigma[l_x \mapsto 6],\ \rho(x) = l_x$$

4.6.4 Strutture di controllo

Le strutture di controllo manipolano il flusso di esecuzione e possono essere implementate

- Tra espressioni nei linguaggi con le espressioni condizionali
- Tra comandi (if, while, for)
- Tra procedure

Esistono diversi tipi di comandi di controllo, ad esempio:

- **Comandi di selezione del controllo**: sono i comandi che permettono di scegliere tra più opzioni di esecuzione. I comandi di selezione si dividono in:

– **Selettori a due vie**: permettono di scegliere tra due opzioni a seconda della guardia a due valori, come ad esempio l'if. La sintassi dell'if è:

$$\texttt{if } guardia \texttt{ then } code_1 \texttt{ else } code_2$$

Gli aspetti progettuali per implementare un if sono:

* Forma e tipo di espressione di controllo (la guardia). Alcuni linguaggi interpretano i numeri come guardia, altri invece richiedono solo espressioni booleane
* Specifica e delimitazione i rami `then` e `else`. Cioè quali elementi sintattici si usano per delimitare i rami, ad esempio usando le parentesi graffe o usando l'indentazione, ecc...
* Come viene gestito l'annidamento di condizionali. Ad esempio nel seguente codice

```
if (sum == 0)
    if (count == 0)
        result = 0;
else result = 1;
```

bisogna specificare se l'`else` è associato al primo `if` o al secondo.

– **Selettori a più vie**: permettono di scegliere tra più opzioni a seconda di una singola guardia a più valori, come ad esempio il case. La sintassi di un tipico selettore a più vie è:

$$\begin{aligned}&\texttt{switch (}expr\texttt{) \{}\\&\quad\texttt{case } val_1\texttt{:}\quad stmt_1\texttt{;}\\&\quad\vdots\\&\quad\texttt{case } val_n\texttt{:}\quad stmt_n\texttt{;}\\&\quad[\texttt{default:}\quad stmt_d\texttt{;}]\\&\texttt{\}}\end{aligned}$$

dove $stmt$ è un singolo comando o blocco di codice.

Gli aspetti progettuali per implementare un case sono:

* Forma e tipo delle espressioni di guardia. Tipicamente espressioni non booleane
* Come si selezionano i vari rami
* Come procede il flusso di controllo.
* Cosa succede per i valori non rappresentati. Alcuni linguaggi dopo aver eseguito un ramo escono dal case, altri invece continuano ad eseguire i rami successivi, oppure alcuni richiedono un ramo default in caso di mancata corrispondenza con i valori di guardia

• **Comandi di iterazione**: sono i comandi che permettono di ripetere una sequenza di comandi. Ad esempio, while, for.

Gli aspetti progettuali per implementare un comando di iterazione sono:

– Com'è controllata l'iterazione. I possibili controlli sono:

* **Controllo logico** (while)

    * **Controllo numerico** (for)
  - Dove si trova il meccanismo di controllo. Alcuni costrutti eseguono il corpo del ciclo prima di controllare la guardia (do-while), altri invece controllano la guardia prima di eseguire il corpo del ciclo (while)

Ci sono due tipi di iterazione:

- **Iterazione determinata**: è possibile determinare a priori quante volte si esegue il ciclo (soliltamente il for).

  Questo tipo di iterazione contiene una variabile di ciclo con un valore iniziale e uno finale.

  Gli aspetti progettuali per implementare un'iterazione determinata sono:

  * Tipo di variabile di controllo e la sua visibilità nel ciclo.
  * Legalità della modifica della variabile di ciclo
  * Quando avviene la valutazione dei valori di riferimento della variabile di ciclo. Alcuni linguaggi valutano le espressioni una volta e bloccano il valore che verrà usato per ogni iterazione, altri invece valutano le espressioni ad ogni iterazione e questo rende l'iterazione indeterminata siccome il numero di iterazioni dipende dai valori di riferimento che possono cambiare ad ogni iterazione

  La sintassi di un tipico for è:

  $$\texttt{for}\ indice\ \texttt{:=}\ inizio\ \texttt{to}\ fine\ \texttt{by}\ passo\ \texttt{do}\ corpo$$

  dove:

  * $inizio$ è il valore di partenza
  * $fine$ è il valore di terminazione
  * $passo$ indica come cambia il valore di $indice$ ad ogni iterazione
- **Iterazione indeterminata**: non è possibile, in generale, determinare a priori quante volte si esegue il ciclo (solitamente il while).

  Questo tipo di iterazione ha come controllo una guardia logica che determina se continuare o terminare l'iterazione.

  Gli aspetti progettuali per implementare un'iterazione indeterminata sono:

  * Quando avviene il controllo (do-while o while-do)
  * Forma e tipo di guardia

4.6.5 Semantica dell'if

- **Semantica statica**:
  La semantica statica dell'if verifica che l'if sia ben formato, cioè se soddisfa gli eventuali vincoli. In questo caso i vincoli sono:

  - La guardia deve essere di tipo booleano
  - I rami devono essere ben formati

  Definiamo quindi la semantica statica per l'if:

– Gli identificatori liberi sono:

$$\mathrm{FI}(\texttt{if b then } c_1 \texttt{ else } c_2) = \mathrm{FI}(b) \cup \mathrm{FI}(c_1) \cup \mathrm{FI}(c_2)$$

– Gli identificatori definiti sono:

$$\mathrm{DI}(\texttt{if b then } c_1 \texttt{ else } c_2) = \mathrm{DI}(c_1) \cup \mathrm{DI}(c_2)$$

questo perchè nei blocchi c_1 e c_2 possono esserci delle dichiarazioni (da inserire nella sintassi)

– Regola per l'if ben formato (l'unico vincolo è che la guardia sia di tipo booleano):

$$\frac{\Delta \vdash b : \texttt{bool} \quad \Delta \vdash c_1 \quad \Delta \vdash c_2}{\Delta \vdash \texttt{if b then } c_1 \texttt{ else } c_2}$$

- **Semantica dinamica**:
La semantica dinamica dell'if descrive come si esegue un if, cioè come si eseguono i comandi a seconda della memoria. Di conseguenza **agisce solo su ciò che si deve eseguire** e **non cambia lo stato**.

Definiamo quindi la semantica dinamica per l'if:

– Regola per l'esecuzione dell'if: per eseguire un if bisogna valutare la guardia b fino al valore $t \in \texttt{bool}$.

$$\frac{\rho \vdash \langle b, \sigma\rangle \to_e^* \langle t, \sigma\rangle}{\rho \vdash \langle \texttt{if b then } c_1 \texttt{ else } c_2, \sigma\rangle \to \langle \texttt{if t then } c_1 \texttt{ else } c_2, \sigma\rangle}$$

– Assioma per il `true`: se la guardia è $\mathtt{true}$ allora si esegue il ramo c_1

$$\rho \vdash \langle \texttt{if true then } c_1 \texttt{ else } c_2, \sigma\rangle \to_c \langle c_1, \sigma\rangle$$

– Assioma per il `false`: se la guardia è $\mathtt{false}$ allora si esegue il ramo c_2

$$\rho \vdash \langle \texttt{if false then } c_1 \texttt{ else } c_2, \sigma\rangle \to_c \langle c_2, \sigma\rangle$$

Esempio 4.14. Consideriamo il seguente if:

```
if x = y then x := 5 else x := 6
```

Gli identificatori liberi sono $\{x, y\}$ e quindi dobbiamo fornire un ambiente e la memoria per poter eseguire l'if. Consideriamo ad esempio:

$$\begin{aligned}\rho &= [x \mapsto l_x, y \mapsto 2]\\ \sigma_1 &= [l_x \mapsto 3]\\ \sigma_2 &= [l_x \mapsto 2]\\ \Delta &= [x \mapsto \texttt{intloc}, y \mapsto \texttt{int}]\end{aligned}$$

Utilizziamo due memorie σ_1 e σ_2 per mostrare sia il caso then che il caso else.

- **Semantica statica**: Verifichiamo che l'if sia ben formato usando l'ambiente statico:

Per prima cosa verifichiamo che le espressioni e le dichiarazioni siano ben formate:

1. Guardia

$$\frac{\Delta \vdash x : \texttt{int} \quad \Delta \vdash y : \texttt{int}}{\Delta \vdash x = y : \texttt{bool}} \; \Delta(x) = \texttt{intloc} \wedge \Delta(y) = \texttt{int}$$

2. Ramo then

$$\frac{\Delta \vdash x : \texttt{int}}{\Delta \vdash x \texttt{ := } 5} \; \Delta(x) = \texttt{intloc}$$

3. Ramo else

$$\frac{\Delta \vdash x : \texttt{int}}{\Delta \vdash x \texttt{ := } 6} \; \Delta(x) = \texttt{intloc}$$

Dopo aver verificato tutte le componenti dell'if possiamo dire che è ben formato:

$$\frac{\Delta \vdash x \texttt{ = } y : \texttt{bool} \quad \Delta \vdash x \texttt{ := } 5 : \texttt{int} \quad \Delta \vdash x \texttt{ := } 6 : \texttt{int}}{\Delta \vdash \texttt{if } x \texttt{ = } y \texttt{ then } x \texttt{ := } 5 \texttt{ else } x \texttt{ := } 6}$$

- **Semantica dimanica**:
  - Utilizzando $\sigma_1 = [\lambda_x \mapsto 3]$:
    La regola che bisogna applicare è la seguente:

$$\frac{\rho \vdash \langle x \texttt{ = } y, \sigma_1 \rangle \rightarrow_e^* \langle t, \sigma_1 \rangle}{\begin{array}{c}\rho \vdash \langle \texttt{if } x \texttt{ = } y \texttt{ then } x \texttt{ := } 5 \texttt{ else } x \texttt{ := } 6, \sigma_1 \rangle \\ \rightarrow \rho \vdash \langle \texttt{if } t \texttt{ then } x \texttt{ := } 5 \texttt{ else } x \texttt{ := } 6, \sigma_1 \rangle\end{array}} \tag{8}$$

    Per applicare questa regola bisogna prima valutare la guardia `x = y`:

    1. Valutiamo `x`:

$$\frac{\rho \vdash \langle x, \sigma_1 \rangle \rightarrow \langle 3, \sigma_1 \rangle}{\rho \vdash \langle x \texttt{ = } y, \sigma_1 \rangle \rightarrow \langle 3 = y, \sigma_1 \rangle} \; \rho(x) = l_x \wedge \sigma_1(l_x) = 3$$

    2. Valutiamo `y`:

$$\frac{\rho \vdash \langle y, \sigma_1 \rangle \rightarrow \langle 2, \sigma_1 \rangle}{\rho \vdash \langle 3 = y, \sigma_1 \rangle \rightarrow \langle 3 = 2, \sigma_1 \rangle} \; \rho(y) = 2$$

    3. Valutiamo `3 = 2`:

$$\rho \vdash \langle 3 = 2, \sigma_1 \rangle \rightarrow \langle \mathit{false}, \sigma_1 \rangle$$

Ora che abbiamo valutato la guardia possiamo completare l'esecuzione dell'if applicando la regola 8:

$$\frac{\rho \vdash \langle \texttt{x = y}, \sigma_1 \rangle \rightarrow_e^* \langle \mathit{false}, \sigma_1 \rangle}{\begin{array}{c}\rho \vdash \langle \texttt{if x = y then x := 5 else x := 6}, \sigma_1 \rangle \\ \rightarrow \rho \vdash \langle \texttt{if } \mathit{false} \texttt{ then x := 5 else x := 6}, \sigma_1 \rangle\end{array}}$$

Applichiamo l'assioma per il `false`:

$$\rho \vdash \langle \texttt{if } \mathit{false} \texttt{ then x := 5 else x := 6}, \sigma_1 \rangle \rightarrow_c \langle \texttt{x := 6}, \sigma_1 \rangle$$

Infine eseguiamo `x := 6`:

$$\rho \vdash \langle \texttt{x := 6}, \sigma_1 \rangle \rightarrow_c \sigma_1[l_x \mapsto 6] = [l_x \mapsto 6]$$

– Utilizzando $\sigma_2 = [\lambda_x \mapsto 2]$: La regola che bisogna applicare è la regola 8. Per applicarla bisogna prima valutare la guardia `x = y`:

1. Valutiamo `x`:

$$\frac{\rho \vdash \langle x, \sigma_2 \rangle \rightarrow \langle 2, \sigma_2 \rangle}{\rho \vdash \langle \texttt{x = y}, \sigma_2 \rangle \rightarrow \langle 2 = y, \sigma_2 \rangle} \; \rho(x) = l_x \wedge \sigma_2(l_x) = 2$$

2. Valutiamo `y`:

$$\frac{\rho \vdash \langle y, \sigma_2 \rangle \rightarrow \langle 2, \sigma_2 \rangle}{\rho \vdash \langle 2 = y, \sigma_2 \rangle \rightarrow \langle 2 = 2, \sigma_2 \rangle} \; \rho(y) = 2$$

3. Valutiamo `2 = 2`:

$$\rho \vdash \langle 2 = 2, \sigma_2 \rangle \rightarrow \langle \mathit{true}, \sigma_2 \rangle$$

Ora che abbiamo valutato la guardia possiamo completare l'esecuzione dell'if applicando la regola 8:

$$\frac{\rho \vdash \langle \texttt{x = y}, \sigma_2 \rangle \rightarrow_e^* \langle \mathit{true}, \sigma_2 \rangle}{\begin{array}{c}\rho \vdash \langle \texttt{if x = y then x := 5 else x := 6}, \sigma_2 \rangle \\ \rightarrow \rho \vdash \langle \texttt{if } \mathit{true} \texttt{ then x := 5 else x := 6}, \sigma_2 \rangle\end{array}}$$

Applichiamo l'assioma per il `true`:

$$\rho \vdash \langle \texttt{if } \mathit{true} \texttt{ then x := 5 else x := 6}, \sigma_2 \rangle \rightarrow_c \langle \texttt{x := 5}, \sigma_2 \rangle$$

Infine eseguiamo `x := 5`:

$$\rho \vdash \langle \texttt{x := 5}, \sigma_2 \rangle \rightarrow_c \sigma_2[l_x \mapsto 5] = [l_x \mapsto 5]$$

4.6.6 Semantica della composizione

- **Semantica statica**

- Gli identificatori liberi sono:

$$\mathrm{FI}(c_1; c_2) = \mathrm{FI}(c_1) \cup \mathrm{FI}(c_2)$$

- Gli identificatori definiti sono:

$$\mathrm{DI}(c_1; c_2) = \mathrm{DI}(c_1) \cup \mathrm{DI}(c_2)$$

- Regola per la composizione ben formata:

$$\frac{\Delta \vdash c_1 \quad \Delta \vdash c_2}{\Delta \vdash c_1; c_2}$$

- **Semantica dinamica**

  - Regola per l'esecuzione di c_1: si esegue completamente c_1 che restituisce una nuova memoria σ_1 e poi partendo da σ_1 si esegue c_2

$$\frac{\rho \vdash \langle c_1, \sigma\rangle \rightarrow_c^* \sigma_1}{\rho \vdash \langle c_1; c_2, \sigma\rangle \rightarrow_c \langle c_2, \sigma_1\rangle}$$

4.6.7 Semantica del while

- **Semantica statica**:

  - Gli identificatori liberi sono:

$$\mathrm{FI}(\texttt{while } e \texttt{ do } c) = \mathrm{FI}(e) \cup \mathrm{FI}(c)$$

  - Gli identificatori definiti sono:

$$\mathrm{DI}(\texttt{while } e \texttt{ do } c) = \mathrm{DI}(c)$$

  - Regola per il while ben formato:

$$\frac{\Delta \vdash e : \texttt{bool} \quad \Delta \vdash c}{\Delta \vdash \texttt{while } e \texttt{ do } c}$$

- **Semantica dinamica**:

  - Regola per la valutazione della guardia: questa regola determina la **posizione del controllo della guardia** e se **verrà rivalutata ad ogni iterazione**. In questo caso viene valutata prima la guardia

$$\frac{\rho \vdash \langle e, \sigma\rangle \rightarrow_e^* \langle t, \sigma\rangle}{\rho \vdash \langle \texttt{while } e \texttt{ do } c, \sigma\rangle \rightarrow_c \langle \texttt{while t do } c, \sigma\rangle}$$

  - Assioma per il `false`: se la guardia è $\mathtt{false}$ allora si termina l'iterazione e quindi non si esegue il corpo del ciclo

$$\rho \vdash \langle \texttt{while false do } c, \sigma\rangle \rightarrow_c \sigma$$

– Assioma per il `true`: se la guardia è $true$ allora si esegue il corpo del while c e poi si riesegue il ciclo while con la stessa guardia che verrà rivalutata ad ogni iterazione

$$\rho \vdash \langle \texttt{while true do c}, \sigma\rangle \to_c \langle \texttt{c; while } e \texttt{ do c}, \sigma\rangle$$

Questa regola non è scritta correttamente siccome l'espressione e non è presente nelle premesse, quindi riscriviamo tutte le regola in una forma più corretta fondendo la regola di valutazione con i due assiomi

– Regola completa per il while con espressione `false`:

$$\frac{\rho \vdash \langle e, \sigma\rangle \to_e^* \langle false, \sigma\rangle}{\rho \vdash \langle \texttt{while } e \texttt{ do c}, \sigma\rangle \to_c \sigma}$$

– Regola completa per il while con espressione `true`:

$$\frac{\rho \vdash \langle e, \sigma\rangle \to_e^* \langle true, \sigma\rangle}{\rho \vdash \langle \texttt{while } e \texttt{ do c}, \sigma\rangle \to_c \langle \texttt{c; while } e \texttt{ do c}, \sigma\rangle}$$

Esempio 4.15. Consideriamo il seguente while:

```
while not x = 0 do x := x - 1
```

L'identificatore libero è $\{x\}$ quindi dobbiamo fornire un ambiente e una memoria per poter eseguire il while. Consideriamo ad esempio:

$$\rho = [x \mapsto l_x]$$
$$\Delta = [x \mapsto \texttt{intloc}]$$
$$\sigma = [l_x \mapsto 2]$$

- **Semantica statica**:
  Verifichiamo che le espressioni e le dichiarazioni siano ben formate:

  1. Guardia
  $$\frac{\Delta \vdash x : \texttt{int}}{\Delta \vdash \texttt{not x = 0} : \texttt{bool}}\ \Delta(x) = \texttt{intloc}$$
  2. Corpo del ciclo
  $$\frac{\Delta \vdash \texttt{x - 1} : \texttt{int}}{\Delta \vdash \texttt{x := x - 1}}\ \Delta(x) = \texttt{intloc}$$

  Dopo aver verificato tutte le componenti del while possiamo dire che è ben formato:

  $$\frac{\Delta \vdash \texttt{not x = 0} : \texttt{bool} \quad \Delta \vdash \texttt{x := x - 1}}{\Delta \vdash \texttt{while not x = 0 do x := x - 1}}$$

- **Semantica dinamica**:

1. Primo ciclo:
   Per prima cosa bisogna valutare la guardia `not x = 0` per capire se eseguire o terminare il ciclo:

   (a) Valutiamo `not x = 0`:

   $$\frac{\rho \vdash \langle x, \sigma\rangle \to_e \langle 2, \sigma\rangle}{\rho \vdash \langle \texttt{x = 0}, \sigma\rangle \to_e \langle \texttt{2 = 0}, \sigma\rangle}\ \rho(x) = l_x \wedge \sigma(l_x) = 2$$
   $$\rho \vdash \langle \texttt{not x = 0}\rangle \to_e \langle \texttt{not 2 = 0}, \sigma\rangle$$

   (b) Valutiamo `not 2 = 0` usando l'assioma:

   $$\rho \vdash \langle \texttt{2 = 0}, \sigma\rangle \to_e \langle \texttt{false}, \sigma\rangle$$
   $$\rho \vdash \langle \texttt{not 2 = 0}, \sigma\rangle \to_e \langle \texttt{true}, \sigma\rangle$$

   Ora che abbiamo valutato la guardia, sappiamo che il ciclo deve essere eseguito e quindi applichiamo la regola per il while con espressione `true`:

   $$\frac{\rho \vdash \langle \texttt{not x = 0}, \sigma\rangle \to_e^* \langle \texttt{true}, \sigma\rangle}{\rho \vdash \langle \texttt{while not x = 0 do x := x - 1}, \sigma\rangle \to_c \langle \texttt{x := x - 1; while not x = 0 do x := x - 1}, \sigma\rangle}$$

   Cioè si esegue `x := x - 1` e poi si riesegue il ciclo while con la stessa guardia che verrà rivalutata su una nuova memoria σ_1 che sarà il risultato dell'esecuzione di `x := x - 1`:

   $$\frac{\rho \vdash \langle \texttt{x := x - 1}, \sigma\rangle \to_c^* \sigma_1}{\rho \vdash \langle \texttt{x := x - 1; while not x = 0 do x := x - 1}, \sigma\rangle \to_c \langle \texttt{while not x = 0 do x := x - 1}, \sigma_1\rangle}$$

   Valutiamo quindi `x := x - 1` usando la seguente regola:

   $$\frac{\rho \vdash \langle \texttt{x - 1}, \sigma\rangle \to_e^* \langle n, \sigma\rangle}{\rho \vdash \langle \texttt{x := x - 1}, \sigma\rangle \to_c \langle \texttt{x := } n, \sigma\rangle} \tag{9}$$

   Per applicare questa regola bisogna prima valutare l'espressione `x - 1`:

   (a) Valutiamo `x`:

   $$\frac{\rho \vdash \langle x, \sigma\rangle \to_e \langle 2, \sigma\rangle}{\rho \vdash \langle \texttt{x - 1}, \sigma\rangle \to_e \langle 2 - 1, \sigma\rangle}\ \sigma(\rho(x)) = 2$$

   (b) Applichiamo l'assioma:

   $$\rho \vdash \langle 2 - 1, \sigma\rangle \to_e \langle 1, \sigma\rangle$$

Applichiamo la regola 9:

$$\frac{\rho \vdash \langle \texttt{x - 1}, \sigma\rangle \to_e^* \langle 1, \sigma\rangle}{\rho \vdash \langle \texttt{x := x - 1}, \sigma\rangle \to_c \langle \texttt{x := 1}, \sigma\rangle}$$

L'esecuzione di `x := 1` restituisce una nuova memoria σ_1:

$$\rho \vdash \langle \texttt{x := 1}, \sigma\rangle \to_c \sigma[l_x \mapsto 1] = [l_x \mapsto 1] = \sigma_1$$

2. Secondo ciclo:
   Ora bisogna rieseguire il while con la nuova memoria σ_1. Partiamo rivalutando la guardia `not x = 0`:

   (a) Valutiamo `not x = 0`:

   $$\frac{\dfrac{\rho \vdash \langle x, \sigma_1\rangle \to_e \langle 1, \sigma_1\rangle}{\rho \vdash \langle \texttt{x = 0}, \sigma_1\rangle \to_e \langle 1 = 0, \sigma_1\rangle}\ \sigma_1(\rho(x)) = 1}{\rho \vdash \langle \texttt{not x = 0}, \sigma_1\rangle \to_e \langle \texttt{not 1 = 0}, \sigma_1\rangle}$$

   (b) Valutiamo `not 1 = 0`:

   $$\frac{\rho \vdash \langle \texttt{1 = 0}, \sigma_1\rangle \to_e \langle \texttt{false}, \sigma_1\rangle}{\rho \vdash \langle \texttt{not 1 = 0}, \sigma_1\rangle \to_e \langle \texttt{true}, \sigma_1\rangle}$$

   Il ciclo va eseguito di nuovo e quindi applichiamo la regola per il while con espressione `true`:

   $$\frac{\rho \vdash \langle \texttt{not x = 0}, \sigma_1\rangle \to_e^* \langle \texttt{true}, \sigma_1\rangle}{\begin{array}{r}\rho \vdash \langle \texttt{while not x = 0 do x := x - 1}, \sigma_1\rangle \to_c \\ \langle \texttt{x := x - 1; while not x = 0 do x := x - 1}, \sigma_1\rangle\end{array}}$$

   L'eseguzione di `x := x - 1` restituisce una nuova memoria σ_2:

   $$\frac{\rho \vdash \langle \texttt{x := x - 1}, \sigma_1\rangle \to_c \sigma_2}{\begin{array}{r}\rho \vdash \langle \texttt{x := x - 1; while not x = 0 do x := x - 1}, \sigma_1\rangle \\ \to_c \langle \texttt{while not x = 0 do x := x - 1}, \sigma_2\rangle\end{array}}$$

   Valutiamo `x := x - 1` sulla memoria σ_1 usando la regola 9:

   (a) Valutiamo `x`:

   $$\frac{\rho \vdash \langle x, \sigma_1\rangle \to_e \langle 1, \sigma_1\rangle}{\rho \vdash \langle \texttt{x - 1}, \sigma_1\rangle \to_e \langle 1 - 1, \sigma_1\rangle}\ \sigma_1(\rho(x)) = 1$$

   (b) Applichiamo l'assioma:

   $$\rho \vdash \langle 1 - 1, \sigma_1\rangle \to_e \langle 0, \sigma_1\rangle$$

Applichiamo la regola 9:

$$\frac{\rho \vdash \langle \texttt{x - 1}, \sigma_1 \rangle \to_e^* \langle 0, \sigma_1 \rangle}{\rho \vdash \langle \texttt{x := x - 1}, \sigma_1 \rangle \to_c \langle \texttt{x := 0}, \sigma_1 \rangle}$$

L'esecuzione di `x := 0` restituisce una nuova memoria σ_2:

$$\rho \vdash \langle \texttt{x := 0}, \sigma_1 \rangle \to_c \sigma_1[l_x \mapsto 0] = [l_x \mapsto 0] = \sigma_2$$

3. Terzo ciclo:
   Ora bisogna rieseguire il while con la nuova memoria σ_2. Partiamo rivalutando la guardia `not x = 0`:

   (a) Valutiamo `not x = 0`:

   $$\frac{\dfrac{\rho \vdash \langle x, \sigma_2 \rangle \to_e \langle 0, \sigma_2 \rangle}{\rho \vdash \langle \texttt{x = 0}, \sigma_2 \rangle \to_e \langle 0 = 0, \sigma_2 \rangle} \; \sigma_2(\rho(x)) = 0}{\rho \vdash \langle \texttt{not x = 0}, \sigma_2 \rangle \to_e \langle \texttt{not 0 = 0}, \sigma_2 \rangle}$$

   (b) Valutiamo `not 0 = 0`:

   $$\frac{\rho \vdash \langle \texttt{0 = 0}, \sigma_2 \rangle \to_e \langle \texttt{true}, \sigma_2 \rangle}{\rho \vdash \langle \texttt{not 0 = 0}, \sigma_2 \rangle \to_e \langle \texttt{false}, \sigma_2 \rangle}$$

   La guardia è $false$ quindi si termina l'iterazione, di conseguenza applichiamo la regola per il while con espressione `false`:

   $$\frac{\rho \vdash \langle \texttt{not x = 0}, \sigma_2 \rangle \to_e^* \langle \texttt{false}, \sigma_2 \rangle}{\rho \vdash \langle \texttt{while not x = 0 do x := x - 1}, \sigma_2 \rangle \to_c \sigma_2}$$

   Cioè il ciclo termina e restituisce la memoria σ_2 inalterata:

   $$\rho \vdash \langle \texttt{while not x = 0 do x := x - 1}, \sigma_2 \rangle \to_c \sigma_2 = [l_x \mapsto 0]$$

4.7 Blocco di codice

Bisogna gestire come viene creato l'ambiente in cui si vuole eseguire un codice e quindi aggiungere **un'integrazione tra memoria e ambiente**. Per fare ciò introduciamo il **blocco**, cioè una porzione di programma potenzialmente isolata da altri pezzi di codice. I blocchi:

- Permettono di rendere modulare il codice e quindi di renderlo più chiaro
- Permettono la gestione locale degli identificatori
- Permettono di gestire l'allocazione della memoria
- Permettono la ricorsione

Gli aspetti implementativi dei blocchi sono:

- Visibilità delle dichiarazioni, quando una dichiarazione è visibile, ad esempio solo dopo la dichiarazione o in tutto il programma
- Ordine delle dichiarazioni

4.7.1 Blocco inline

Un blocco inline è un blocco senza nome che specifica le dichiarazioni D che definiscono l'ambiente in cui eseguire i comandi C. Il blocco inline è quindi:

$$\mathrm{D}; \mathrm{C}$$

Questa definizione non ha limitatori, ma in alcuni linguaggi i blocchi sono delimitati da parentesi graffe {blocco}. Aggiungiamo quindi i blocchi alla grammatica dei comandi:

$$\begin{aligned} \mathrm{Com}: \ \mathrm{C} \rightarrow\ & \texttt{skip} \mid \mathrm{x} := e \\ & \mid \mathrm{C}; \mathrm{C} \\ & \mid \texttt{while B do C} \quad \mid \texttt{if B then C else C} \\ & \mid \mathrm{D}; \mathrm{C} \end{aligned}$$

Siccome nel nostro linguaggio i blocchi fanno parte dei comandi è possibile inserire dichiarazioni in qualunque punto del programma. Con questa sintassi (e con la semantica che verrà specificata) una dichiarazione è visibile solo da dove viene inserita fino alla fine del programma (perchè non ha limitatori).

4.7.2 Blocchi annidati

I blocchi annidati devono essere **completamente contenuti**, cioè un blocco non può essere parzialmente contenuto in un altro blocco, ad esempio:

- Il blocco 2 è completamente contenuto nel blocco 1

```
{ // Blocco 1 aperto
  ...
  [ // Blocco 2 aperto
  ...
  ] // Blocco 2 chiuso
} // Blocco 1 chiuso
```

- Il blocco 2 è parzialmente contenuto nel blocco 1 e quindi non è valido

```
{ // Blocco 1 aperto
  ...
  [ // Blocco 2 aperto
  ...
} // Blocco 1 chiuso
  ...
  ] // Blocco 2 chiuso
```

Questa regola di sovrapposizione tra blocchi permette di definire la regola di visibilita delle dichiarazioni.

4.7.3 Regola di visitbilità delle dichiarazioni

Una dichiarazione locale ad un blocco è visibile solo all'interno di quel blocco e in tutti i blocchi annidati, a meno che in tali blocchi non intervenga una dichiarazione dello stesso nome.

Esempio 4.16. Consideriamo il seguente esempio:

```
{ // Blocco 1
  const i: int = 3;
  { // Blocco 2
    var i: real = 1.2;
    i := i+1;
  } // Blocco 2
  i := i+1;
} // Blocco 1
```

Il blocco 2 è detto **scope di** `var i`, mentre il blocco 1 è detto **scope di** `const i`.

Esempio 4.17. Consideriamo il seguente esempio:

```
{ // A
  int a = 1;

  { // B
    int b = 2;
    int c = 2;

    { // C
      int c = 3;
      int d;

      d = a + b + c;
      write(d);
    }

    { // D
      int e;
      e = a + b + c;
      write(e);
    }
  }
}
```

- **Blocco A**:
  Se il blocco `A` rappresenta tutto il programma, allora la variabile `a` è detta **variabile globale**, cioè visibile a tutti i blocchi essendo tutti i blocchi annidati in `A`. L'unica variabile visibile al blocco `A` è `a`.
  - **Blocco B**:
    Le variabili nel blocco `B` sono dette **variabili locali** e il blocco `B` vede le sue dichiarazioni locali e quelle del blocco `A`.
    - **Blocco C**:
      Nel blocco `C` la variabile `c` è una variabile locale che viene ridefinita, cioè **nasconde dentro** `c` **la precedente dichiarazione** (shadowing)
    - **Blocco D**:
      Nel blocco `D` ci sono solo variabili locali che non nascondono

nessuna dichiarazione precedente. Il valore di c che il blocco D vede è quello dichiarato nel blocco esterno B siccome **la visibilità può andare solo all'interno del blocco**.

4.7.4 Scope

Lo scope di una dichiarazione di x è la porzione di codice in cui tutte le occorrenze di x fanno riferimento alla dichiarazione tramite il legame definito dalla dichiarazione. La **regola di visibilità definisce lo scope**, ovvero la porzione di codice in cui le occorrenze della variabile di chiarata fanno riferimento alla dichiarazione. Lo scope è un concetto **statico** che lega un identificatore all'ambiente di riferimento a **tempo di compilazione** in quanto (senza l'uso di procedure) **eseguiamo il codice dove viene definito**.

Una **variabile** può avere una dichiarazione:

- **Globale**: quando è visibile a tutti i blocchi, quindi all'intero programma
- **Rispetto al blocco** può essere:
  - **Locale**: quando è inserita nel blocco che sovrascrive eventuali variabili esterne
  - **Non locale** (globali incluse): quando è visibile al blocco, ma posizionata in blocchi esterni

e un **ambiente** può essere:

- **Globale**: visibile a tutti i blocchi
- **Rispetto al blocco** può essere:
  - **Locale**: definito nel blocco e non visibile esternamente
  - **Non locale**: ereditato dai blocchi esterni

Lo scoping è un concetto che riguarda porzioni di codice e determina a quali legami (dell'ambiente) fa riferimento ogni occorrenza.

4.7.5 Tempo di vita

Ogni variabile ha un tempo di vita (lifetime), ovvero il tempo di esecuzione in cui l'associazione della locazione (memoria) rimane valida. Il tempo di vita è delimitato da:

- Allocazione in memoria
- Deallocazione

In alcuni lingauggi se l'allocazione e deallocazione non è esplicita, allora il tempo di vita coincide con il tempo di esecuzione del programma.

Esempio 4.18. Consideriamo il seguente programma

```
{ // Blocco 1           // -----------+ Scope    --+
  const i: int = 3;     // -----------+ const i    |
  { // Blocco 2         // -+                       |
    var i: real = 1.2; //  | Scope + lifetime     | Lifetime
    i := i+1;          //  | var i                | const i
  } // Blocco 2         // -+                       |
  i := i+1;             // -----------+ Scope      |
} // Blocco 1           // -----------+ const i --+
```

- Lo scoping è un **concetto statico** nel senso che è completamente determinabile dal codice fissate delle regole (ad esempio di visibilità)
- Il tempo di vita è un **concetto dinamico** nel senso che dipende dall'esecuzione del programma

Esempio 4.19. Consideriamo il seguente codice:

```
{
  D_0 dich.
  while ...
   {
     D_1 dich.
     C_11 comandi
     {
        D_2 dich.
        C_12 comandi
     }
     C_13 comandi
   }
}
```

Utilizziamo uno stack per rappresentare gli ambienti:

1. Nel primo blocco viene creato l'ambiente ρ_0 che contiene le dichiarazioni D_0:

   $$[\rho_0]$$
   $$\uparrow$$

   (a) All'interno del while viene creato l'ambiente ρ_1 che contiene le dichiarazioni D_1:

   $$[\rho_0, \rho_1]$$
   $$\uparrow$$

   i. All'interno del terzo blocco viene creato l'ambiente ρ_2 che contiene le dichiarazioni D_2:

   $$[\rho_0, \rho_1, \rho_2]$$
   $$\uparrow$$

   Il tempo di vita delle dichiarazioni D_2 è la durata dell'ambiente ρ_2 nello stack.

ii. All'uscita del blocco più interno D_2 viene rimosso l'ambiente ρ_2 dallo stack e quindi termina il tempo di vita delle dichiarazioni D_2 del **primo ciclo**:

$$[\rho_0, \rho_1]$$
$$\uparrow$$

Il tempo di vita delle dichiarazioni D_1 è la durata dell'ambiente ρ_1 nello stack.

iii. Ulteriori cicli sono analoghi alla ripetizione dei due punti precedenti. Allocazioni di ρ_1 e ρ_2 successive nello stack durante l'esecuzione del while sono di fatto **nuovi ambienti diversi** perchè sono stati "distrutti" e riallocati ad ogni iterazione.

2. All'uscita del while viene rimosso l'ambiente ρ_1 dallo stack e quindi termina il tempo di vita delle dichiarazioni D_1:

$$[\rho_0]$$
$$\uparrow$$

Il tempo di vita delle dichiarazioni D_0 è la durata dell'ambiente ρ_0 nello stack, che coincide con l'esecuzione del programma.

4.7.6 Semantica dei blocchi

Definiamo la semantica dei blocchi con le seguenti regole:

- **Semantica statica**:
  - Identificatori liberi:

$$FI(d; c) = FI(d) \cup (FI(c) \setminus DI(d))$$

  - Identificatori dichiarati:

$$DI(d; c) = DI(d) \cup DI(c)$$

  - Regola per un blocco ben formato:

$$\frac{\Delta \vdash d : \Delta_0 \qquad \Delta\,[\Delta_0] \vdash c}{\Delta \vdash d; c}$$

- **Semantica dinamica**:
  - Regola per l'esecuzione delle dichiarazioni di un blocco:

$$\frac{\rho \vdash \langle d, \sigma\rangle \rightarrow_c^* \langle \rho_0, \sigma'\rangle}{\rho \vdash \langle d; c, \sigma\rangle \rightarrow_c \langle \rho_0; c, \sigma'\rangle}$$

  - Regola per l'esecuzione di un blocco:

$$\frac{\rho[\rho_0] \vdash \langle c, \sigma\rangle \rightarrow_c^* \sigma'}{\rho \vdash \langle \rho_0; c, \sigma\rangle \rightarrow_c \sigma'}$$

Esempio 4.20. Consideriamo il seguente blocco:

$$\underbrace{\texttt{var x: int = 3}}_{\mathrm{d}}\texttt{;}\underbrace{\texttt{x := x+1;}}_{\mathrm{c}}$$

- **Semantica statica**: Verifichiamo che il blocco sia ben formato usando la seguente formula:

$$\frac{\vdash \mathrm{d} : \Delta \qquad \Delta \vdash \mathrm{c}}{\vdash \mathrm{d};\mathrm{c}} \tag{10}$$

Per poterla applicare bisogna prima controllare che entrambe le dichiarazioni siano ben formate:

1. Controlliamo che la dichiarazione d sia ben formata:

$$\frac{\vdash 3 : \texttt{int}}{\vdash \texttt{vad x: int = 3} \rightarrow [\mathrm{x} \mapsto \texttt{intloc}] = \Delta}$$

2. Controlliamo che la dichiarazione c sia ben formata, per farlo bisogna prima verificare che sia ben formata l'espressione `x+1`:

   (a) Verifichiamo `x+1`:

$$\frac{[\mathrm{x} \mapsto \texttt{intloc}] \vdash \mathrm{x} : \texttt{int} \qquad \Delta \vdash 1 : \texttt{int}}{[\mathrm{x} \mapsto \texttt{intloc}] \vdash \mathrm{x} + 1 : \texttt{int}}\ \Delta(\mathrm{x}) = \texttt{intloc}$$

   Ora possiamo verificare che c sia ben formata:

$$\frac{[\mathrm{x} \mapsto \texttt{intloc}]\, \mathrm{x} + 1 : \texttt{int}}{[\mathrm{x} \mapsto \texttt{intloc}] \vdash \texttt{x := x+1}}\ \Delta(\mathrm{x}) = \texttt{intloc}$$

Ora che abbiamo controllato che le dichiarazioni sono ben formate possiamo applicare la regola 10:

$$\frac{\vdash \mathrm{d} : [\mathrm{x} \mapsto \texttt{intloc}] \qquad [\mathrm{x} \mapsto \texttt{intloc}] \vdash \mathrm{c}}{\vdash \mathrm{d};\mathrm{c}}$$

Abbiamo quindi mostrato che il blocco è ben formato

- **Semantica dinamica**: Eseguiamo il blocco partendo da una memoria iniziale vuota.

1. Per prima cosa eseguiamo la dichiarazione variabile d:

$$\rho \vdash \langle \texttt{var x: int = 3}, \varnothing \rangle \rightarrow_{\mathrm{c}} \langle [\mathrm{x} \mapsto \mathrm{l}_{\mathrm{x}}], \varnothing\, [\mathrm{l}_{\mathrm{x}} \mapsto 3] = \sigma_1 \rangle$$

2. Ora eseguiamo il comando c usando la nuova memoria σ_1, ma per prima cosa bisogna valutare l'espressione `x+1`:

(a) Valutiamo x+1:

$$\frac{\rho \vdash \langle x, \sigma_1\rangle \to_e \langle 3, \sigma_1\rangle \qquad \rho \vdash \langle 1, \sigma_1\rangle}{\rho \vdash \langle x+1, \sigma_1\rangle \to_e \langle 3+1, \sigma_1\rangle}\ \sigma_1(\rho(x)) = 3$$

$$\rho \vdash \langle 3+1, \sigma_1\rangle \to_e \langle 4, \sigma_1\rangle$$

Ora possiamo eseguire il comando c:

$$\frac{\rho \vdash \langle x+1, \sigma_1\rangle \to_e^* \langle 4, \sigma_1\rangle}{\rho \vdash \langle x \ := \ x{+}1, \sigma_1\rangle \to_c \langle x \ := \ 4, \sigma_1\rangle}$$

Applichiamo l'assioma per l'assegnamento:

$$\rho \vdash \langle x \ := \ 4, \sigma_1\rangle \to_c \sigma_1[l_x \mapsto 4] = [l_x \mapsto 4] = \sigma_2$$

Dopo aver eseguito d e c possiamo eseguire il blocco completo:

1. Eseguiamo prima d nel blocco:

$$\frac{\rho \vdash \langle d, \varnothing\rangle \to_c^* \langle [x \mapsto l_x], \varnothing\,[l_x \mapsto 3]\rangle}{\rho \vdash \langle d; c, \varnothing\rangle \to_c \langle [x \mapsto l_x]; c, \varnothing\,[l_x \mapsto 3]\rangle}$$

2. Infine eseguiamo il blocco completo:

$$\frac{\rho[x \mapsto l_x] \vdash \langle c, [l_x \mapsto 3]\rangle \to_c^* [l_x \mapsto 4]}{\rho \vdash \langle [x \mapsto l_x]; c, [l_x \mapsto 3]\rangle \to_c [l_x \mapsto 4]}$$

4.8 Astrazione del controllo (Procedure)

L'obiettivo è quello di **astrarre il controllo**, cioè nascondere dei dettagli implementativi fornendo al programmatore un'interfaccia più semplice da utilizzare. Per fare ciò bisogna **identificare cosa è necessario descrivere** e mantenere pubblico e cosa si può nascondere, in modo da poter focalizzare la programmazione sulle questioni rilevanti. Questa astrazione permette di:

- Ottimizzare l'implementazione
- Ridurre gli errori
- Migliorare la leggibilità

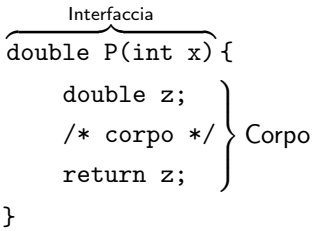
L'astrazione del controllo **introduce dei legami**, cioè dei **blocchi di codice con un nome**, che vanno gestiti separando la definizione dall'esecuzione.

Gli aspetti principali dell'astrazione del controllo sono:

- **Nome**: si riferisce ad un **insieme di comandi**, ma può essere usato in vari modi. Essenzialmente il nome identifica un blocco di codice che **esegue sempre nello stesso modo**
- **Parametri**: permettono di creare un "ponte" tra l'ambiente che **chiama** e l'ambiente **eseguito** che permette di passare informazioni tra i due ambienti. Essenzialmente permettono di usare un pezzo di codice su valori (input) differenti ed eseguire computazioni simili usando lo stesso nome.

Il nome, i parametri e il **tipo di risultato** rappresentano **l'interfaccia** di un blocco di codice, che è la parte pubblica esposta al programmatore. La parte nascosta dell'astrazione è il **corpo della procedura**, cioè il codice eseguito.

Esempio 4.21. Un esempio di procedura, un tipo di astrazione del controllo, è la seguente:

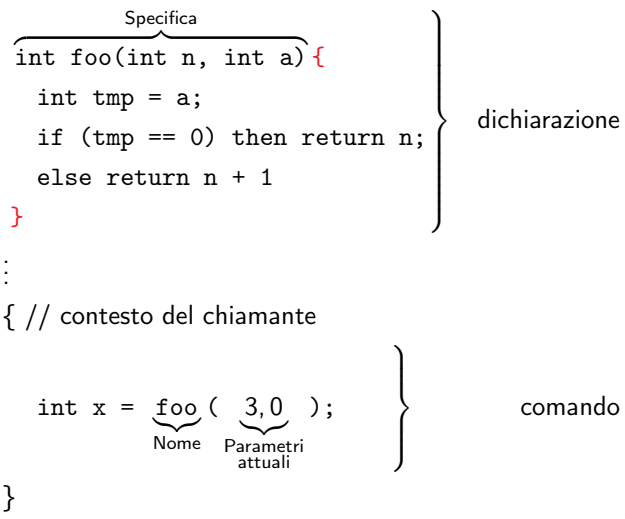


Per definire procedure il linguaggio di programmazione deve avere strumenti sintattici per:

- Specificare (definire) la procedura (interfaccia)
- Scrivere il corpo della procedura (corpo), che insieme all'interfaccia rappresenta la **definizione completa di una procedura**
- Utilizzare la procedura, cioè chiamare l'esecuzione del corpo della procedura

La definizione viene fatta tramite **dichiarazioni**, mentre l'utilizzo viene fatto tramite **comandi**.

Esempio 4.22. Consideriamo il seguente codice:



- Il blocco definisce un ambiente locale che contiene:
  - Dichiarazioni interne (ad esempio `tmp`)
  - Parametri formali (`n` e `a`)

- I parametri attuali sono valori con cui si vogliono inizializzare i parametri formali della procedura
- Il tipo della procedura e il tipo del risultato devono coincidere sia nella definizione che nell'utilizzo

4.8.1 Ambiente di riferimento di una procedura

L'ambiente di riferimento di una procedura è determinato da:

- **Regole di visibilità**: standard per l'annidamento
- **Regole di scoping**: specificano dove risolvere ambienti non locali di codice eseguito su contesti diversi da quello di definizione
  - **Regole di binding**: risolve regole di scoping ambigue, ad esempio nel caso di una funzione come parametro ad un'altra funzione
- **Regole di passaggio di parametri**
- **Regole specifiche del linguaggio**

4.8.2 Regole di scoping

Consideriamo il seguente codice:

```
int foo(int n){
  int tmp = a;
  if (tmp == 0) then return n;
  else return n + 1
}
```

In questo caso la variabile a è un **identificatore non locale**, cioè un identificatore libero per cui si deve trovare il significato nel contesto. Il problema è che nel contesto del chiamante potrebbero esserci più definizioni di a e quindi bisogna gestire quale utilizzare. Di conseguenza la regola di visibilità delle dichiarazioni 4.7.3 non è più sufficiente a risolvere i riferimenti quando si usano le procedure se la procedura ha un ambiente non locale.

Esempio 4.23. Nel seguente codice la variabile x è dichiarata correttamente, ma non sappiamo a quale dichiarazione la funzione fa riferimento:

```
{
  int x = 3;
  int foo() {
    return x;
  }

  {
    int x = 5;
    int res = foo(); // a quale x fa riferimento foo()?
  }
```

```
}
```

La risoluzione di questo problema è gestita diversamente da ogni linguaggio di programmazione. Per risolverlo va definita la **regola di scoping** che può essere:

- **Statico**: l'ambiente è quello della definizione
- **Dinamico**: l'ambiente è quello della chiamata

Le regole di scoping quindi servono per risolvere riferimenti **non locali** presenti in procedure, ovvero dove il contesto di definizione può non coincidere con il contesto di esecuzione.

4.8.3 Scoping statico

Esempio 4.24. Consideriamo il seguente codice:

```
{
  int x = 10; // x_1
  void foo() {
    // x non e' un parametro formale
    // e non e' dichiarata nel blocco
    x++; // Ambiente non locale
  }
  void fie() {
    int x = 0; // x_2
    foo();
  }

  fie();
}
```

Non possiamo sapere se alla fine si ottiene `x = 1` (scoping dinamico) o `x = 11` (scoping statico).

- Scoping statico: si sceglie l'ambiente di definizione a tempo di compilazione, quindi utilizza la dichiarazione di `x` nello scope della definizione della proceduta
- Scoping dinamico: si sceglie l'ambiente di chiamata a tempo di esecuzione, quindi utilizza la dichiarazione di `x` nello scope della chiamata

Consideriamo il seguente codice:

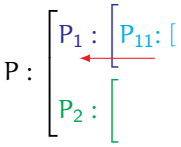
```
def(P) {
  def(x)

  def(P_1) {
    def(P_11) {
      use(x)
    }

    P_11()
  }
```

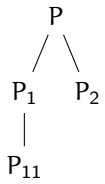
```
11
12    def(P2) {
13      def(x)
14
15      P1()
16    }
17
18    P2()
19  }
```

Nella sequenza di **annidamento statico** (delle definizioni) P_{11} non vede P_2, quindi l'uso di x in P_{11} si riferisce alla definizione di x in P_1.



Lo scoping statico consiste nell'applicare la regola di visibilità usando come relazione di annidamento **quello di definizione**.

- Sequenza di definizione:



  La visibilità di P_{11} è limitata a P_1. Grazie a questo la risoluzione dei riferimenti avviene a tempo di compilazione

- Sequenza di chiamate:

$$P \to P_2 \to P_1 \to P_{11}$$

Esempio 4.25. Consideriamo il seguente esempio:

```
1  {
2    int x = 0;
3    void pippo(int n) {
4      x = n+1; // x non locale, n locale
5    }
6
7    pippo(3); // x = 4
8    write(x); // 4
9    {
10     int x = 0; // <-+
11     pippo(4);  //   -|---> x = 5
12     write(x);  // --+ 0
13   }
```

```
14   write(x); // 5
15 }
```

1. Alla priima esecuzione di `pippo(3)` si modifica `x` globale e si ottiene `x = 4`
2. Alla seconda esecuzione di `pippo(4)` si modifica ancora `x` globale e il valore stampato è quello della `x` locale, quindi `0`
3. L'ultima write stampa il valore di `x` globale, quindi `5`

Definizione 4.12. Lo scoping statico garantisce l'indipendenza del riferimento dalla posizione della chiamata.

Lo scoping statico:

- Rende il codice più comprensibile
- Ma è più difficile da implementare

Esempio 4.26. Alcuni esempi di scoping statico in linguaggi di programmazione comuni sono i seguenti:

1. JavaScript:
   - Esempio 1

```
1  var n = 12;
2
3  function addn() {
4      return n + 30;
5  }
6
7  console.log(n + '\n');
8
9  n = addn();
10
11 console.log(n);
```

   In output si ottiene:

```
1  12
2
3  47
```

   - Esempio 2

```
1  var n = 12;
2
3  function addn() {
4      return n + 30;
5  }
6
```

```
console.log(n + '\n');

var n = 17;
n = addn();

console.log(n);
```

In output si ottiene:

```
12

47
```

In entrambi i casi viene sovrascritta la variabile `n` globale

- Esempio 3

```
var n = 12;

function addn() {
    return n + 30;
}

function setn() {
    var n = 17;
    n = addn();
    console.log(n + '\n');
}

setn();

console.log(n);
```

In output si ottiene:

```
42

12
```

Da questo esempio si nota che JavaScript utilizza scoping statico

2. Python:

- Esempio 1

```
n = 12;

def addn():
        return(n + 30);

def setn():
    n = 17;
    n = addn();
    print(n);

setn();
print(n);
```

In output si ottiene:

```
42

12
```

- Esempio 2

```
n = 12;

def addn():
    return(n + 30);

def setn():
   n = 17;
   n = addn();
   print(n);

n = 13;
setn();
print(n);
```

In output si ottiene:

```
43

13
```

- Esempio 3

```
x = 4;
z = 3;

def f(y):
    return(x * y);

def setn():
    x = 5;
    print(f(z) + x)

setn();
print(x);
```

In output si ottiene:

```
17
4
```

Anche python utilizza scoping statico

4.8.4 Scoping dinamico

Consideriamo il seguente codice:

```
def(P) {
  def(x)

  def(P1) {
    def(P11) {
      use(x)
    }

```

```
    P11()
  }

  def(P2) {
    def(x)

    P1()
  }

  P2()
}
```

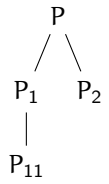
Lo scoping dinamico consiste nell'applicare la regola di visibilità usando come relazione di annidamento quella della **sequenza di chiamate**.

- Sequenza di chiamate:

  $$P \to P_2 \to P_1 \to P_{11}$$

  P_{11} cerca la definizione di x in P_1 e non la trova, quindi cerca in P_2 e la trova, quindi utilizza quella definizione di x a tempo di esecuzione

- Sequenza di definizione:



Il riferimento (legame) corretto **dipende dalla posizione della chiamata**, quindi chiamate diverse possono fare riferimento a legami diversi.

Esempio 4.27. Consideriamo il seguente codice:

```
{
  int x = 0;
  void pippo(int n) {
    x = n + 1;
  }

  pippo(3);
  write(x); // 4

  {
    int x = 0; // <-+<+
    pippo(4);  // --+ |
    write(x);  // 5 --+
  }
  write(x); // 4
}
```

La dichiarazione di x utilizzata da pippo dipende da dove viene chiamata, quindi:

- Alla prima chiamata di `pippo(3)` viene modificata la `x` globale e si ottiene `x = 4`
- Alla seconda chiamata di `pippo(4)` viene modificata la `x` locale e si ottiene `x = 5`
- L'ultima write stampa il valore di `x` globale, quindi `4`

Le cause che rendono due risultati diversi in base allo scoping sono:

- Il codice contiene una procedura con un ambiente non locale
- Esistono diverse definizioni di questi riferimenti non locali nel codice

Esempio 4.28. Alcuni esempi di scoping dinamico in linguaggi di programmazione comuni sono i seguenti:

1. Lisp:
   - Esempio 1

```
(defvar n 12)

(defun addn () (+ n 30))

(defun setn ()
  (let ((n 17))
    (print (addn)))
)

(setn)
(print n)
```

     In output si ottiene:

```
47
12
```

   - Esempio 2

```
(defvar x 4)
(defvar z 3)

(defun f (y) (* x y))

(let ((x 5))
  (print (+ (f z) x)))

(print x)
```

     In output si ottiene:

```
20
4
```

2. Perl:
   - Esempio 1

```
$n = 12;

sub addn() {return $n + 30};

sub setn() {
    $n = 17;
    $n = addn();
    print $n
};

setn();

print ' ';
print $n;
```

In output si ottiene:

```
47 47
```

In questo codice non c'è alcun tipo di scoping perchè le variabili sono tutte globali. Per avere variabili locali bisogna utilizzare la parola chiave `local`.

- Esempio 2

```
$n = 12;

sub addn() {return $n + 30};

sub setn() {
    local $n = 17;
    $n = addn();
    print $n
};

setn();

print ' ';
print $n;
```

In output si ottiene:

```
47 12
```

Se si forza la località delle variabili si ottiene scoping dinamico.

Lo scoping dinamico ha un'implementazione più semplice, ma rende il codice meno leggibile e siccome è eseguito a tempo di esecuzione è più difficile fare analisi statica del codice.

4.9 Allocazione della memoria

Bisogna gestire come e quando viene allocato lo spazio in memoria per gli elementi del programma. Ha effetto sugli algoritmi di risoluzione dei riferimenti non locali. Esistono due tipi principali di allocazione della memoria:

- **Allocazione statica**: la memoria è allocata a tempo di compilazione
- **Allocazione dinamica**: la memoria è allocata a tempo di esecuzione. Utilizza uno stack (LIFO) e lo heap per gestire la memoria

Esempio 4.29. Consideriamo il seguente codice (illegale) di fortran:

```
SUBROUTINE ERROR(N)
        IF (N.LE.1) RETURN
yyy     CALL ERROR(N-1)
        PRINT N
        END
        ...
xxx     CALL ERROR(3)
```

Supponiamolo legale: eseguiamolo nel modello di memoria statica

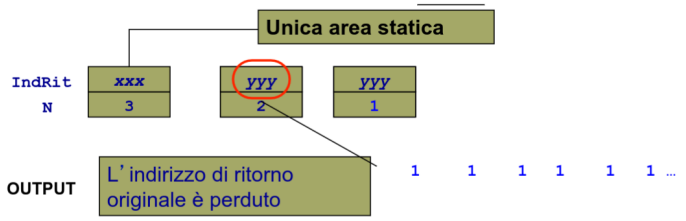


Figura 9: Esempio in fortran

4.9.1 Allocazione dinamica su pila

L'allocazione dinamica su pila associa la memoria ad una struttura dati allocata dinamicamente e gestita in modalità LIFO. Per ogni chiamata a procedura viene creato un **RdA** (Record di Attivazione), cioè un insieme di infomazioni che permettono di eseguire la procedura e permettono di ritornare correttamente allo stato valido al momento della chiamata. L'RdA ha una dimensione nota staticamente e contiene:

- **Indirizzo di ritorno**: rappresenta l'indirizzo a cui ritornare dopo l'esecuzione della procedura
- **Variabili locali**: contiene parametri formali e l'ambiente dichiarato nella procedura
- **Puntatore alla testa della pila**
- **Puntatore statico**: è il puntatore all'RdA della procedura genitore nella catena storica delle definizioni (nello stack può essere distante dall'RdA corrente). Serve per ricostruire la sequenza di annidamento in caso di scoping statico
- **Puntatore dinamico**: è il puntatore all'RdA della procedura chiamante. serve per ricostruire la sequenza di chiamate in caso di scoping dinamico
- Altre informazioni specifiche

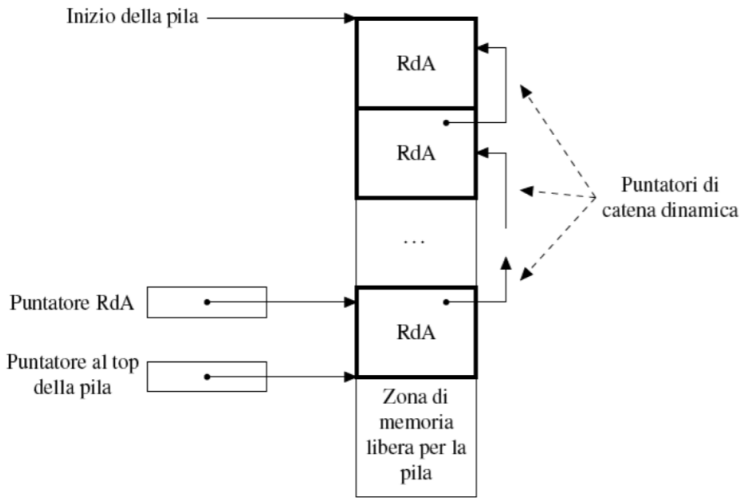


Figura 10: Contenuti dello stack

4.9.2 Procedure di chiamata a funzione

I passi da eseguire per chiamare una funzione sono i seguenti:

1. Valutare i parametri (in funzione del metodo di passaggio)
2. Allocare un RdA sullo stack con un ambiente locale
3. Salvare lo stato del chiamante al momento della chiamata
4. Trasferire il controllo alla procedura chiamata

I passi da eseguire per ritornare da una funzione sono i seguenti:

1. Recupero delle informazioni dal RdA (passaggio valore-risultato)
2. Deallocazione del RdA dallo stack
3. Recupero dello stato del chiamante
4. Ritornare il controllo al chiamante

Esempio 4.30. Consideriamo il seguente codice:

```
1  void fun1(float r) {
2    int s, t;
3    ...
4
5    fun2(s);
6
7    ...
8  }
9
10 void fun2(int x) {
11   int y;
12   ...
```

```
13
14    fun3(y);
15
16    ...
17  }
18
19  void fun3(int q) {
20    ...
21  }
22
23  void main() {
24    float p;
25    ...
26
27    fun1(p);
28
29    ...
30  }
```

- main chiama fun1
- fun1 chiama fun2
- fun2 chiama fun3

E l'evoluzione dello stack per questo programma è la seguente:

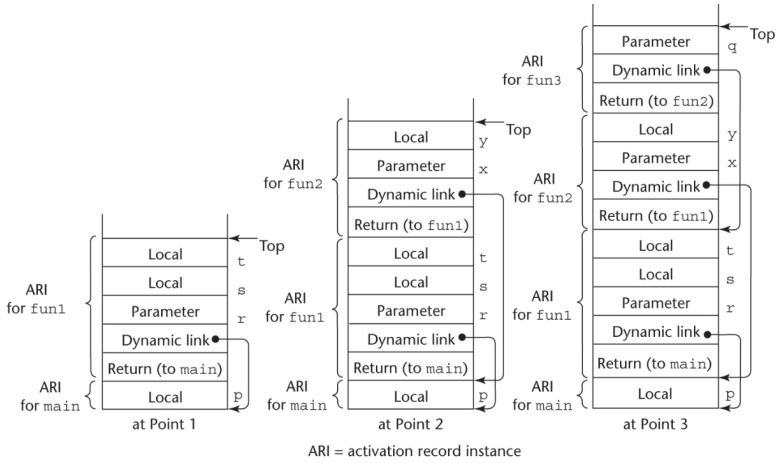


Figura 11: Evoluzione dello stack

Esempio 4.31. Consideriamo il seguente codice:

```
1  int factorial (int n) {
2    // <---------------------------- 1
3    if (n <= 1) return 1;
4    else return (n * factorial(n - 1));
5    // <---------------------------- 2
6  }
```

```
7
8  void main() {
9    int value;
10   value = factorial(3);
11   // <---------------------------- 3
12 }
```

- Appena viene chiamata `factorial(3)` viene creato un RdA per ogni chiamata a `factorial`, quindi si ottiene il seguente stack:

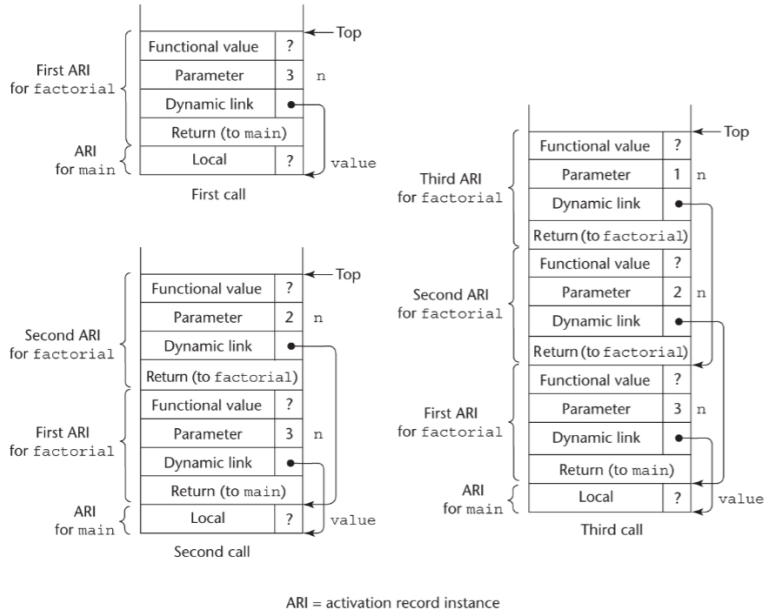


Figura 12: Evoluzione dello stack con ricorsione

- Il valore della funzione viene calcolato partendo da quello più in alto sullo stack e poi viene passato al RdA sottostante, quindi si ottiene il seguente stack:

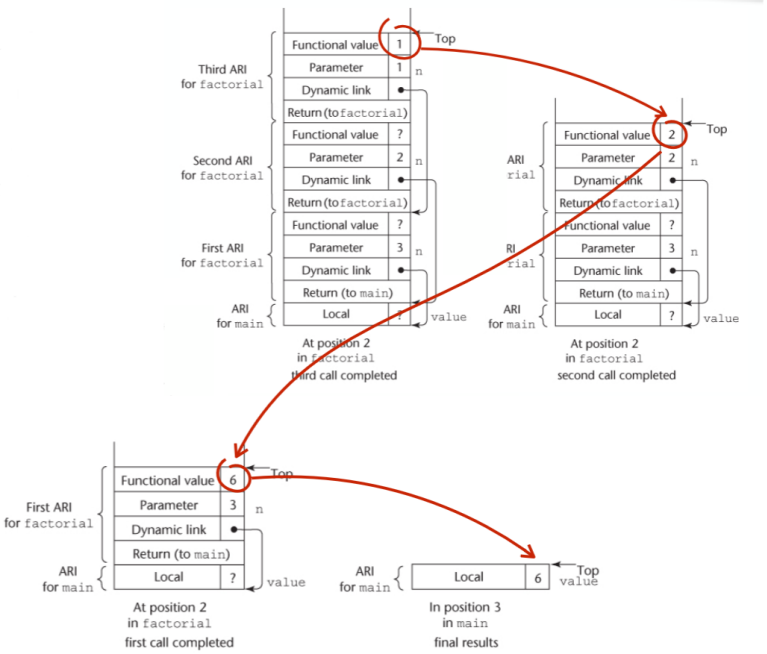


Figura 13: Evoluzione dello stack con ricorsione (calcolo del valore)

4.9.3 Implementazione scoping statico

Lo scoping statico consiste nella costruzione del **link statico** alla chiamata di ogni procedura. Il link statico punta all'indirizzo del RdA della procedura che contiene la definizione della procedura chiamata in atto.

- **Catena statica** (CS): è la sequenza di RdA che si ottiene seguendo i link statici a partire da un RdA
- **Profondità statica** (SD): è il valore che denota la profondità di annidamento.

Esempio 4.32. Consideriamo i seguenti nomi di procedura:

$$A, B, C, D, E$$

Con la seguente sequenza di chiamate:

$$A \underbrace{\rightarrow}_{\text{Determina i link dinamici}} B \rightarrow C \rightarrow D \rightarrow E \rightarrow C$$

I link dinamici si ottengono seguendo al contrario la sequenza di chiamate. Supponiamo che il codice sia definito nel seguente modo:

- B e C sono definiti dentro A
- D e E sono definiti dentro C

$$A : \begin{bmatrix} B : [\\ \\ C : \begin{bmatrix} D : [\\ \\ E : [\end{bmatrix} \end{bmatrix}$$

La profondità statica è la seguente:

- $SD(A) = 0$
- $SD(B) = 1 + SD(A) = 1 = SD(C)$
- $SD(D) = 1 + SD(C) = 2 = SD(E)$

Lo stack è composto da:

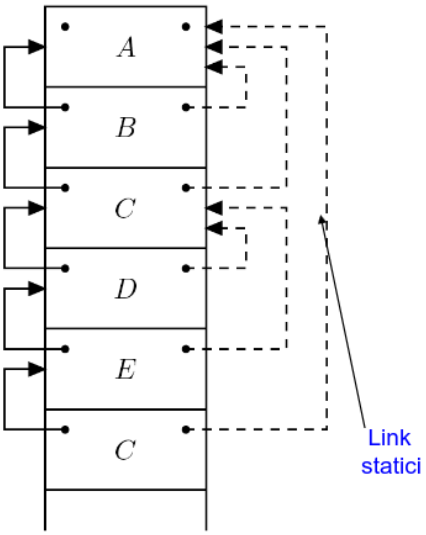


Figura 14: Stack con scoping statico

Consideriamo la seguente notazione:

- Ch: Rappresenta il chiamante

Le informazioni che abbiamo per calcolare il link statico sono:

- Profondità di annidamento (SD)
- RdA della procedura (con struttura nota staticamente)

Supponiamo che Ch chiama P, per determinare il link statico di P:

- Se P è definito in Ch allora il link statico di P coincide con il link dinamico
- Se P è definito in un blocco che si trova k passi prima di Ch allora bisogna risalire la catena statica di k passi. Dove

$$k = SD(Ck) - SD(P) + 1$$

Ciò implica che:

- Se $k = 0$ allora Ch definisce P e quindi $CS(P) = Ch$. Di conseguenza il link statico per P è l'indirizzo del chiamante.
- Se $k > 0$ allora $CS(P) = \overbrace{CS(\ldots CS(Ch))}^{k \text{ volte}}$, cioè il link statico si ottiene risalendo k volte la catena statica (la sequenza di link statici)

Esempio 4.33. Consideriamo lo stesso codice dell'esempio precedente:

1. Calcoliamo il link statico per B:
   Calcoliamo k:

   $$k_b = SD(A) - SD(B) + 1 = 0 - 1 + 1 = 0$$

   $$\Downarrow$$

   $$CS(B) = A$$

   Quindi il link statico di B è l'indirizzo del chiamante A.

2. Calcoliamo il link statico per C:

   $$k_c = SD(B) - SD(C) + 1 = 1 - 1 + 1 = 1$$

   $$\Downarrow$$

   $$CS(C) = CS(B) = A$$

   Quindi il link statico di C è l'indirizzo del link statico di B, che coincide con A.

3. Calcoliamo il link statico per D:

   $$k_d = DS(C) - SD(D) + 1 = 1 - 2 + 1 = 0$$

   $$\Downarrow$$

   $$CS(D) = C$$

   Quindi il link statico di D è l'indirizzo del chiamante C.

4. Calcoliamo il link statico per E:

   $$k_e = SD(D) - SD(E) + 1 = 2 - 2 + 1 = 1$$

   $$\Downarrow$$

   $$CS(E) = CS(D) = C$$

   Quindi il link statico di E è l'indirizzo del link statico di D, che coincide con C.

5. Calcoliamo il link statico per C chiamato da E:

$$k_{c_2} = SD(E) - SD(C) + 1 = 2 - 1 + 1 = 2$$

$$\Downarrow$$

$$CS(C) = CS(CS(E)) = CS(CS(D)) = CS(C) = CS(B) = A$$

Risoluzione dei riferimenti

Nota: se il riferimento è locale allora la risoluzione è ovvia nell'RdA attivo.

Se si ha x usato in P ma non locale, bisogna individuare il blocco più vicino nella catena statica che definisce x. Per farlo:

- Si utilizza SD e l'informazione data dai blocchi che definiscono x
- Si utilizza la catena statica

Esempio 4.34. Consideriamo il seguente codice:

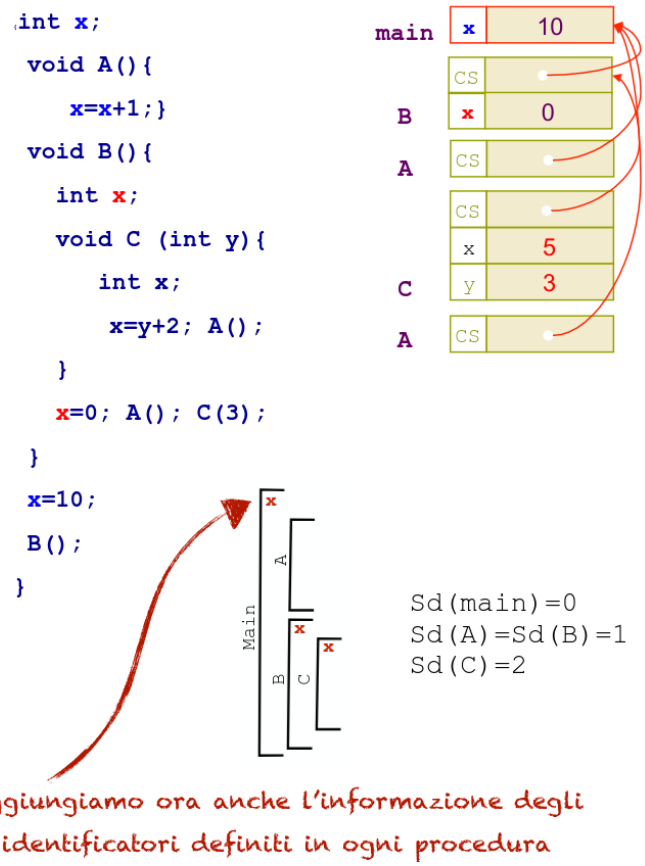


Figura 15: Esempio di risoluzione dei riferimenti con scoping statico

Usiamo x in A, quindi cerchiamo nella catena di annidamento, risalendo da A, il blocco più vicino che definisce x. Il blocco più vicino ad A che definisce x è il $main$ e lo sappiamo a tempo di compilazione.

$$\overbrace{N_x}^{\text{Quanto bisogna risalire la catena statica per trovare la corretta x}} = \underbrace{SD(A)}_{\text{Blocco che usa x}} - \underbrace{SD(main)}_{\text{Blocco più vicino ad A che definisce x nella catena di annidamento}}$$

4.9.4 Implementazione scoping dinamico

Per implementare lo scoping dinamico si utilizza l'algoritmo **shallow access** basato su **CRT** (Tabella Centrale dei Riferimenti) che utilizza strutture di dati centralizzate.

Tabella centrale dei riferimenti

La tabella centrale dei riferimenti ha come obiettivo quello di evitare ricerche lineari mantenendo in modo **centralizzato** le informazioni necessarie per accedere al RdA dinamico corretto. La tabella contiene una riga per ogni identificatore usato nel programma e ogni **identificatore punta ad una lista** che ha in testa le informazioni dell'**ultimo RdA (dinamico) che definisce l'identificatore**.

Esempio 4.35. Consideriamo la seguente sequenza di chiamate:

$$A \to B \to C \to D$$

Con il seguente schema statico di annidamento delle definizioni (scoping statico):

$$A : \left[\begin{array}{l} x, y \\ \quad B : \left[x, v \right. \\ \quad C : \left[w, x \quad D : \left[w \right. \right. \end{array} \right.$$

La tabella centrale dei riferimenti è la seguente:

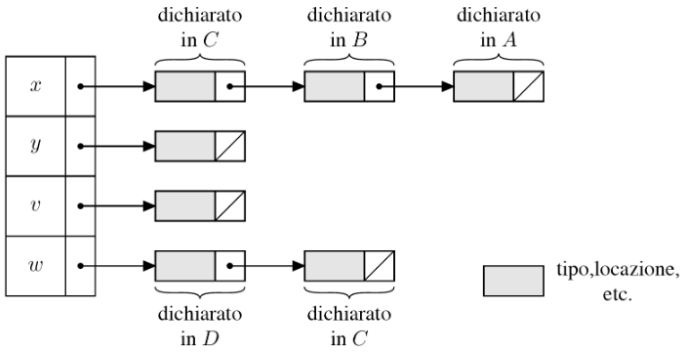


Figura 16: Esempio di tabella centrale dei riferimenti

1. Alla chiamata di A viene creato l'RdA per A sullo stack e viene inserito l'indirizzo dell'RdA in testa alla lista di x. Lo stesso vale per y.
2. Alla chiamata di B y rimane inalterata e x viene ridefinita e quindi viene inserito l'indirizzo dell'RdA di B in testa alla lista di x. Lo stesso vale per v.
3. Alla chiamata di C x viene ridefinita e quindi viene inserito l'indirizzo dell'RdA di C in testa alla lista di x. Lo stesso vale per w.
4. Alla chiamata di D w viene ridefinita e quindi viene inserito l'indirizzo dell'RdA di D in testa alla lista di w.

All'uscita da un blocco viene distrutto l'RdA e viene eliminata la testa di ogni lista linkata da un identificatore definito in quel blocco.

4.10 Implementazione delle procedure senza parametri

4.10.1 Dichiarazione di procedure

Le procedure canno definite, quindi bisogna modificare le dichiarazioni:

$$\begin{aligned}\mathcal{D} : \mathrm{D} \to\ & \texttt{nil} \\ & \mid \texttt{const x: } \tau = e \\ & \mid \texttt{var x: } \tau = e \\ & \mid \texttt{proc P() C} \\ & \mid \mathrm{D}; \mathrm{D} \mid \mathrm{D}\ \texttt{in}\ \mathrm{D} \\ & \mid \rho\end{aligned}$$

dove:

- P è un identificatore per la procedura senza parametri
- C è il corpo, cioè una sequenza di comandi

Aggiungiamo agli oggetti denotabili le **astrazioni**:

$$\lambda\varepsilon\ .\ \mathrm{C}$$

cioè una funzione (λ) con una lista vuota di parametri (ε) e un corpo C.

Definiamo la regola per la dichiarazione di una procedura:

$$\rho \vdash \langle \texttt{proc P() C}, \sigma\rangle \to \langle [\mathrm{P} \mapsto \lambda\varepsilon\ .\ \mathrm{C}'], \sigma\rangle$$

L'associazione di P al suo codice è costante, quindi non si ha nessuna locazione. C' è definito in modo diverso in base allo scoping:

- Se si ha uno scoping statico si fissa l'ambiente presente al momento della definizione di P per averlo alla sua esecuzione:

$$\mathrm{C}' = \rho; \mathrm{C}$$

- Se si ha uno scoping statico si usa l'ambiente alla chiamata di P:

$$\mathrm{C}' = \mathrm{C}$$

4.10.2 Chiamata di procedure

Le procedure vanno chiamate, quindi bisogna modificare i comandi:

$$\begin{aligned}\mathrm{Com} : \mathrm{C} \to\ & \texttt{skip} \mid \mathrm{x} := e \\ & \mid \mathrm{C}; \mathrm{C} \\ & \mid \texttt{while B do C} \\ & \mid \texttt{if B then C else C} \\ & \mid \texttt{P()}\end{aligned}$$

Definiamo la regola per la chiamata di una procedura:

$$\overline{\rho} \vdash \langle \mathrm{P}(), \sigma\rangle \to \langle \mathrm{C}', \sigma\rangle \quad \overline{\rho}(\mathrm{P}) = \lambda\varepsilon\ .\ \mathrm{C}'$$

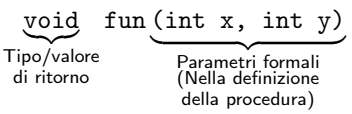
con una definizione di `proc P() C` nell'ambiente ρ e

$$C' = \begin{cases} \rho; C & \text{se si ha scoping statico} \\ C & \text{se si ha scoping dinamico} \end{cases}$$

Nel caso di scoping statico si avrà $\overline{\rho}[\rho]$

4.11 Implementazione delle procedure con passaggio di parametri

Le procedure senza parametri sono semplicemente zucchero sintattico, si vorrebbe però istanziare un blocco di codice in base a input diversi. I **parametri** sono quindi l'input della procedura che permettono una comunicazione tra chiamante e chiamato. La sintassi della funzione è la seguente:



Quando si chiama una procedura, i parametri in input sono detti parametri attuali:

fun(2, 5)

Ci sono diversi tipi di comunicazione tra chiamante e chiamato:

- Passaggio per valore: il chiamante passa al chiamato una copia del valore del parametro
- Passaggio per riferimento: il chiamante passa al chiamato un riferimento alla variabile del parametro, quindi il chiamato può modificare il valore del parametro nel chiamante
- Passaggio per risultato: il chiamante passa al chiamato una locazione di memoria, quindi il chiamato può modificare il valore del parametro nel chiamante, ma non può leggere il valore iniziale del parametro
- Passaggio per valore-risultato: combina il passaggio per valore e per risultato

Gli aspetti implementativi da considerare per il passaggio di parametri sono i seguenti:

- Modalità di passaggio (per valore, per riferimento, per risultato, per valore-risultato)
- Come avvinee l'associazione tra parametri attuali e parametri formali (tipicamente è posizionale)

4.11.1 Direzione del passaggio dei parametri

Ci sono diverse direzioni per il passaggio dei parametri:

- **In-mode**: chiamante $\rightarrow$ chiamato. Viene generata una comunicazione in un unica direzione che va dal chiamante al chiamato. I parametri vengono copiati e inviati al chiamato. La modalità di passaggio dei parametri in-mode è il **passaggio per valore**
- **Out-mode**: chiamante $\leftarrow$ chiamato. I valori vengono inviati dal chiamato al chiamante tramite return, oppure tramite i parametri. La modalità di passaggio dei parametri out-mode è il **passaggio per risultato**
- **In-out-mode**: chiamante $\leftrightarrow$ chiamato. È una combinazione di in-mode e out-mode e fornisce una doppia comunicazione. Le modalità di passaggio dei parametri in-out-mode sono il **passaggio per valore-risultato** e il **passaggio per riferimento**.

Queste direzioni non sono mutualmente esclusive. Anche l'ambiente globale è un canale di comunicazione che può essere usato in modo bidirezionale.

4.11.2 Passaggio per valore

Il valore del parametro attuale (eventualmente ottenuto come valutazione di un'espressione) viene passato come valore iniziale del corrispondente parametro formale. Viene implementato creando una **copia** del valore del parametro attuale nel formale. Ad esempio:

$$\texttt{fun(}\underbrace{\texttt{x+1}}_{\substack{\text{Il parametro attuale}\\ \text{è un r-value}}}\texttt{, y+2)}$$

Il passaggio per valore permette una certa **versabilità nella scritturaa dei parametri** attuali, ma lo svantaggio principale è l'**utilizzo della memoria** perchè ogni chiamata crea una copia dei parametri attuali.

Considerando il seguente codice:

```
void foo(value int x) {
  x = x + 1;
}

y = 1;
foo(y + 1);
```

Si ha che:

1. La chiamata a `foo` alloca `x` nell'ambiente locale di `foo`.
2. Alla chiamata `y+1` viene valutato, in questo caso a 2, e il valore viene **copiato** nella cella riferita da `x`. Non c'è **nessun legame** tra `x` e `y`.
3. Alla terminazione di `foo` il RdA contenente `x` viene distrutto, perdendo tutte le modifiche fatte a `x`. Questa metodologia **non** permette di comunicare al hciamante attraverso i parametri.

Esempio 4.36. Consideriamo il seguente codice, con i parameteri passati per valore:

```
void swap(value int a, value int b) {
  int tmp = a;
```

```
3   a = b;
4   b = tmp;
5 }
6
7 int c = 3;
8 int d = 4;
9 swap(c, d);
```

Si ha che:

1. Nell'esecuzione del main si ha che $[c \mapsto 3]$ e $[d \mapsto 4]$ in memoria
2. Alla chiamata di swap vengono create le celle in memoria per i parametri:

$$[a \mapsto \varnothing] \quad e \quad [b \mapsto \varnothing] \quad e \quad [tmp \mapsto \varnothing]$$

3. All'esecuzione di `swap` si ha che il valore di c viene copiato in a e il valore di d viene copiato in b, quindi:

$$[a \mapsto 3] \quad e \quad [b \mapsto 4] \quad e \quad [tmp \mapsto \varnothing]$$

4. Viene eseguito il corpo di `swap` e si scambiano i valori di a e b, quindi:

$$[a \mapsto 4] \quad e \quad [b \mapsto 3] \quad e \quad [tmp \mapsto 3]$$

5. Alla terminazione di `swap` viene eliminato l'RdA di `swap`, quindi tutte le trasformazioni fatte vengono perse e rimane solo l'RdA del `main`:

$$[c \mapsto 3] \quad e \quad [d \mapsto 4]$$

Esempio 4.37. Consideriamo il seguente codice JavaScript:

```
1 var a = 2;
2 var b = 3;
3
4 function double(x, y) {
5   x = 2 * x;
6   y = 2 * y;
7
8   console.log(x + ' ' + y); // 4 6
9 }
10
11 console.log(double(a,b)); // undefined
12
13 console.log(a + ' ' + b) // 2 3
```

Si osserva che i parametri sono stati passati per valore perchè a e b sono rimasti invariati.

4.11.3 Passaggio per riferimento

Viene passato un cammino di accesso al parametro formale (modalità In-Out). Il vantaggio è un uso efficiente della memoria, ma lo svantaggio è che vengono creati

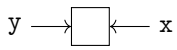
degli alias, cioè si hanno più nomi per la stessa cella di memoria che può avere degli effetti collaterali indesiderati.

Considerando il seguente codice:

```
void foo(ref int x) {
  x = x + 1;
}

y = 1;
foo(y);
```

1. In questo caso i parametri attuali possono essere **solo l-value**, quindi riferimenti, ovvero oggetti che possono ricevere valori e non generarli.
2. Alla chiamata di `foo` si crea un RdA con una cella per `x`
3. All'esecuzione di `foo` `x` diventa un riferimento alla cella riferita da `y`, quindi:



4. Alla terminazione di `foo` viene distrutto l'RdA di `foo` contenente il riferimento `x`, ma tutte le trasformazioni fatte alla cella di memoria che era riferita da `x` rimangono riferibili da `y`.

Esempio 4.38. Consideriamo il seguente codice con parametri passati per riferimento:

```
void swap(ref int a, ref int b) {
  int tmp = a;
  a = b;
  b = tmp;
}

int c = 3;
int d = 4;
swap(c, d);
```

Si ha che:

1. All'esecuzione del `main` si creano le celle di memoria per `c` e `d`:

$$[\mathtt{c} \mapsto 3] \quad \text{e} \quad [\mathtt{d} \mapsto 4]$$

2. Alla chiamata di `swap` si crea l'RdA per `swap` con le celle per i parametri `a` e `b` che diventano riferimenti alle celle di `c` e `d`, quindi:

$$[\mathtt{c} \to 3 \leftarrow \mathtt{a}] \quad \text{e} \quad [\mathtt{d} \to 4 \leftarrow \mathtt{b}] \quad \text{e} \quad [\mathtt{tmp} \mapsto \varnothing]$$

3. All'esecuzione di `swap` si scambiano i valori di `a` e `b`, quindi:

$$[\mathtt{c} \to 4 \leftarrow \mathtt{a}] \quad \text{e} \quad [\mathtt{d} \to 3 \leftarrow \mathtt{b}] \quad \text{e} \quad [\mathtt{tmp} \mapsto 3]$$

4. Alla terminazione di `swap` viene eliminato l'RdA di `swap`, ma tutte le trasformazioni fatte alle celle di memoria che erano riferite da `a` e `b` rimangono riferibili da `c` e `d`, quindi:

$$[c \mapsto 4] \quad \text{e} \quad [d \mapsto 3]$$

Esempio 4.39. Consideriamo il seguente codice JavaScript:

```
var a = {val: 2};
var b = {val: 3};

function double(x, y) {
  x.val = 2 * x.val;
  y.val = 2 * y.val;

  console.log(x.val + ' ' + y.val) // 4 6
}

console.log(double(a,b)); // undefined
console.log(a.val + ' ' + b.val) // 4 6
```

Si osserva che i parametri sono stati passati per riferimento perchè `a` e `b` sono stati modificati.

4.12 Funzioni di ordine superiore

Nel caso di **funzioni di ordine superiore**, cioè in cui una funzione può essere passata come argomento ad un altra funzione, le regole di visibilità e di scoping possono non essere più sufficienti. Il problema principale è la risoluzione dei riferimenti **non locali** in procedure/funzioni passate come parametro ad altre funzioni. In particolare nel caso dinamico sono necessarie anche delle **regole di binding**.

4.12.1 Regole di binding

Per le procedure con ambiente **non locale**, in base allo scoping le **regole di binding** sono:

- **Dinamico**:
  - **Deep binding**: Per le procedure passate per parametro, l'ambiente valido è quello al momento del passaggio della funzione, cioè il momento della creazione del legame con il parametro formale.
  - **Shallow binding**: Per le procedure passate per parametro, l'ambiente valido è quello al momento dell'esecuzione della procedura mediante il parametro formale.
- **Statico**: Per tutte le procedure, anche quelle passate per parametro, l'ambiente valido è quello al momento della definizione della procedura.

Esempio 4.40. Consideriamo il seguente codice:

```
int x = 2;
int z = 0;

int f(int y) {
  // x non locale
  return x * y;
}

int g(int h(int n)) {
  int x = 7;
  z = h(3) + x;
}

{
  int x = 5;
  g(f);
}
```

Si ha una procedura (`f`) con ambiente non locale, quindi c'è bisogno di regole di scoping, ed eventualmente di regole di binding. Inoltre la funzione `f` con ambiente non locale viene passata come parametro ad un altra funzione, quindi lo scoping può non essere sufficiente e bisogna introdurre le regole di binding.

Le variabili non locali per `g` sono `z` e `x`, per `x` bisogna usare le regole di scoping e di binding, mentre per `z` basta usare le regole di scoping.

- L'ambiente con scoping statico è: `x = 4`
  1. Si esegue la chiamata di `g(f)`
     (a) `z = h(3) + x`
     (b) `x` è locale a `g` e vale `x = 7`
        i. Si esegue la chiamata di `h`
        ii. `x * 3 = 4 * 3 = 12`
     (c) `z = 12 + 7 = 19`
- L'ambiente dinamico con deep binding è: `x = 5`
  1. Si esegue la chiamata di `g(f)`
     (a) `z = h(3) + x`
     (b) `x` è locale a `g` e vale `x = 7`
        i. Si esegue la chiamata di `h`
        ii. `x` non è locale e si usa lo scoping dinamico con deep binding, quindi si prende l'ambiente al momento del passaggio di `f` a `g`, che è `x = 5`
        iii. `x * 3 = 5 * 3 = 15`
     (c) `z = 15 + 7 = 22`
- L'ambiente dinamico con shallow binding è: `x = 7`
  1. Si esegue la chiamata di `g(f)`

(a) `z = h(3) + x`

(b) `x` è locale a `g` e vale `x = 7`

i. Si esegue la chiamata di `h`

ii. `x` non è locale e si usa lo scoping dinamico con shallow binding, quindi si prende l'ambiente al momento dell'esecuzione di `h`, che è `x = 7`

iii. `x * 3 = 7 * 3 = 21`

(c) `z = 21 + 7 = 28`

Esempio 4.41. Alcuni esempi in linguaggi comuni:

- **JavaScript**:

```
x = 4;

function f(y) {
  return x * y;
}

function g(h) {
  var x = 7;
  return h(3) + x;
}

function callmap() {
  var x = 5;
  return g(f);
}

console.log(callmap()); // 19
```

In questo caso si ha scoping statico, quindi x vale 4.

- **Lisp**:

```
(defvar x 4)
(defun f (y) (* x y))

(defun g (h)
  (let ((x 7))
    (+ x (funcall h 3))
  )
)

(let ((x 5))
  (print (g 'f)) ; 28
)
```

In questo caso si ha scoping dinamico con shallow binding, quindi x vale 7.

- **Perl**:

```
$x = 4;

```

```
my $f = sub {
  my ($y) = @_;
  $y * $x;
};

my $g = sub {
  my $x = 7;
  my ($h) = @_;
  sub {
    $h->(@_) + $x;
  };
};

sub callmap {
  local $x = 5;
  my $call = $g->($f);
  print $call->(3);
};

callmap; // 22
```

In questo caso si ha scoping dinamico con deep binding, quindi x vale 5.

- **Python**:

```
x = 4

fef f(y):
    return x * y

def g(h):
    x = 7
    return result(n):
        return h(n) + x

def callmap():
    x = 5
    call = g(f)
    print(call(x))

callmap() # 27
```

In questo caso si ha scoping statico, quindi x vale 4.

4.13 Ricorsione

La ricorsione è un metodo alternativo alla costruzione di procedure iterative.

Definizione 4.13. Una procedura è **ricorsiva** se è definita in funzione di se stessa, cioè se all'interno della procedura esiste una chiamata alla procedura stessa.

Esempio 4.42. Un esempio di procedura ricorsiva è la funzione fattoriale:

$$\mathtt{fact}(n) = \begin{cases} n & \text{se } n \leqslant 1 \\ n \cdot \mathtt{fact}(n-1) & \text{altrimenti} \end{cases}$$

```
int fattoriale(int n) {
  if (n <= 1) {
    return n;
  } else {
    return n * fattoriale(n - 1);
  }
}
```

Le funzioni ricorsive terminanti devono essere ben fondate, cioè devono avere una condizione di terminazione, altrimenti si ha una ricorsione infinita.

Esempio 4.43. Le seguenti funzioni non sono ben fondate:

```
int fun1(int n) {
  if (n == 1) {
    return fun1(n);
  }
}

int fun2(int n) {
  if (n == 0) {
    return 1;
  } else {
    return fun2(n) + 1
  }
}
```

4.13.1 Ricorsione vs iterazione

- L'iterazione è sempre possibile in qualunque linguaggio di programmazione turing completo.
- La ricorsione è possibile se le procedure possono essere definite in termini di se stesse. Ci sono due tipi di ricorsione:
  - Tramite allocazione dinamica
  - Ricorsione in coda

Teorema 4.1. Ogni programma iterativo può essere riscritto in modo ricorsivo e viceversa ogni programma iterativo può essere riscritto in modo ricorsivo.

Esempio 4.44. Consideriamo il codice del fattoriale:

```
int fact(int n) {
  if (n <= 1) {
    return n;
  } else {
    return n * fact(n - 1);
  }
}
```

La chiamata di `fact(3)` genera la seguente sequenza di chiamate:

1. Lo stack al primo passo è:

4.13.2 Ricorsione in coda

Consiste nell'aumentare il numero di parametri (solitamente uno) in cui si memorizza il calcolo temporaneo. Con questo metodo, la chiamata ricorsiva è l'ultima operazione eseguita dalla procedura.

Definizione 4.14. Siano f e g due funzioni, dove f chiama g.

- La chiamata di g è detta in coda se f restituisce il valore restituito da g senza ulteriori computazioni.
- f è ricorsiva in coda se la chiamata ricorsiva a se stessa è in coda

Esempio 4.45. La funzione fattoriale non è ricorsiva in coda perchè dopo la chiamata ricorsiva viene moltiplicato il risultato per n.

Esempio 4.46. Scriviamo il fattoriale con ricorsione in coda:

```
int fact(int n) {
  // il secondo parametro e' il valore iniziale per res
  // che indicativamente e' il risultato del caso base
  fact-rc(n, 1);
}

int fact-rc(int n, int res) {
  if (n == 1) { // questo indica il valore del risultato
    iniziale
    return res;
  } else {
    return fact-rc(n - 1, n * res);
  }
}
```

L'esecuzione di `fact(3)` genera la seguente sequenza di chiamate:

1. `fact(3)` chiama `fact-rc(3, 1)`
2. `fact-rc(3, 1)` chiama `fact-rc(2, 3)`

3. `fact-rc(2, 3)` chiama `fact-rc(1, 6)`

4. `fact-rc(1, 6)` siccome `n = 1` restituisce `res`, cioè 6

Di fatto si inverte l'ordine di esecuzione rispetto alla versione non ricorsiva in coda.

Esempio 4.47. Un altro esempio di ricorsione è la funzione di Fibonacci:

$$\begin{cases} \mathrm{fib}(0) = 1 \\ \mathrm{fib}(1) = 1 \\ \mathrm{fib}(n) = \mathrm{fib}(n-1) + \mathrm{fib}(n-2) \quad \text{se } n \geqslant 2 \end{cases}$$

```
int fib(int n) {
  if (n <= 1) {
    return n;
  } else {
    return fib(n - 1) + fib(n - 2);
  }
}
```

Questa funzione ha complessità esponenziale sia in tempo che in spazio. La ricorsione in coda ottimizza sia la complessità in spazio che in tempo. Il problema principale è che ad ogni chiamata ricorsiva si fa lo stesso calcolo più volte. La funzione in ricorsione in coda è la seguente:

```
int fib(int n) {
  fib_rc(n, 1, 1);
}

// res1 e' il risultato al passo precedente
// res2 e' il risultato calcolato
int fib_rc(int n, int res1, int res2) {
  if (n <= 1) {
    return res2;
  } else {
    return fib_rc(n - 1, res2, res1 + res2);
  }
}
```